

StahlTrace · The Architecture · v4.0

An ex-ante learning engine for persistent entities, with belief-level posterior updating

Authored for: Ronald Moesker · CEO/CTO · COO (open) · Founding Technical Architect (open) **Scope:** Ex-ante learning engine, agent architecture, data substrate, epistemic discipline, belief-level Bayesian updating, launch path **Supersedes:** v3.1 in full. v4.0 is a self-contained specification: it carries the agent architecture, substrate, and epistemic discipline forward from v3.1, and adds the belief journey as a second epistemic spine, five new tables, scoring extended to belief-update quality, posterior separation, price as Bayesian evidence, and sequential section numbering. No part of v3.1 is left as a deferred reference; this document is the canonical reading. **Status:** Confidential · draft · pre-operating

Notice

N/01 · Confidentiality. For the named recipients only. This document is confidential and is shared with the named recipients on the cover for internal review and early-stage diligence. It is not to be forwarded, quoted, excerpted, or published in whole or in part without the prior written consent of Ronald Moesker. Recipients are asked to treat the contents as they would any non-public technical architecture under an implied non-disclosure obligation.

N/02 · Pre-operating status. StahlTrace is not yet a full operating company. StahlTrace Lda is incorporated in Portugal but is not yet a full operating company. It has no commercial customers, no public product, and no contractual engagements with third parties as of the date of this document. The architecture described here is a design in active development, not a running system. Capabilities, timelines, and commercial structure described herein reflect current intent and are subject to change as the company forms.

N/03 · Intellectual property. © Ronald Moesker / StahlTrace Lda. All architectural concepts, naming (including StahlTrace, Threadweave, the six-layer stack terminology, the belief journey, and the update-case corpus), diagrams, schemas, and written material in this document are the intellectual property of Ronald Moesker and StahlTrace Lda. No licence, express or implied, is granted to recipients by receipt of this document. Reproduction, adaptation, or derivative work requires prior written consent.

N/04 · Forward-looking content. Designs are not commitments. Sections describing the launch path, the sealed client codification layer, the commercial tier structure, the v4.0 phase plan, and any named capabilities of a future system are forward-looking. They describe the design the company intends to build. They are not representations, warranties, or contractual commitments. Any recipient making a decision that depends on those plans should seek direct confirmation from Ronald Moesker rather than relying on this document.

N/05 · No offer, no solicitation. This document is a technical architecture, not an offering memorandum. It does not constitute an offer to sell, or a solicitation of an offer to buy, any security or interest in StahlTrace Lda, and it contains none of the disclosures that would be required for such purposes. It is shared for technical and strategic review only.

N/06 · Draft status. v4.0 canonical, still a draft. This is the canonical v4.0 of the architecture document, the authoritative reference for internal design conversations, and it is nonetheless a draft. Open questions are flagged in §25. Sections marked **new in v4** have been added since v3.1; sections marked **amended in v4** have been substantively rewritten. Future revisions will be labelled v4.1, v4.2, and so on, or v5.0 if the substrate changes materially again.

Questions, corrections, or requests for additional context should be directed to Ronald Moesker. Comments on specific sections are welcome and will be incorporated into the next revision.

CONTENTS

a reading path from premise to shipping plan

PART I · THE PREMISE

01	The product, in one page	AMENDED	§ 01
02	First principles, the four non-negotiables		§ 02
03	The epistemic core, fog of war not narrative fallacy	AMENDED	§ 03
04	The five epistemic rules	AMENDED	§ 04
05	The delta journey, the operational spine		§ 05
06	The belief journey, the epistemic spine	NEW	§ 06

PART II · THE AGENT LAYER

07	The six-layer learning stack		§ 07
08	Deep Insight Agents, qualitative understanding of the entity		§ 08
09	Financial Model Agents, the numbers test		§ 09
10	Synthetic Futures Agents, reachable states not just TAM		§ 10
11	CrossValidator, formal contradiction detection	AMENDED	§ 11
12	Dot Connector / Insight Engine, cross-layer synthesis	AMENDED	§ 12
13	Threadweave, the path engine projected through intent		§ 13
14	Support agents, Resolver Orchestrator Forecaster Evaluator		§ 14
15	Scoring methodology, what "scored by reality" actually means	AMENDED	§ 15
16	Identity resolution, anchoring claims to canonical entities		§ 16
17	The runtime, how the agents actually execute		§ 17
18	Security model, authentication authorization encryption response		§ 18

PART III · THE DATA SUBSTRATE

19	Data acquisition, what feeds the substrate	§ 19
20	Data substrate, eleven stores one ID space AMENDED	§ 20
21	Schemas, the tables that carry the moat AMENDED	§ 21
22	Access matrix, who reads and writes what AMENDED	§ 22

PART IV · THE OPERATION

23	The loop, a worked example end to end AMENDED	§ 23
24	Launch scope, what to build AMENDED	§ 24
25	Open questions, what still needs decisions AMENDED	§ 25
26	Operational concerns deliberately out of scope	§ 26

PART V · THE SEALED CLIENT LAYER

27	The sealed client workbench, additive tenant-isolated	§ 27
28	The workbench, what customers can build	§ 28
29	The seal, what never crosses the boundary AMENDED	§ 29
30	Delivery and commercial shape	§ 30
31	Metering, billing, and customer accounts	§ 31
32	API and access surface	§ 32
33	The Delta Feed AMENDED	§ 33

PART VI · R&D ROADMAP

34	Framing, what this part is and is not	§ 34
35	The corpus, belief-update trajectories paired with reality AMENDED	§ 35
36	Prompt evolution, calibration applied to the prompts not just the outputs	§ 36
37	Coverage economics	§ 37
38	Public Actor Trace, the Worldview & Game Layer	§ 38
39	Preconditions	§ 39

PART VII · THE BAYESIAN BELIEF-UPDATE LAYER

40	Price as Bayesian evidence NEW	§ 40
41	Posterior separation and the falsifier ladder NEW	§ 41

Part I • The premise

What we are actually building, and why its shape is the shape it is. Six sections. The product, the principles, the epistemic core, the five rules, the delta journey, and the belief journey. Read these before anything else. The rest of the document is consequence.

§01 The product, in one page

StahlTrace is an ex-ante learning engine for persistent entities. It automates the construction, challenge, and scoring of company models under uncertainty, so that every change of mind is preserved and every forecast is fixed before the outcome.

Entities are not event streams. They are path-dependent systems with structure, memory, incentives, vulnerabilities, and trajectories. For each tracked entity the system maintains a living baseline (what it is, how it works, what drives it, what constrains it, what makes it resilient, what makes it fragile, what path it appears to be on) and a living **belief vector** (the named, addressable claims that compose the thesis on the entity, each with a prior confidence, a primary falsifier, and a rival explanation set). New information is not presented as an event. It is evaluated against that entity's own baseline and belief vector, and only deviations that matter, *deltas* in the operational reading and *belief updates* in the epistemic reading, are surfaced.

The work is organised as a stack. Three layers build models in parallel: Deep Insight agents understand the company qualitatively; Financial Model agents quantify the economics; Synthetic Futures agents map the space of reachable states. A CrossValidator tests them against each other. A Dot Connector turns model changes into investable hypotheses with explicit falsifiers. Threadweave, the top layer, weaves the probable path of the entity through opportunity space over time, pulling from any of the layers below as the question requires. Underneath all of them, the **belief journey** runs inside every material delta: prior, evidence decomposition, rival explanations, likelihood logic, posterior, falsifier-ladder movement.

The loop is self-improving. Every delta and every belief update writes to an append-only record. Every forecast-worthy delta emits a prediction with a horizon and a falsifier. Reality resolves predictions and scores belief updates. Per-entity, per-subagent, per-belief calibration compounds over time. True signals sharpen the model. False positives narrow the filter. Missed signals expose blind spots. Confirmed mechanisms accrue calibration weight. Posteriors that prove well-calibrated against later reality earn the belief authority for future updates.

The initial vertical is finance, where markets provide a fast, unforgiving ground truth. The architecture is domain-general; finance is where calibration is cheapest to come by.

What this delivers in practice has two layers, and both matter. On day one, the system is a signal/noise filter tied to entities the customer cares about. An analyst covering 30 companies stops reading 200 unrelated news articles per day and instead reads 12 deltas, each tied to a specific entity, ranked by relevance against that entity's existing baseline and belief vector, stating *which beliefs moved, by how much, and which rival explanation gained*

probability. This is categorically different from a news API (which delivers everything), a market-data terminal (which delivers everything for \$30K a year and asks the analyst to filter), or a generic LLM with web access (which can summarise but cannot tell the analyst what changed about the entity's trajectory and whether that change is more material than yesterday's update). Filter-as-product is the immediate value.

Over time, the system compounds. Every belief update is written to an append-only record. Every forecast-worthy delta emits a prediction with a horizon and a falsifier. Reality resolves the predictions and scores the underlying belief updates on three tracks: posterior calibration, update-direction correctness, and diagnosticity calibration. After enough cycles, the system is no longer just a filter. It is a calibrated record of which mechanisms actually operate, which agents are reliable on which kinds of question, which rival explanations dominate which belief categories under which conditions, and which factors recur across confirmed mechanisms. That record is what makes the predictions trustworthy enough to act on, and it is the asset that does not exist anywhere else.

What the system writes to disk is not “opinions about companies.” It is a time-indexed record of reasoning under uncertainty, with named beliefs as the unit of account, scored against outcomes as they arrived. That record is the proprietary asset.

§02 First principles, the four non-negotiables

Before schemas or agents, four commitments. Every subsequent design decision can be traced back to these. Where a decision conflicts with a principle, the principle wins.

P/01 • Identity-first. One spine, one ID space. Every signal, inference, prediction, representation, belief, and outcome attaches to a stable, auditable identity. The identity graph is the spine. Everything else joins against it.

P/02 • Structured before fluent. Typed schemas, not chat. Agents read and write typed schemas. Free-text reasoning exists only as explanation attached to structured deltas, belief updates, and rival explanation sets the substrate can verify. If the system cannot verify a claim's shape, it cannot score it.

P/03 • Outcomes close every loop. No prediction is complete until scored. Interpretations only compound once they survive contact with reality. A prediction without an outcome is not yet knowledge, it is a hypothesis held open. A belief update without a calibration target is not yet a belief update, it is a model adjustment.

P/04 • Model-agnostic, moat-bearing. The record is the asset. Claude is the reasoning engine today. The defensible layer is the longitudinal record of reasoning itself, not the model producing it. Tomorrow's model inherits the ledger; the ledger does not inherit the model.

These four carry the entire design. The six-layer stack described later is a consequence of P/02 (structured exchange between specialists) and P/03 (every layer's output must be scorable). The substrate described in Part III is a consequence of P/01 (join everything by entity_id) and P/04 (append-only, so the record outlives any single model). The belief journey added in v4 is a consequence of P/02 sharpened (named, addressable claims, not implicit fields) and P/03 sharpened (every belief update must have a calibration target, not just a delta).

§03 The epistemic core, fog of war not narrative fallacy

There is a temptation to think of the baseline as the thing of value, the “rich dossier” on each entity. It is not. Baselines can be reconstructed by anyone with enough compute and good prompts. What cannot be reconstructed is what the system knew when it didn’t yet know what followed.

Consider two StahlTraces. Both arrive at the same current baseline today. System A builds it by running the current model over all historical evidence in one pass. System B has been running since entity creation, writing a delta every time its view changed, *recording the belief that moved and the rival explanation that gained probability*, emitting predictions against open horizons, and recording outcomes as they landed. Same endpoint. Different objects entirely.

System A produces a baseline contaminated by hindsight. Every driver it identifies as important is unconsciously selected for its relevance to outcomes the system already knows happened. System A has no track record, because every “prediction” it could retroactively generate was written after the outcome was known. System A cannot detect regime changes, because regime changes are shifts in interpretation, and A has no record of interpretation over time. System A cannot attribute error, because there is no wrong answer in its history, only the current one. **System A also has no belief trajectories:** even if it could reconstruct what it would have said about a company at a given date, it cannot reconstruct *which rival explanation it would have favoured at that date*, because rival weighting is a function of the evidence available at that moment plus the prior the system held going in, and the system did not have a prior going in.

System B is the lived record. Every delta written at wall-clock time. Every prediction timestamped before resolution. Every calibration snapshot versioned as it shifted. Every belief update written with prior, posterior, rival probabilities, and diagnosticity claim before the outcome resolved. System B can prove its forecasts weren’t fitted to outcomes. System B can show when it changed its mind and why. System B has a legally defensible audit trail.

This also settles the business question. Every serious customer or regulator conversation eventually asks a version of “prove you weren’t fitted to outcomes.” System A cannot answer. System B answers trivially with a join. The difference between those two answers is the difference between a subscription product with pricing power and a wrapper over an API with a half-life measured in months.

Given B, you can always derive A as a retrospective view. Given A, you cannot ever reconstruct B. The information that the system didn’t know yet is destroyed permanently the moment you stop recording it. This asymmetry, not taste, not aesthetics, is what forces every major architectural decision downstream. The moat is not what you know. The moat is what you knew, when you didn’t yet know what followed.

The belief corollary, new in v4

A system that records what it believed before the answer was known cannot do so honestly unless what it believed is a *named, addressable claim* with a prior confidence, a rival explanation set, and a calibration target. Otherwise the record is of model state, not belief.

v3.1 met the lived-record commitment at the level of deltas and predictions. v4.0 meets it at the level of beliefs. A delta is an operational change to the model state; it answers *what moved*. A belief update is an epistemic move; it answers *which prior claim about the entity changed, by how much, against which rival explanation, with what*

later calibration. Both are appended; both are immutable; both compound. The belief is the unit of account v4.0 adds because System B's claim to be "the lived record" is only as strong as the granularity at which the record is actually addressable.

§04 The five epistemic rules

An append-only data model is necessary but not sufficient. Five rules, enforced in code and reviewed in every pull request, are what make the system's history epistemically meaningful rather than merely tidy. The first four are unchanged from v3.1; the fifth is new in v4.

Rule 1 • Every delta and every belief update is written at the moment it happens, never retroactively. The `produced_at` field is the wall-clock timestamp of when the system wrote the row. It is distinct from `evidence_time`, which is when the underlying claim happened in the world. If these two fields ever become equal on a row in the Delta or Belief_update table, you are writing retrospective, and the row has no epistemic status.

Enforcement: only the live-mode orchestrator writes to Delta or Belief_update with `produced_at = now()`. Any retrospective-mode write lands in a separate `retrospective_view` table, never in Delta or Belief_update.

Rule 2 • Predictions and posterior commits are immutable after emission. Outcomes attach, never overwrite. A prediction committed at T holds its probability, mechanism, and falsifier forever. A belief update committed at T holds its prior, posterior, rival probabilities, and diagnosticity claim forever. If new information arrives, a new prediction is emitted and a new belief update is written. The old ones still stand, possibly destined to be wrong. That is the track record.

This is what protects Brier scores from leakage, posterior calibration from leakage, and the record from being legally defensible.

Rule 3 • Calibration snapshots are versioned, because weights change over time. When the orchestrator applies a weight of 0.65 to a subagent's delta in May 2026, the calibration state at that moment must be recoverable. A baseline built with May-2026 calibration is a different object from one built with today's calibration, even if the underlying evidence is identical. The same applies to belief-update scoring tracks.

Without versioned calibration, you cannot fairly score past deltas or past belief updates. You would be grading 2026 work by 2027 standards, which either flatters or punishes unfairly.

Rule 4 • Retrospective passes are additive, labelled, never authoritative. When a new subagent is added, a new evidence type becomes available, or an old reading looks wrong in context, running a retrospective pass is legitimate. Store the output in a parallel `retrospective_view` table, labelled with pass ID and date. Lived and retrospective views can be queried side by side. The authoritative delta record, the authoritative belief_update record, and the predictions that underwrite Brier and posterior calibration scores remain the lived ones, immutably.

Retrospective views are useful for analysis, for back-filling historical context on newly added entities, and for migration validation (see §42). They are not a substitute for the track record, and they must never be queryable as if they were.

Rule 5 • Every material update names the belief it moved, the prior confidence, and the rival explanation set considered. New in v4.

A material delta that does not attach to at least one `belief_id` is recorded with `context_only=true` and excluded from belief calibration aggregates. *Material* is defined by the v3.1 delta magnitude enum: deltas of magnitude `material` or `regime` must attach. Deltas of magnitude `noise` or `minor` may attach but are not required to.

Enforcement is hard-gated in code. The Orchestrator's commit step rejects any `material / regime` delta whose belief attachment field is empty. A rejected delta is re-routed to belief-extraction (step 04a of §06), which either attaches to an existing belief or proposes and authors a new one autonomously. There is no override path. The architecture commits to either finding the belief that moved or declaring the delta non-material. The rejection-and-reroute rate is a published per-agent metric: a layer agent that produces many material deltas requiring re-routing is exposing a gap between its operational reasoning and its belief vocabulary, and the gap is itself diagnostic.

Rule 5 is the rule that makes v4.0 a Bayesian update engine rather than v3.1 with extra tables. Without it, beliefs become a layer of decoration that the operational system can ignore; with it, the operational system has to find the belief, every time it changes its mind in a way that matters.

§05 The delta journey, the operational spine

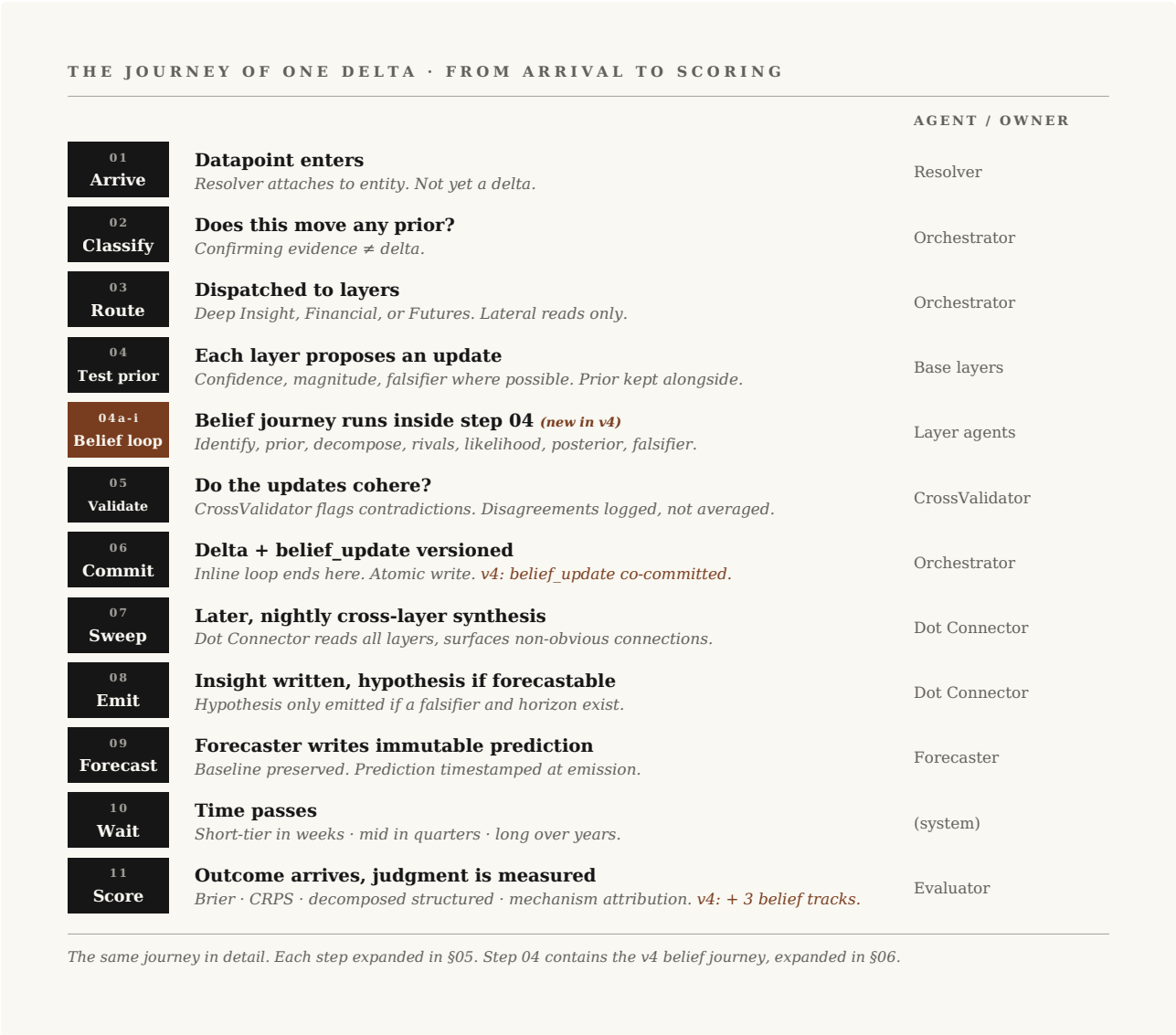


FIGURE 1 · THE DELTA JOURNEY AT A GLANCE. THE V4 BELIEF JOURNEY RUNS INSIDE STEP 04, SHARING TRANSACTIONAL COMMIT SEMANTICS WITH THE DELTA ITSELF. STEPS 1 TO 6 ARE INLINE (UNDER 14 SECONDS TYPICAL); STEPS 7 ONWARD ARE SCHEDULED.

The architecture can be understood either as a stack of agents or as a journey of a single signal from arrival to resolution. Both are correct; the journey is the one worth reading first, because it explains what every layer is for. v4.0 maintains the v3.1 delta journey as the operational spine and adds the belief journey (§06) as the epistemic spine. They run together: the belief journey is inserted between *Test prior* and *Cross-validate*, sharing the same Temporal workflow.

The journey of one delta, from arrival to scoring

#	Step	Owner	What happens
01 · Arrive	Datapoint enters	Resolver	A filing lands. A call is transcribed. An expert conversation is logged. A market move prints. The Resolver attaches the item to the right entity. It is not yet a delta, it is a claim.
02 · Classify	Does this move any prior?	Orchestrator	The Orchestrator inspects the claim against the current baseline and asks which model layers it could touch. A claim that reaffirms the baseline is logged as confirming evidence, not a delta. A claim that moves anything, even slightly, is a delta, with a magnitude attached.
03 · Route	Dispatched to relevant layers	Orchestrator	The Orchestrator sends the delta to Deep Insight, Financial Model, Synthetic Futures, whichever layers the claim touches. Lateral consultation is allowed: a Financial agent may read a Deep Insight agent's proposed delta before forming its own, but does not write on its behalf.
04 · Test prior	Each layer proposes an update	Base layers	Every layer that receives the delta proposes a specific change to its model state, with evidence, confidence, and where possible a falsifier. The prior is preserved alongside the proposed update. The system keeps both until the delta clears validation.
04a–04i · Belief journey	The full belief journey runs here	See §06	The belief journey is the inner loop: identify or extract beliefs, retrieve priors, decompose evidence, generate rival explanations, estimate likelihood logic, produce posterior, adjust falsifier distance, cross-validate posteriors, commit belief_update + delta atomically.
05 · Cross-validate	Do the updates cohere?	CrossValidator	Five checks (coherence, dependence, adjacency, preservation, probability conservation, see §11). Contradictions are not averaged away, they are logged as unresolved disagreements. A delta that breaks adjacent models without justification is rejected or escalated.
06 · Commit	Delta + belief_updates committed, baseline materialised	Orchestrator	The Orchestrator writes the delta and the belief_update rows to the substrate in one transaction. The replay engine re-materialises the baseline snapshot. The inline loop ends here. Users may see a notice that beliefs have moved; the system has not yet claimed any pattern, insight, or forecast from this delta alone.
07 · Sweep	Scheduled cross-layer synthesis	Dot Connector	On a schedule (nightly for most entities; weekly for low-activity ones), the Dot Connector reads the full substrate (recent deltas across every layer, recent belief_updates across every belief category, current base-model states, the hypothesis ledger, the mechanism-check ledger, the factor co-occurrence ledger, CrossValidator's unresolved disagreements) and looks for non-obvious patterns that connect dots across layers and across beliefs. This is where the system produces understanding, not just reaction.
08 · Emit	Insight written, hypothesis if forecastable	Dot Connector	The Dot Connector writes an insight to the Insight table, with its full evidence chain (which deltas, which belief_updates, from which layers, in what sequence). If the insight implies a forecastable claim (has a falsifier and a bounded horizon), it also emits a hypothesis as a child. Most insights are not forecastable, and that is fine: they still compound understanding and become input for future sweeps.
09 · Forecast	Prediction emitted, timestamped, immutable	Forecaster	If a hypothesis was emitted in step 08, the Forecaster writes a prediction: direction, probability, horizon, mechanism, falsifier, mechanism evidence requirements. The prior baseline is preserved; the new one is committed alongside. Nothing is overwritten.

#	Step	Owner	What happens
10 · Wait	Time passes	(system)	Short-horizon forecasts score in weeks. Mid-horizon in quarters. Long-horizon, Threadweave-scale claims about path through opportunity space, may score over years. Threadweave's reliability on any entity is, in part, a function of how much scored judgment has accumulated at each horizon tier.
11 · Score	Outcome arrives, judgment is measured	Evaluator	The Evaluator matches the outcome to the open prediction, computes a score (Brier, CRPS, decomposed structured), runs mechanism attribution as an ensemble, and writes belief-update verdicts on the three tracks (posterior calibration, update-direction correctness, diagnosticity calibration). Calibration is snapshotted, per layer, per horizon tier, per belief category. The system has learned.

Every architectural choice in Parts II and III exists to make this journey scorable, auditable, and compounding. If any step is weakened, if deltas are overwritten, if predictions are edited after emission, if belief_updates are mutable after commit, if the orchestrator commits without cross-validation, the system can still look useful but it stops being a learning engine.

§06 The belief journey, the epistemic spine (*new in v4*)

The delta journey answers *what moved in the model*. The belief journey answers *which named claim about the entity changed, by how much, against which rival explanation, with what later calibration*. They run together. The belief journey is the inner loop inside step 04 of the delta journey. Nothing in the v3.1 spine is removed.

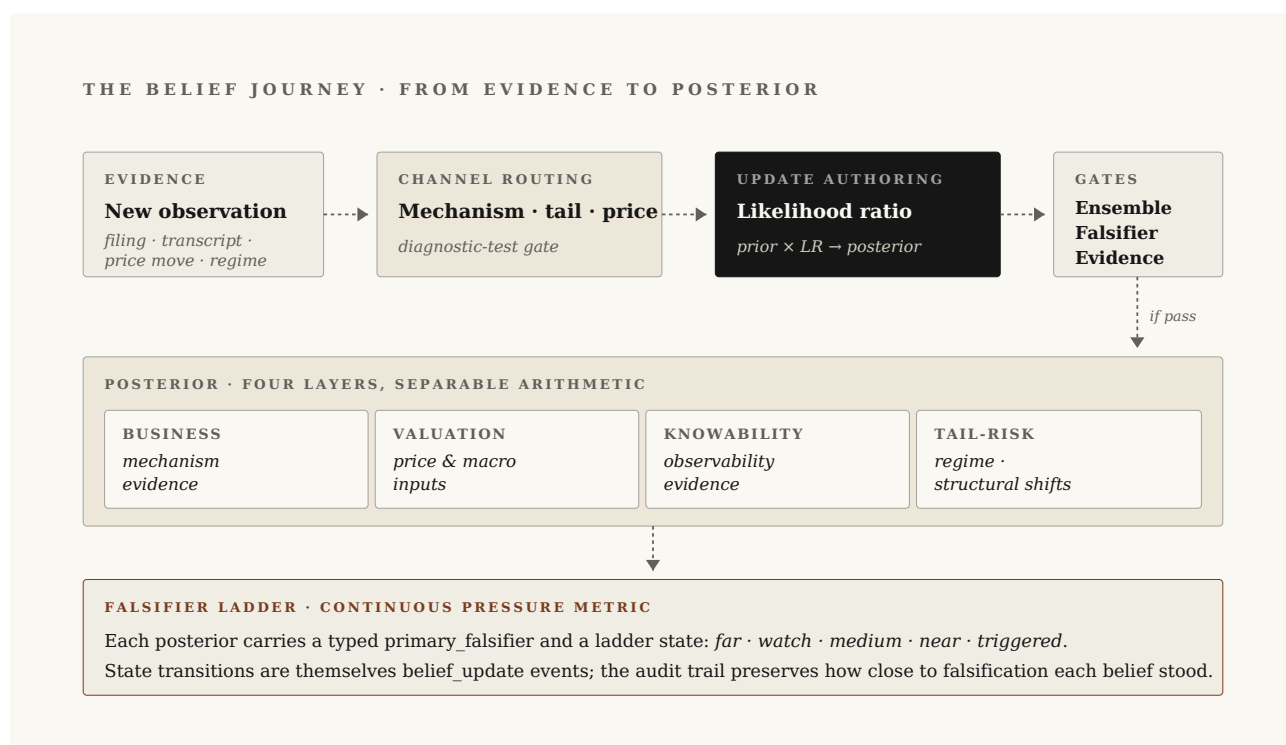


FIGURE V1 · THE BELIEF JOURNEY, FROM EVIDENCE ARRIVAL THROUGH CHANNEL ROUTING, AUTONOMOUS GATE ENFORCEMENT, POSTERIOR DECOMPOSITION INTO FOUR SEPARABLE LAYERS, AND THE FALSIFIER LADDER THAT RUNS ALONGSIDE EVERY BELIEF.

The journey of one belief update, from prior to posterior to scored result

#	Step	Owner	What happens
04a · Identify or extract beliefs	The proposed delta is mapped to one or more existing <code>belief_id</code> s	Orchestrator + ensemble extractor	If no belief matches and the delta is material, the ensemble belief-extractor runs: multiple extraction passes with different prompt seeds and different evidence-window scopes propose candidate beliefs. A candidate is authored to the <code>belief</code> table only if (a) the ensemble produces the same belief statement above an agreement threshold, (b) the candidate carries a non-empty <code>primary_falsifier</code> , (c) it carries non-empty <code>expected_evidence</code> . A candidate that fails any of the three gates is logged in the <code>belief_extraction_attempts</code> audit table and discarded; the originating delta is re-classified non-material if no existing belief absorbs it. Ensemble disagreement on belief authoring is logged and treated as signal, mirroring the §15 Evaluator-ensemble pattern.
04b · Retrieve prior and expected evidence	For each affected belief, read the current <code>prior_confidence</code> and the <code>expected_evidence</code> registered when the belief was authored	Orchestrator	The prior is the belief's <code>current_confidence</code> at this moment. The <code>expected_evidence</code> is the structured record of what the belief predicts should appear in the world if the belief is true. Both are inputs to step 04c and 04e.
04c · Decompose evidence	The evidence packet is decomposed	Layer agents	Five components: reliability (source-tier, 0..1), magnitude (numeric or categorical), persistence (one-period vs structural vs unknown), independence (does this duplicate a prior signal already counted, 0..1), diagnosticity (does it discriminate between rival explanations, 0..1, claimed at update time and verified at resolution). The decomposition is what makes likelihood logic tractable in step 04e.
04d · Generate rival explanations	For each affected belief, enumerate the rival explanations consistent with the evidence and assign prior probability to each	Layer agents	Default rival set: clean confirmation, temporary or transitory, opposing mechanism, noise. Layer-specific rival sets override the default (e.g., a moat belief uses {moat-supplier-specific, moat-operational-not-structural, moat-unaffected, supplier-independent moat}). Probabilities sum to 1. The rival set is written as an <code>alternative_explanation_set</code> row.
04e · Estimate likelihood logic	For each rival explanation, judge whether the evidence looks more like <i>this</i> explanation, more like <i>that</i> one, or roughly the same	Layer agents	Internal representation: likelihood ratios $LR_i = \frac{P(\text{evidence} \mid \text{explanation}_i)}{P(\text{evidence} \mid \text{rival_explanation})}$. The system uses likelihood ratios at full numeric precision internally. The customer-facing surface presents posteriors as confidence bands (see §15).

#	Step	Owner	What happens
04f · Produce posterior belief vector	Posterior odds = prior odds × likelihood ratios, normalised across the rival explanation set	Layer agents	The posterior is bounded by floors and ceilings tied to evidence reliability: a single low-tier source cannot swing a high-confidence prior past a configured threshold. The posterior is then projected back into a <code>current_confidence</code> value on the belief and into per-explanation posterior probabilities on the <code>alt_explanation_set</code> . The dominant shift (which rival gained the most mass) is recorded.
04g · Adjust falsifier distance	The belief's <code>falsifier_distance</code> field is updated on the five-state ladder	Layer agents	States: <code>far</code> , <code>watch</code> , <code>medium</code> , <code>near</code> , <code>triggered</code> . Movement on the ladder is itself a tracked quantity: the prior state and posterior state are both written to the <code>belief_update</code> row. The ladder semantics are detailed in §41.
04h · Cross-validate posteriors	Probability conservation across rival explanations, coherence of posteriors across layers for shared beliefs	CrossValidator	This is Check/05 of §11. The check is mostly deterministic (sum-to-one within tolerance, no single explanation moving past its evidence-bounded ceiling, dominant shift supported by recorded diagnosticity). Genuinely ambiguous cases escalate to the Orchestrator's critique pass.
04i · Commit belief_update + delta atomically	Both rows are written in one transaction, sharing a transaction id	Orchestrator	The delta is the operational change to the model state; the <code>belief_update</code> is the epistemic move. They are durably linked. The replay engine re-materialises the baseline snapshot.

Why this insertion is non-disruptive

The Orchestrator already runs a multi-agent dispatch with proposed deltas held until cross-validation clears. Steps 04a through 04g extend what each agent produces inside its proposal, not when it proposes. Step 04h is a new CrossValidator check, not a new orchestration phase. Step 04i is one extra row write per material delta.

The inline loop budget (under 14 seconds in the v3.1 worked example) is extended by approximately 2 to 4 seconds for the rival-explanation and likelihood passes. Latency budgets in §17 absorb this. If they do not, the layer-side computation moves to the post-commit sweep without changing semantics.

What the belief journey does not do

It does not author beliefs in isolation from the delta journey; beliefs only emerge when an actual material delta needs one. It does not score belief updates in real time; scoring waits for the resolution window in step 11 of the delta journey. It does not require any human in the loop; the architecture commits to autonomous belief authoring (gated by ensemble agreement and the three deterministic checks) precisely so that the corpus property of being purely system-authored and system-scored is preserved end-to-end. See §42 for the autonomy commitment in full.

Rule of the upgrade. *No prior, no update. No named belief, no prior. A material delta that does not attach to at least one belief is flagged context-only and excluded from belief calibration aggregates.*

Part II · The agent layer

Six layers that make reasoning measurable. Three parallel model-building layers, a cross-validator, a hypothesis engine, a path engine on top. Plus four support agents that close the loop with reality. Thirteen specialists are organised into functional tiers; two layers, Dot Connector and Threadweave, sit at the apex.

§07 The six-layer learning stack

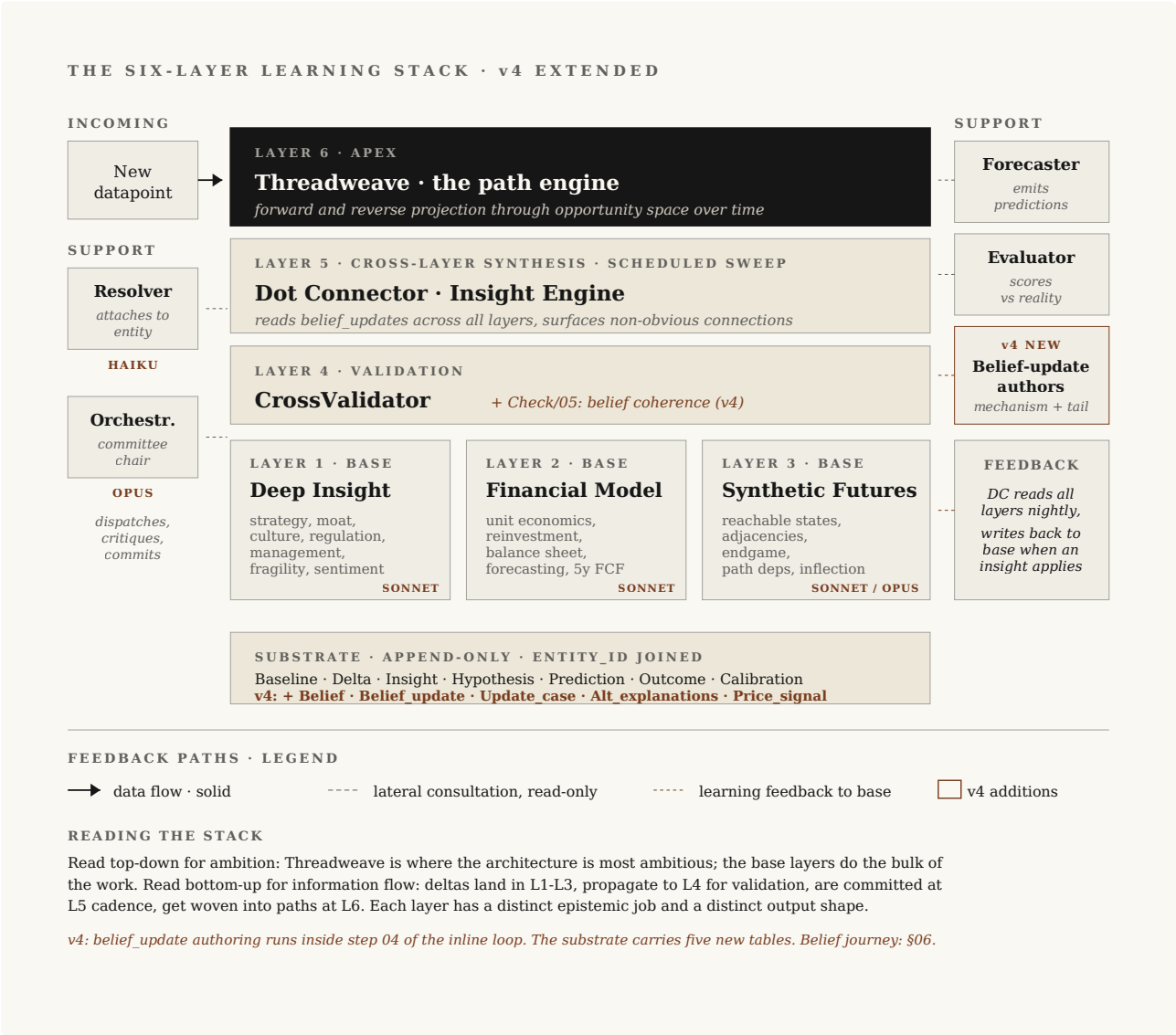


FIGURE 2 · THE SIX-LAYER LEARNING STACK WITH SUPPORT AGENTS, MODEL-TIER ASSIGNMENTS, AND FEEDBACK PATHS. V4 EXTENDS THE SUBSTRATE WITH FIVE NEW TABLES (HIGHLIGHTED IN RED), ADDS CHECK/05 TO CROSSVALIDATOR FOR BELIEF COHERENCE, AND INTRODUCES A DEDICATED BELIEF-UPDATE AUTHOR. READ TOP-DOWN FOR AMBITION, BOTTOM-UP FOR INFORMATION FLOW.

The agent layer is organised as a stack, read top-down in the order ambition increases and bottom-up in the order information flows. Each layer has a distinct epistemic job, a distinct output shape, and a distinct place in the delta journey and the belief journey.

Layer 6 • apex • Threadweave. Weaves the probable path of the entity through opportunity space over time. Reads the insight graph primarily, with confirmed hypotheses sharpening the path and unresolved ones widening uncertainty. Falls back to the base models when an insight does not cover a specific question. The core investment question lives here: can this entity turn opportunity space into economic reality faster and more efficiently than competitors?

Layer 5 • cross-layer synthesis • Dot Connector / Insight Engine. Reads across every layer below (all recent deltas, all recent belief_updates, base-model states, the hypothesis ledger, CrossValidator's disagreements) on a scheduled sweep, and synthesises insights that no single layer could produce alone. Insights are the primary output. Hypotheses are emitted as children only when an insight is forecast-worthy. Writes to the Insight table, and to Hypothesis when forecastable.

Layer 4 • validation • CrossValidator. Formal contradiction detection across the three base layers, plus probability conservation across rival explanation sets. Five typed checks (coherence, dependence, adjacency, preservation, probability conservation). Disagreements are preserved as unresolved disagreements rather than averaged into a number.

Layer 3 • base model • Synthetic Futures Agents. Probable futures, reachable states, path dependencies, key uncertainty nodes. Not “how big is the market today” but how many new reachable states can this entity assemble over time? Assembly-theoretic framing. Writes to the futures sub-model.

Layer 2 • base model • Financial Model Agents. Revenue drivers, margins, capital efficiency, unit economics, reinvestment runway, balance sheet risk, dilution, operating leverage. Every update shows exactly what changed. Writes to the financial sub-model.

Layer 1 • base model • Deep Insight Agents. Qualitative understanding: strategy, product, culture, moat, customer love, regulation, management, competition, incentives. Outputs living insight models with evidence, confidence, falsifiers, and explicit deltas. Writes to the insight sub-model.

Layers 1 to 3 run in parallel on a given delta, with lateral read-only consultation allowed. Layer 4 validates every delta inline before it commits, with the belief journey (§06) running inside its window. Layer 5, the Dot Connector, runs on a different cadence entirely: it reads the full persisted substrate on a scheduled sweep (typically nightly) and looks for patterns across layers and across belief_updates that no single delta would reveal. Layer 6 consumes Layer 5's insight graph as its primary input and falls back to the base models only when an insight does not cover a specific path question.

Four support agents (Resolver, Orchestrator, Forecaster, Evaluator) are described in §14. They are not layers in the stack; they wrap it. The Orchestrator is the committee chair for the whole stack. The Resolver decides what entity an incoming claim belongs to. The Forecaster emits the prediction when the Insight Engine flags an insight as forecastable. The Evaluator closes the loop when reality resolves.

§08 Deep Insight Agents, qualitative understanding of the entity

The Deep Insight layer is what automates the work of a thoughtful analyst reading everything a company says and does, and building a mental model that survives more than a quarter. Its output is a living insight model: structured claims about strategy, moat, culture, regulation, management, competition, incentives, each attached to evidence, confidence, a falsifier, a history of deltas, and (in v4) the beliefs each delta moves.

Six specialists work this layer. Each is a Claude instance with a narrow system prompt, a typed tool set, and strict read/write scope. They do not chat with each other; they exchange structured proposed deltas through the substrate. Lateral read-only consultation is allowed before commit.

DI/01 · Supply (leadership). Quality of thinking inside the company. Talent density, incentive alignment, resistance to institutional drift, operational execution. Does the organisation still think, or has it hardened into process? Δ exec quality drivers. *Tier 1 · Foundation · Sonnet.*

DI/02 · Effective Utility. The real function the company serves. Reduces the company to its real function: the uncertainty it removes, the desire it serves, the constraint it solves, the behaviour it monetises. Δ real_function, monetised_behaviour. *Tier 2 · Lens · Sonnet.*

DI/03 · Anomalies. Where the company is structurally different. New routes to demand, bypassed friction, compressed time, created trust, expanded market boundary, cult following. Durability matters more than novelty. Δ anomaly type, durability. *Tier 2 · Lens · Sonnet.*

DI/04 · Moat Mechanisms. Does the advantage reinforce itself? Does scale improve economics? How will competitors respond? Is the moat stable or exposed? The question is not width, it is the derivative. Δ moat type, reinforcement, exposure. *Tier 2 · Lens · Sonnet.*

DI/05 · Fragility. What unusually smooth results may hide. Paradigm-shift exposure, unsustainable debt, over-optimisation. Low error rates treated with caution; adaptability is read as a property, not a statistic. Δ fragility band, adaptability. *Tier 2 · Lens · Sonnet.*

DI/06 · Sentiment. Is consensus aligned with the facts? Consensus, media framing, analyst behaviour, ownership, short interest, insider behaviour. The fact-sentiment gap is the feature, not the average of the two. Δ consensus, fact-sentiment gap. *Tier 2 · Lens · Sonnet.*

The layer as a whole writes to the Insight sub-model. Individual deltas are typed: each specialist can only write into its own domain paths, which makes contradictions cheap to detect at the CrossValidator layer (two specialists touching the same path with incompatible proposals is a syntactic flag, not a semantic one). Each specialist also produces, as part of its delta, the belief_update rows that attach to the affected beliefs (per §06).

§09 Financial Model Agents, the numbers test

The Financial layer forces every qualitative read into numbers. Without this layer the system produces plausible narratives that never have to clear a P&L. With it, every delta in the Deep Insight or Synthetic Futures layer is pressure-tested against what would actually have to be true on a cashflow statement, and every belief that touches the financials is updated with a quantitatively decomposed evidence packet.

Two specialists work this layer. The first reads what the financials are telling you now; the second projects what they will have to tell you if the insight and futures layers are right.

FM/01 • Capital. What the financials say now. ROE stability, leverage, cashflow quality, unit economics, capital allocation, financial resilience. Does the system compound or does it depend on external support? Every balance-sheet claim produced elsewhere in the stack must eventually pass through this agent. Δ capital drivers, balance-sheet risk, reinvestment runway. *Tier 1 • Foundation • Sonnet.*

FM/02 • Forecasting / Models. What the financials will have to say. Forces the company read into numbers. 5y FCF, CAGR paths, reinvestment runway, growth limits, drawdown convexity, variables that must work. Does the advantage reach shareholders per-share? Triggers the Forecaster when a delta produces a quantitative claim. Δ quantitative drivers • *Trigger Forecaster. Tier 3 • Synthesis • Opus.*

What this layer owes the others

Every Deep Insight delta that implies a financial consequence (a moat hypothesis implies pricing power; an anomaly implies unit economics deviation) must produce a FM delta within the same orchestration window, or the orchestrator flags the insight as unsupported. An insight that cannot find a home in the financials is a narrative, not an investment view.

What this layer receives from the others

A Synthetic Futures delta that claims reachable-state expansion must hand the Financial layer a scenario shape (markets opened, capital required, timeline) that the Forecasting agent can attach to a discounted cashflow. Without that, the future is decorative.

The FM layer's most important output is not the model itself, it is the list of variables that must work for the thesis to hold. Those variables become watch-items for the Forecaster, scoring items for the Evaluator, and (in v4) the `expected_evidence` field on the financial-quality beliefs the layer writes to.

§10 Synthetic Futures Agents, reachable states not just TAM

The Synthetic Futures layer asks a different question than the others. Deep Insight asks *what is this company*. Financial asks *what will it earn*. Synthetic Futures asks *what states can this company reach, and with what probability and through what path*.

The right frame is assembly-theoretic rather than addressable-market-theoretic. Instead of estimating a static TAM and multiplying by share, the layer enumerates the adjacent reachable states (new products, new markets, new business models, competitive responses, constraint shifts) and attaches probability weights and path dependencies. The output is a tree, not a number.

SF/01 • Demand (strategy). The strategic direction of travel. Expanding addressable market, defending position, or competing in shrinking space. Tested through case comparisons and direct checks. Δ demand drivers, direction-of-travel. *Tier 1 • Foundation • Sonnet.*

SF/02 • Dynamic Exploration. Paradigm shifts and inflection points. Expanding opportunity spaces, fat-tailed outcomes, adoption inflection, ROE turning points, escape-velocity readiness. Δ inflection, scale readiness. *Tier 2 • Lens • Sonnet.*

SF/03 · Endgame Paths. Where the industry resolves. Winner-takes-all, winner-takes-most, scope-widening, bounded, average, displaced. Which path, and why, and what this company's probability weight is on each. Δ endgame class, path probability. *Tier 2 · Lens · Sonnet.*

SF/04 · Technicals (path evidence). What the price path already implies. Long-term: log-growth path intact, broken, accelerating, reverting? Short-term: drawdowns, volatility, relative strength, volume, recoveries. Feeds path evidence into Threadweave. **In v4, also feeds the formal price_signal evidence layer described in §40, which decomposes price into market, sector, factor, and event-specific components and routes the components to the appropriate posterior layers.** Δ log-path state, relative strength. *Tier 2 · Lens · Sonnet.*

Why this layer is separate from Deep Insight

Deep Insight is about the company as it stands. Synthetic Futures is about the company as it could become. The two are deliberately decoupled so that an insight delta (e.g. “product culture has sharpened”) can be evaluated against futures independently of whether the futures agents agree the opportunity space is widening. Collapsing them into one layer conflates two different epistemic operations and makes contradictions invisible.

Output shape

The futures sub-model is a weighted tree of reachable states, each with: a path description, a probability weight, the key uncertainty nodes along the path, the constraint shifts required, and the expected competitive response. Deltas are updates to this tree (new branches added, probabilities shifted, paths pruned). Belief_updates attach to the beliefs that the tree restructuring implies (typically `category=structural_future` beliefs).

Two scopes: entity-bound reachable states, and structural futures

Synthetic Futures produces output at two distinct scopes, and the distinction is architecturally meaningful.

Entity-bound reachable states. What configurations could this specific company be in, 1 to 5 years out? A reachable state describes a possible future for a particular entity given its current structure, resources, and trajectory. Reachable states are written as deltas in the entity's substrate.

Structural futures. What configurations will the market or vertical be in, regardless of which specific company occupies which position? A structural future describes equilibrium properties of an industry: how concentrated it will be, what the Pareto distribution of revenue will look like, how many dominant players it will support, what economic regime will prevail. Structural futures are independent of any individual entity. They are written to a separate `future_state` table that any entity can be evaluated against. Beliefs of `category=structural_future` are evaluated against this table.

The two scopes are complementary. Entity-bound reachable states feed Threadweave's forward direction (where can this entity go). Structural futures feed Threadweave's reverse direction (what would have to be true for this entity to occupy a future position the structural future predicts will exist). Synthetic Futures is the only layer that produces both; the architectural commitment is that structural-future output is first-class, not a byproduct of entity-bound analysis.

Why structural claims are often more confident than identity claims

“Vertical-AI radiology will have a dominant player by 2032 with approximately \$20B revenue” is a structural claim grounded in demand curves, capability requirements, and historical patterns of market formation. “Company A will be that player” is an identity claim that depends on Company A’s specific decisions, competitor responses, regulatory accidents, and timing. Most quant systems conflate these or skip the structural layer entirely. Synthetic Futures separates them by design.

§11 CrossValidator, formal contradiction detection (*amended in v4*)

CrossValidator is a rules engine, not an agent. It does not propose deltas of its own. It does not propose belief updates. It sees the proposed deltas and proposed belief_updates from all three base layers for a given orchestration window and runs a narrow set of typed checks. Most of these checks are deterministic schema-level invariants (coherence, dependence, adjacency, preservation, probability conservation) expressible as SQL or typed code rather than as LLM judgment. Genuinely ambiguous coherence cases escalate to the Orchestrator’s critique pass (§14), where LLM reasoning legitimately belongs. CrossValidator’s value is being fast, cheap, and consistent on the 95% of contradictions that are mechanically detectable; the remaining 5% deserves an agent, not a rule.

Check/01 • Coherence

Does the insight model support the financial? If Deep Insight says the moat is widening, the Financial model should show (or be on the path to show) improving unit economics. If it does not, the deltas are flagged as incoherent, not averaged.

Check/02 • Dependence

Does the future require the present? If a Synthetic Future requires a capability or position the Deep Insight layer has not claimed, the future is flagged as unsupported. An unsupported future is not rejected; it is logged as hypothesis, not projection.

Check/03 • Adjacency

Do untouched models remain consistent? A delta to the Insight layer about customer behaviour may have implications for a Financial claim that was not touched. CrossValidator checks whether adjacent, untouched models remain consistent with the committed update.

Check/04 • Preservation

What priors would now be violated? If committing the delta would break a prior with high calibration weight, the orchestrator is notified and must either downgrade the prior (with reasoning logged) or reject the proposed delta.

Check/05 • Probability conservation across rival explanations (*new in v4*)

For any belief whose `alternative_explanation_set` was updated by the current orchestration window, three sub-checks run:

1. **Sum-to-one within tolerance.** The posterior probabilities across the rival set must sum to 1 within a small numerical tolerance.

2. **Evidence-bounded ceilings.** No single explanation can move past a configured ceiling tied to evidence reliability. A single tier-3 source cannot drive a rival explanation from prior 0.10 to posterior 0.85.
3. **Dominant-shift support.** The dominant shift (which rival gained the most mass) must be supported by the evidence diagnosticity recorded at step 04c. A dominant shift toward a rival the recorded evidence is not diagnostic for is flagged.

Violations are logged as unresolved disagreements, the same treatment as the four existing checks. The check is mostly deterministic and runs as SQL or typed code, consistent with CrossValidator's philosophy.

What CrossValidator writes

Contradictions, deltas, confidence changes, downstream updates, or annotations. It does not average. It does not vote. It does not propose beliefs. A disagreement surfaced by CrossValidator lives as an unresolved disagreement until either new evidence resolves it or the orchestrator makes an explicit, logged call to commit one side and mark the other as minority view.

Disagreement is a feature. It is information about the limit of current understanding, and it is lost forever the moment the system averages the two readings together.

CrossValidator corresponds roughly to v1's SA-12 Multi-Model Validation, promoted to its own layer because in v2 the three base layers have distinct operational character and it is no longer acceptable for their contradictions to surface only in the orchestrator's critique pass. v2.1 refinement: CrossValidator is implemented as a deterministic rules engine, not an LLM agent. Cross-validation is structural, not stylistic, and most structural checks do not require reasoning, they require correct typed invariants enforced consistently. v4 extension: Check/05 is added in the same deterministic spirit; the architecture commits to noisy posteriors being honestly noisy in the substrate, the same way it commits to noisy mechanisms being honestly noisy.

§12 Dot Connector / Insight Engine, cross-layer synthesis (*amended in v4*)

The Dot Connector is the layer that connects the dots. It reads across every layer below it (Deep Insight, Financial Model, Synthetic Futures, CrossValidator, the full delta history, **the full belief_update history**, the hypothesis ledger) and synthesises insights that no single layer could produce alone.

The name is literal. An individual delta is a point. **An individual belief_update is a point.** A pattern across multiple deltas in multiple layers (a shift in capital allocation that matches a strategic pivot that matches a widening opportunity space) is a shape. **A pattern across multiple belief_updates on multiple beliefs (a moat belief weakening, a fragility belief strengthening, an opportunity-space belief contracting, all on the same entity within a 30-day window) is also a shape.** The Dot Connector's job is to see the shape.

Cross-layer, not delta-by-delta

An earlier draft of this architecture had the Dot Connector firing every time a delta landed, converting each delta into a hypothesis. That is not what this layer is. A delta-by-delta hypothesis generator would be a transformation pipeline; the Dot Connector is a synthesiser. It runs on a schedule (nightly for active entities, weekly for quieter ones) and its inputs are the full substrate at the time of the sweep, not a single incoming event.

The practical consequence: if three deltas land in three different layers on the same day and each is individually unremarkable, the Dot Connector sees all three together on the next sweep and may recognise a pattern none of them implies alone. If they do not form a pattern, nothing is emitted, and that is also a valid outcome.

Belief-update patterns as first-class input (new in v4)

In v4, the Dot Connector also reads the belief_update stream as a first-class input alongside the delta stream. A pattern across multiple belief_updates becomes available even when the underlying deltas look diverse. Three examples:

1. **Cross-belief coherence.** B-481 (moat belief) posterior falls, B-617 (over-optimisation belief) posterior rises, B-512 (capital efficiency belief) posterior falls. Across one entity, this is a coherent thesis-level shift. Across three peer entities in the same sector within a month, it is a sector pattern.
2. **Rival-explanation regime shift.** “Rising competitive pressure” gains posterior mass across many entities in the same sector during the same window. A single update is noise; many updates is a regime signal.
3. **Diagnosticity drift.** A diagnosticity claim that was historically reliable starts proving over-claimed across many recent updates. The drift itself is an insight: the kinds of evidence the system relied on are losing their signal-to-noise.

These patterns are written as insights. When forecastable, they spawn hypotheses that cite both the contributing deltas *and* the contributing belief_updates as evidence chain.

Insights are the primary output

The Dot Connector writes insights, not hypotheses. An insight is a structured statement with an evidence chain (which deltas, which belief_updates, from which layers, over what window) and a confidence. A hypothesis is a child of an insight: when an insight implies a forecastable claim with a falsifier and a bounded horizon, a hypothesis is emitted alongside. When it does not, the insight stands on its own.

Most valuable patterns are not forecastable in a clean sense. “Capital allocation has quietly shifted toward defensive adjacencies over six quarters” is an insight that matters for a long-horizon view, shapes Threadweave’s path, and may never produce a scoreable short-horizon claim. Forcing it to become a hypothesis would either invent a falsifier that misrepresents the claim, or suppress the insight entirely. Neither is acceptable.

What the Dot Connector produces

Insight statement. A specific pattern the system has noticed, phrased so that a human analyst could agree or disagree with it.

Layer evidence. Which layers contributed to the insight, and with what weight. A pattern that relies only on the Financial layer is less robust than one that converges across Financial, Deep Insight, and Synthetic Futures.

Evidence chain. The sequence of deltas, belief_updates, and prior insights that, taken together, produce this insight. Each link is a pointer, not a paraphrase, so the chain can be reconstructed from the ledger.

Forecastability flag. Whether the insight can be converted into a hypothesis with a falsifier and a bounded horizon. If true, a hypothesis is emitted as a child.

Child hypothesis (optional). When forecastable: the hypothesis statement, falsifier, horizon tier, and expected confidence trajectory. Only forecastable insights trigger the Forecaster.

Relation to CrossValidator

CrossValidator is a within-sweep gatekeeper: every delta passes through it before committing, to catch contradictions. The Dot Connector is an across-sweep synthesiser: it reads the full substrate (including CrossValidator's unresolved disagreements) and looks for patterns that those very disagreements might reveal. A persistent disagreement between Deep Insight and Financial over several weeks is itself a signal, and the Dot Connector can name it as an insight.

Relation to Threadweave

Threadweave reads the insight graph, not the raw delta stream. An insight that describes a durable pattern shapes Threadweave's path weightings; an insight with a confirmed hypothesis feeds Threadweave's reliability score for the relevant horizon tier. When Threadweave needs something the Insight table does not cover, it falls back to the base models, but that fallback is the exception, not the rule.

Most systems that track changes stop at the change. StahlTrace connects the changes, on a schedule, and writes what it sees. v4 also connects belief moves across entities, sectors, and time, on the same schedule.

§13 Threadweave, the path engine projected through intent

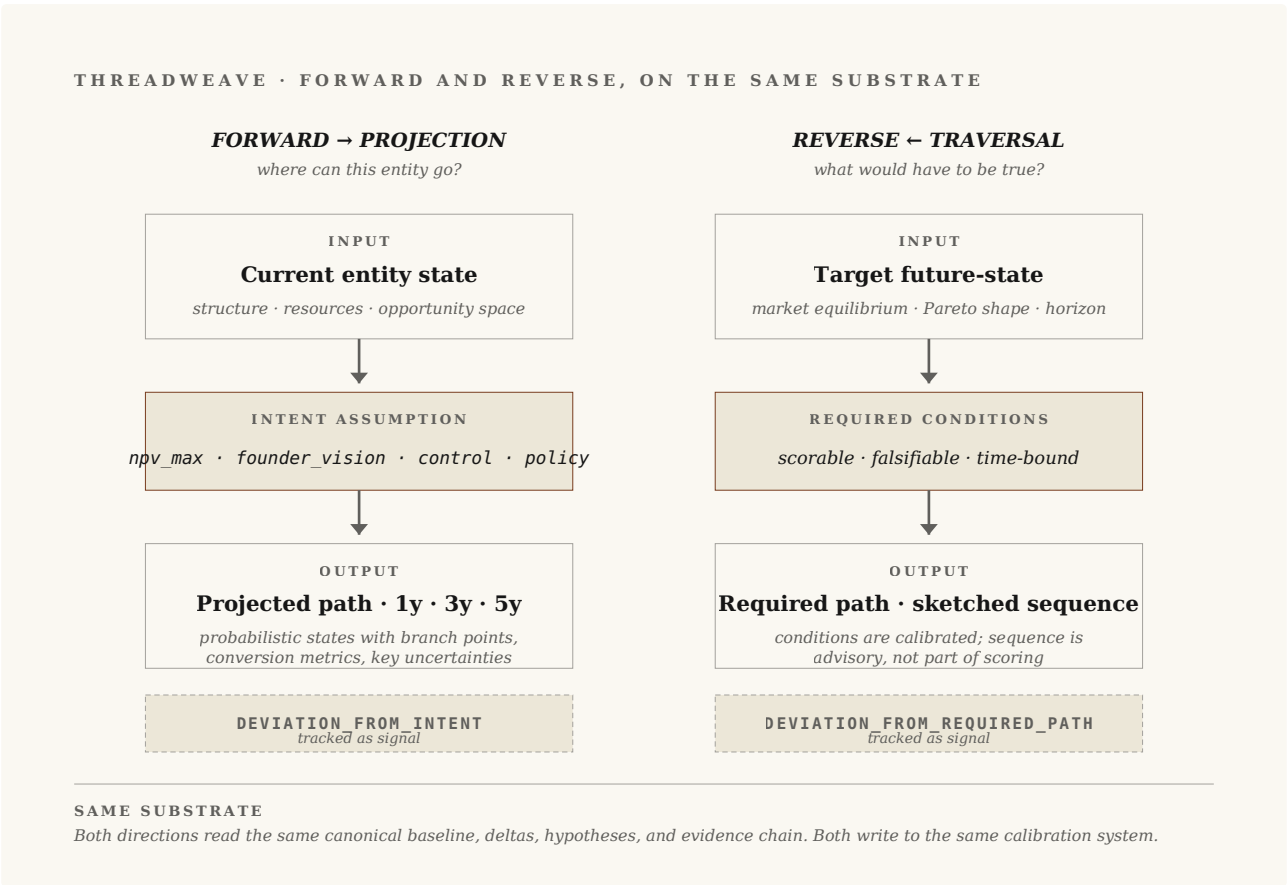


FIGURE 7 · THREADWEAVE BIDIRECTIONAL PROJECTION. FORWARD FROM CURRENT STATE UNDER INTENT; REVERSE FROM TARGET FUTURE-STATE TO REQUIRED CONDITIONS. BOTH DIRECTIONS ARE TRACKED AS SIGNAL WHEN ENTITY BEHAVIOUR DEVIATES FROM THE PROJECTED OR REQUIRED PATH.

Threadweave sits at the apex of the stack. It is the only layer whose output is not a delta. Its output is a path, and the path is not whatever the data passively implies, but what an intent-bearing actor with this company’s structure, finances, and opportunity space would most plausibly do over the projected horizon. The reframe matters: companies are not event streams that unfold, they are agents that decide. Modelling the path as the consequence of an agent’s decisions, rather than as a passive emergence from the data, is what lets Threadweave reason about counterfactuals rather than just extrapolate trends.

The agent the system models is the company itself: a bounded-rational actor pursuing a stated objective under known constraints, with imperfect information and finite capacity to act. The default assumed objective for public-equity finance is net present value maximisation, the obligation a public company’s board carries to its shareholders. The default is overridable per entity when evidence supports a different objective; both default and override are explicit, inspectable, and recorded with the projection.

What Threadweave consumes

Primarily, the insight graph produced by the Dot Connector. An insight is already a cross-layer synthesis with evidence, so it is the most information-dense input Threadweave has. Confirmed hypotheses sharpen the path further; unresolved or falsified ones widen the uncertainty bands. Mechanism-confirmation rates from §15 enter the projection too; agents whose mechanisms are reliably confirmed produce inputs Threadweave weights more heavily than agents whose outputs are well-calibrated for unclear reasons.

The lower-layer outputs are not raw inputs to a passive synthesis; they are the agent's self-knowledge and action space. Deep Insight (§08) describes what the company is and how it operates: the agent's structural awareness. Financial (§09) describes what resources it has, what it can spend, what its constraints are: the agent's capacity to act. Synthetic Futures (§10) describes the reachable states: the agent's action space. Threadweave projects what an agent with this self-knowledge, these resources, and this action space would most plausibly do.

When the insight graph does not cover a question Threadweave is asked (a specific cashflow scenario, a competitor's balance-sheet shadow, a still-emerging adjacency), Threadweave falls back to the base models directly. That fallback is the exception, not the default; most path questions at 1y, 3y, and 5y resolve from the insight graph plus the base-model current state.

The core investment question, agentically framed

Given this company's structure, resources, and opportunity space, what would a rational actor in its position most plausibly do, and how fast can it convert that intent into economic reality?

This is not decomposable into any single base layer. It requires the opportunity space (Synthetic Futures), the conversion capability (Deep Insight), the capital efficiency and resource constraints (Financial), the competitive response (Synthetic Futures and Deep Insight), the trajectory evidence (the hypothesis record itself), and the assumed objective function the company is pursuing. Threadweave is the only place where all of these meet, and the agentic framing is what makes the meeting coherent rather than just multivariate.

Intent, the assumption every projection carries

Every Threadweave projection carries an explicit `intent_assumption`, the typed objective function the company is presumed to be pursuing.

npv_max (default). Maximise the net present value of all future cashflows to shareholders, under known constraints. The standard assumption for public-equity finance, defensible because it is what management is legally and commercially obligated to pursue.

founder_vision_weighted. Maximise progress toward a stated long-horizon objective the founder has publicly committed to, with NPV as a secondary constraint rather than the primary objective.

control_preservation. Maximise the probability that current control structure (founder, family, dual-class, controlling shareholder) is preserved over the horizon, with NPV as a constraint.

political_or_regulatory_objective. Optimise for a non-financial objective that overrides shareholder NPV (state-owned enterprises, mission-driven nonprofits in name only, companies whose strategic value to a sovereign trumps their financial returns).

other. Free-form rationale required, with cited evidence.

The intent assumption is overridable per entity when evidence supports a non-default. The override requires a logged citation pattern (which deltas, which insights, which management statements). Threadweave's projection is run with the assumed intent; deviations between projected behaviour and observed behaviour become deltas to the entity's `entity_intent_profile` (described in §21).

Output shape

Threadweave's output is a versioned `threadweave_path` row per entity per run. Each row contains horizons (1y, 3y, 5y), branches with probability weights, conversion metrics (how fast does this entity historically convert intent into outcome on similar paths), competitor shadows (how does projected competitor behaviour shape this entity's available paths), and key uncertainty nodes (the points along the path where the projection is most sensitive to assumptions).

Bidirectional projection

Threadweave operates bidirectionally. **Forward** from current state under intent: where can this entity go. **Reverse** from structural futures back to current state: what would have to be true for this entity to occupy a future position the structural future predicts will exist. Both directions are first-class outputs.

The reverse direction matters because it surfaces preconditions. If the structural future for vertical-AI radiology predicts a dominant player at \$20B revenue by 2032, Threadweave reverse-projects from that position to ask: what would Company A need to do, build, acquire, or change between now and then to occupy it? The required preconditions are themselves deltas (new beliefs about Company A's necessary path) and become watch-items.

Reliability per horizon tier

Threadweave's reliability on any given entity is, in part, a function of how much scored judgment has accumulated at each horizon tier. An entity with twelve resolved short-horizon predictions, four resolved mid-horizon, and zero resolved long-horizon has high short-tier Threadweave reliability and low long-tier reliability. The reliability is published per horizon per entity; it is what tells the customer how much weight to put on the 1y vs the 5y projection.

§14 Support agents, Resolver Orchestrator Forecaster Evaluator

Four support agents wrap the six-layer stack. They are not layers in the stack; they are the control plane that makes the stack addressable, coherent, scorable, and closed.

Resolver. Single-tenant at the identity layer. Attaches incoming claims to canonical entities using LEI-primary, GLEIF-synced identity resolution (§16). Handles mergers, spin-offs, parallel quotations, ticker reuse, and name changes. Writes to Identity. Hands resolved claims to the Orchestrator. Single-writer to Identity; single-reader from the raw evidence channel.

Orchestrator. The committee chair. Plans dispatch (which layers does this claim touch), routes, holds proposed deltas until cross-validation clears, runs the critique pass on genuinely ambiguous cases, commits deltas + belief_updates atomically, schedules sweeps. Writes to Delta, Belief_update (in v4), and triggers Baseline materialisation through the replay engine. The only writer of authoritative semantic state. Implemented on Opus.

Forecaster. Triggered by forecastable hypotheses from the Dot Connector. Writes the prediction: direction, probability, horizon, mechanism, falsifier, mechanism evidence requirements (the specificity gates from §15). Writes to Prediction, write-once. Prediction shape is typed by the prediction taxonomy in §15. Implemented on Sonnet.

Evaluator. Closes the loop. Independent of all other agents (does not read originating reasoning chains). Reads only prediction text, pre-stated mechanism evidence requirements, and resolution-window evidence. Runs as an ensemble (multiple invocations with different prompt seeds, different evidence-window scopes). Writes outcome verdicts, mechanism verdicts, and (in v4) belief-update verdicts on the three tracks (posterior calibration, update-direction correctness, diagnosticity calibration). Writes to Outcome, Calibration, and write-once retrospective fields on Delta, Belief_update, Insight, Hypothesis, and Prediction. Implemented on Opus for verdict reasoning, with deterministic scoring computation in typed code.

What changed from v3.1

The Orchestrator no longer critiques in isolation; it invokes CrossValidator as a dedicated step. The Forecaster takes hypothesis rows, not raw deltas, as its trigger; every prediction is backed by an explicit hypothesis with falsifier and mechanism already authored. The Evaluator is horizon-tier-aware and (in v4) belief-aware: calibration is computed per (entity, subagent, horizon tier, belief category), not flat. Long-horizon work is not punished for being unresolved early.

What did not change

Access discipline. The Orchestrator is still the only writer to Delta and Belief_update. The Evaluator is still the only writer of retrospective scores into Delta, Belief_update, Insight, Hypothesis, and Prediction, and write-once at that. The Resolver is still single-tenant at the identity layer. The five epistemic rules of §04 still hold: every delta and every belief_update at wall-clock time, every prediction immutable, every calibration snapshot versioned, every retrospective pass labelled and non-authoritative, every material update naming the belief it moved.

§15 Scoring methodology, what “scored by reality” actually means (amended in v4)

PREDICTION TYPES · HOW EACH IS SCORED			
PREDICTION TYPE	A TYPICAL CLAIM	SCORING RULE	RESOLUTION
BINARY DIRECTION type 1 of 4 confidence $\in [0, 1]$	"Next earnings will revise guidance downward."	Brier score $(p - o)^2$ lower is better	Outcome is true or false
DISCRETE PROBABILISTIC type 2 of 4 vector summing to 1	"Of {beats, meets, misses} consensus, the probability is {0.40, 0.45, 0.15}."	Multi-class Brier + log-score $-\log p_{\text{realised}}$ both reported per agent	Outcome is one of a finite set
CONTINUOUS type 3 of 4 distribution, not point	"5y operating cashflow CAGR, central estimate 9.2%, 80% interval [6.5%, 12.1%]."	CRPS Continuous Ranked Probability Score generalises Brier to distributions	Outcome is a real number
STRUCTURED mechanism + magnitude + horizon type 4 of 4 three components, falsifier	"Margin compression of ~150bps within 2 quarters, driven by supplier shock."	Decomposed: direction · binary · Brier mechanism · binary horizon · binary in-range magnitude · in-band	Outcome resolves into 4 components scored separately, aggregated per agent

FIGURE 8 · FOUR PREDICTION TYPES, EACH WITH ITS OWN SCORING RULE AND RESOLUTION CRITERION. THE STRUCTURED TYPE CARRIES A FALSIFIER AND RESOLVES INTO FOUR COMPONENTS SCORED SEPARATELY.

The Evaluator in §14 is described as the agent that matches outcomes to predictions and writes calibration. Both descriptions hide the actual mechanics. This section specifies what scoring rule applies to which prediction type, how partial resolution is handled, what happens to predictions whose outcomes never arrive, how mechanism attribution distinguishes right-answer-for-wrong-reason cases, and (in v4) how belief-update quality is scored as a parallel track alongside prediction outcome.

This is the section that turns the architecture’s central commercial claim, *scored by reality*, from rhetoric into engineering. A reader who finds the rest of the architecture rigorous but discovers “Brier scores” as the entire treatment of how reasoning is evaluated will discount everything else. The methodology has to be specified and defended.

Prediction types, a taxonomy

Predictions in the system are not all the same shape. Four types, each scored differently.

Binary direction. “The next earnings release will revise guidance downward.” Outcome is true or false. Confidence is a single probability in [0, 1]. Scoring rule: Brier score $(p - o)^2$, lower is better.

Discrete probabilistic. “Of {beats, meets, misses} consensus next quarter, the probability is {0.40, 0.45, 0.15}.” Outcome is one of a finite set. Confidence is a probability vector summing to 1. Scoring rule: multi-class Brier (sum-of-squared-errors over the probability vector against the one-hot outcome) and log-score ($-\log p_{\text{realised}}$), both reported. Log-score is more punishing for confident wrong answers; Brier is more familiar. Both per-agent so users with different loss preferences can read calibration their own way.

Continuous. “5y operating cashflow CAGR, central estimate 9.2% with 80% interval [6.5%, 12.1%].” Outcome is a real number; confidence is a distribution. Scoring rule: Continuous Ranked Probability Score (CRPS), the integral of the squared difference between the predicted CDF and the empirical CDF (a step function at the realised value). CRPS reduces to mean absolute error for point predictions and to Brier for binary outcomes; the right family for this taxonomy.

Structured (mechanism + magnitude + horizon). The form most Dot Connector hypotheses take: a directional claim, a mechanism, a falsifier, and a bounded horizon. Resolution is decomposed: did direction prove right, did mechanism prove right, did horizon prove right, was magnitude in range. Each component scores separately. Aggregate “structured score” is the geometric mean of the components, which weights all four equally and penalises a single failure proportionally.

Partial resolution, when an outcome triggers some but not all of a prediction

Structured predictions resolve component-by-component. If an outcome confirms direction but contradicts mechanism, the prediction is recorded as `resolved_partial` with the specific decomposition: `direction=confirmed, mechanism=falsified, horizon=confirmed, magnitude=in_range`. Calibration aggregates each component separately; a partial resolution does not collapse to either “resolved” or “unresolved” but contributes to four distinct calibration tracks.

The architectural commitment: partial resolution is the typical case, not the exception. Predictions about complex systems rarely come true in exactly the form they were written. Forcing every outcome into a binary verdict throws away the information that distinguishes a system that is wrong about mechanism from one that is wrong about direction. The decomposition is the calibration signal.

Unresolvable predictions, when the question becomes moot

Some predictions never resolve cleanly. The entity is acquired before the horizon expires. The market regime shifts so the original mechanism no longer applies. The prediction’s specific falsifier is rendered un-checkable by an exogenous event. Three handlings:

Marked unresolvable. Flagged with a recorded reason. Excluded from calibration aggregates so as not to penalise agents for outcomes that were never their question. Counted in a separate `resolution_rate` metric.

Resolution rate as a signal. An agent whose predictions are systematically unresolvable is producing predictions that are not actually predictions, they are observations dressed up as falsifiable claims. Resolution rate is a per-agent diagnostic that surfaces this. A low resolution rate triggers prompt review, not silent calibration drift.

The “non-decision” trap. An adversarial implementation could mark every wrong prediction as unresolvable to keep calibration looking clean. The architectural mitigation: the unresolvable flag can only be set by the Evaluator on a defined trigger (acquisition event, regime-shift event explicitly logged, falsifier becoming structurally un-checkable), never by the predicting agent itself. The unresolvable rate is a published per-agent metric subject to review.

A note on horizon vocabulary: the schema enum `horizon_tier` uses three values (short, mid, long) mapping to nominal horizons of approximately 1y, 3y, and 5y respectively. Threadweave projects at the literal year horizons; calibration aggregates at tier granularity. The mapping is approximate, not strict: a 14-month prediction is short-tier, a 4-year projection is long-tier.

Calibration aggregation

Calibration is computed per (entity, agent, horizon-tier) cell and (in v4) also per (entity, agent, horizon-tier, belief-category). Published in three forms:

Time-weighted mean score. The headline number. Brier for binary, CRPS for continuous, geometric mean of components for structured. Time-weighted with exponential decay (recent predictions count more) so an agent's calibration tracks its current state, not its full history. Decay half-life is configured per agent and per horizon tier; conservative defaults at launch (6-month half-life for short-horizon, 18-month for mid, 36-month for long).

Reliability diagram. The canonical presentation. Predicted probabilities binned, realised frequencies plotted against bin midpoints. Perfect calibration is the diagonal. Deviations from the diagonal show where an agent is over- or under-confident at specific probability ranges.

Confidence intervals on the calibration estimate. A calibration score with no uncertainty band conflates a hot streak with genuine skill. Bootstrap confidence intervals are published on every calibration metric. An agent with 12 resolved predictions and a 0.18 mean Brier might have a 95% CI of [0.08, 0.31]; that is a different signal from an agent with 2,400 predictions and the same mean. Users see the CI alongside the point estimate.

Outcome-to-prediction matching

An implementation note: the Evaluator's outcome verdict is implemented as deterministic Python. Falsifier matching, Brier/CRPS computation, decomposed component scoring, and verdict writing are pure code with no LLM call. The mechanism verdict (described next) retains the LLM ensemble. The split is architectural: outcome is operations on numbers; mechanism is judgment over evidence chains.

When an outcome arrives, the Evaluator runs a matching pass that resolves all open predictions whose falsifier the outcome triggers:

1. The outcome is recorded with its observed values (direction, magnitude, mechanism if attributable, timestamp).
2. The Evaluator queries open predictions for the entity within the outcome's relevant horizon window.
3. For each open prediction, the falsifier is evaluated against the outcome. The falsifier is a typed condition; matching is deterministic and re-runnable.
4. Predictions whose falsifier matches are scored according to type and recorded with verdict, components (for structured), and the outcome that resolved them.
5. Predictions whose falsifier does not match remain open. They are not penalised for non-resolution; they remain pending until their horizon expires or another outcome resolves them.
6. Predictions whose horizon expires without resolution are marked `resolved_horizon_expired` and scored conservatively (the falsifier was about something specific, the horizon passed, the absence of the falsifying event is itself a kind of resolution).

One outcome can resolve multiple predictions; one prediction stays open until its specific falsifier is triggered. The system makes specific claims with specific falsifiers, and resolution is per-falsifier, not per-outcome.

Mechanism attribution, right answer wrong reason

MECHANISM ATTRIBUTION · 2 × 3 VERDICT MATRIX				
		MECHANISM VERDICT →		
		CONFIRMED <i>mechanism_confirmed</i>	UNCONFIRMED <i>mechanism_unconfirmed</i>	CONTRADICTED <i>mechanism_contradicted</i>
		FULL CREDIT	PARTIAL CREDIT	PENALISED
↑ OUTCOME VERDICT	CONFIRMED <i>verdict.confirmed</i> (outcome arrived as predicted)	Reasoning matched reality. Outcome confirmed, mechanism confirmed. <i>This is what calibration should reward.</i>	Outcome confirmed. Evidence is silent on whether the mechanism operated.	Right answer, wrong reason. Outcome confirmed, but mechanism did not operate as stated.
	FALSIFIED <i>verdict.falsified</i> (outcome arrived contrary)	RARE / NOTED Outcome contrary to prediction, but mechanism operated as stated. <i>Stronger force overrode it. Logged as agent insight.</i>	PENALISED Standard wrong-prediction case. Falsifier triggered in the expected direction.	PENALISED × 2 Wrong outcome and contradicted mechanism. <i>Calibration penalty plus prompt review trigger.</i>

Mechanism attribution scoring is independent of outcome scoring; both verdicts coexist on every prediction.
The "right answer, wrong reason" cell is the architecture's primary defense against accidental calibration drift.

FIGURE 9 · MECHANISM ATTRIBUTION MATRIX, OUTCOME VERDICT × MECHANISM VERDICT, WITH CALIBRATION TREATMENT PER CELL. THE "RIGHT ANSWER, WRONG REASON" DIAGONAL IS WHAT MAKES MECHANISM ATTRIBUTION WORTH THE ENGINEERING COST.

Outcome scoring tells you whether the prediction matched reality. It does not tell you whether the prediction’s reasoning matched reality. A prediction can be confirmed by an outcome that arrived for entirely different reasons than the prediction stated. A prediction can be falsified by an outcome that arrived despite the predicted mechanism actually operating, because some other factor dominated. Calibration that ignores this distinction trains on noise.

Every resolved prediction therefore receives a `mechanism_check` verdict in addition to its outcome verdict. Three states:

`mechanism_confirmed`. The outcome arrived and the stated mechanism is supported by the evidence chain between emission and resolution. The agent reasoned correctly. The case calibration should reward.

`mechanism_unconfirmed`. The outcome arrived but the evidence chain is silent on the stated mechanism. The mechanism may have operated, may not have, the available evidence does not say. Soft signal: not a failure, not a clean win. Calibration treats it as partial credit.

`mechanism_contradicted`. The outcome arrived but the evidence chain shows the stated mechanism did not operate, or operated in the opposite direction, or was overwhelmed by a different factor that drove the outcome. Right-answer-for-wrong-reason. The prediction is recorded as outcome-correct but mechanism-failed; calibration penalises it commensurately.

The Evaluator runs the mechanism check as a second pass after the outcome verdict is recorded.

The 2×3 verdict matrix

	mechanism_confirmed	mechanism_unconfirmed	mechanism_contradicted
outcome_confirmed	Full credit. Reasoning matched reality.	Partial credit. Outcome confirmed, mechanism silent.	Penalised. Right answer, wrong reason.
outcome_falsified	Rare/noted. Falsifier triggered but mechanism operated as stated. Stronger force overrode it.	Penalised. Standard wrong-prediction case.	Penalised ×2. Wrong outcome and contradicted mechanism.

Mechanism confirmation rate

For each (entity, agent, horizon-tier) cell, calibration aggregates publish a new metric alongside Brier/CRPS/decomposed-score: `mechanism_confirmation_rate`, the fraction of outcome-confirmed predictions whose mechanism was also confirmed. An agent with high outcome calibration but low mechanism confirmation rate is silently overfitting to noise; the gap is itself a diagnostic. A persistent gap triggers prompt review, not silent calibration drift.

The reverse case (high mechanism confirmation but lower outcome calibration) indicates an agent that reasons well but emits predictions on questions where the mechanism is operating but is being overwhelmed by exogenous factors. That is a different kind of agent failure (selection of question, not quality of reasoning) and is treated separately.

Mechanism attribution in practice

Mechanism attribution is a load-bearing architectural commitment, and the most likely place where the discipline could turn out aspirational rather than enforced. The honest concern: financial outcomes are over-determined. A stock moves; was it the predicted mechanism, an unrelated FX move, an inventory write-down, a peer-group rotation? The Evaluator has to make a call, and that call is genuinely hard.

Five architectural commitments make this operationally tractable rather than aspirational.

Mechanism specificity gates upstream. Predictions whose stated mechanism is too vague to verify are rejected before emission. A new validator (lightweight, prompt-based, runs as a Forecaster post-check) examines whether the mechanism names specific evidence patterns that would constitute confirmation or contradiction. Vague mechanisms like “demand softness” are rejected with a refinement request; specific mechanisms like “supplier X cost increase of Y% propagating to gross margin Z bps within W months, observable in cost-of-revenue line item” are accepted. Most of the downstream attribution problem dissolves when the mechanism is specified well; the upstream gate is the highest-leverage intervention.

Pre-stated falsification evidence requirements. Every prediction, on emission, names the specific evidence patterns in the resolution window that would confirm the mechanism, contradict it, or be insufficient to judge. Not abstract: which line items, which document types, which thresholds. The prediction record stores this as a typed field (`mechanism_evidence_requirements`). The Evaluator’s job becomes mechanical, check whether the named evidence appeared in the resolution window, rather than interpretive. The Evaluator can still report “evidence was insufficient to judge,” which is a legitimate verdict, but it cannot rationalise an answer the evidence does not support.

Independent Evaluator architecture. The Evaluator never reads the agent’s reasoning chain that produced the prediction. It reads only three inputs: the prediction text, the pre-stated `mechanism_evidence_requirements`, and the resolution-window evidence. The most important commitment in the section. Without independence, mechanism attribution collapses into self-confirmation: the same chain of reasoning that produced the prediction has motivated reasoning to confirm it. The Evaluator is a separate Temporal activity with no access to the originating agent’s run-log.

Ensemble verdicts with disagreement-as-signal. Mechanism verdicts run as ensembles: multiple Evaluator invocations with different prompt seeds, different evidence-window scopes (narrow vs wide), different model versions. The architecture treats Evaluator disagreement as informative rather than as something to average away. If three of five runs say `confirmed` and two say `unconfirmed`, the verdict is `unconfirmed` with a noted ambiguity flag, not `confirmed` with confidence 0.6. The architecture commits to noisy mechanisms being honestly noisy in the substrate.

Unconfirmed as the default expectation. The architecture commits to `mechanism_unconfirmed` being the most common verdict, not the rare exception. Most predictions resolve outcome-confirmed-mechanism-unconfirmed because evidence is genuinely silent on attribution: the outcome happened, multiple plausible mechanisms could have caused it, and the evidence chain in the resolution window does not discriminate. `confirmed` and `contradicted` are the discrimination signals; `unconfirmed` is the honest default. Calibration metrics report all three states with their relative frequencies, not just the binary outcome verdict.

A worked Evaluator example

To make this concrete, walk through one prediction from emission to verdict.

Setup. Mid-cap European industrial supplier, `entity_id` EX-MID-IND-1. Three deltas arrive on 2026-Q1: a Deep Insight delta about supplier exposure (DI-1742), a Financial Model delta about margin compression (FM-988), and a Synthetic Futures delta about the entity’s market cone narrowing. The Forecaster proposes a prediction.

Prediction emitted (T+1h).

Claim: Q3 reported gross margin will compress by 120-180 bps versus Q2, with 80% probability.
Horizon: 90 days (Q3 reporting date).
Mechanism: Primary supplier Z's announced 8% cost increase (effective Q2) propagates to cost-of-revenue at approximately 60% pass-through given the 60-day inventory turnover.
Falsifier: Q3 reported gross margin within ±50 bps of Q2.
Mechanism evidence requirements:
- Cost of revenue / revenue ratio rises ≥150 bps QoQ in Q3 reported financials.
- Management commentary in Q3 earnings call references supplier-cost pressure as material driver.

Specificity gate (T+1h). The mechanism specifies a named supplier, a quantified pass-through ratio, observable line items, and a threshold for confirmation. The validator accepts.

Resolution window (T+90d). Q3 reports land. Reported gross margin compresses 142 bps versus Q2, inside the predicted band. Outcome verdict: `confirmed`.

Evaluator runs (T+90d + 4h). Five Evaluator invocations dispatch in parallel. Each receives only the prediction text, the pre-stated mechanism evidence requirements, and the Q3 resolution-window evidence (the 10-Q filing, earnings call transcript, and any news in the window). None receives the originating agent reasoning.

Evidence the Evaluators examine.

Cost of revenue / revenue ratio QoQ change. Pre-stated requirement: ≥ 150 bps rise. Observed: +163 bps. Met.

Management commentary on supplier-cost pressure. Pre-stated requirement: explicit reference as material driver in Q3 earnings call. Observed: CFO acknowledged “input cost pressures, particularly from our raw material supply base,” without naming the specific supplier. Partially met, ambiguous.

Confounding factors. Three of the Evaluators flagged a Q3 inventory write-down of €18M, which would also push cost-of-revenue higher independently of supplier costs. The write-down is disclosed but not separately reconcilable from the supplier-cost effect.

Ensemble verdicts. Two Evaluators returned `confirmed` (line-item evidence is consistent with the predicted mechanism, management commentary directionally supports it). Three returned `unconfirmed` (the inventory write-down confound prevents clean attribution; the management commentary did not specifically name the supplier the prediction identified).

Final verdict. `mechanism_unconfirmed` with ambiguity flag. The outcome was correct; the mechanism is plausibly operative but evidence does not discriminate between the predicted mechanism and the inventory-write-down confound. The Forecaster’s mechanism-confirmation rate at the 90-day horizon receives no credit for this prediction; the agent gets calibration credit for the outcome verdict only.

What the substrate records. The full Evaluator ensemble (five verdicts, the evidence each examined, the disagreement structure) is written as a single resolution record. Future prompt review of this Forecaster will see “right answer, ambiguous mechanism” as a recurring pattern, not a confident confirmation. Cross-entity mechanism patterns read this record alongside others to surface “agents tend to call this kind of mechanism right when an inventory write-down is also present,” itself an insight the Dot Connector can act on.

The mechanism is not “confirmed” by majority vote. It is “unconfirmed” honestly because the evidence chain does not discriminate. The architecture’s value is in being honest about ambiguity, not in producing confident verdicts.

Cross-entity mechanism patterns

When `mechanism_contradicted` verdicts cluster across entities, agents, and time, the cluster itself is signal. The Dot Connector reads the mechanism-check ledger during its scheduled sweep alongside the canonical substrate. A pattern of “agents in this layer kept being right for the wrong reason about this kind of entity over this window” is exactly the cross-layer insight the Dot Connector exists to surface. The output is an insight that names the recurring reasoning failure, not a causal claim about the world.

The factor co-occurrence ledger

Beyond mechanism attribution at the per-prediction level, the substrate maintains a separate ledger of factors observed when reasoning about each entity. Each delta or insight that explicitly invokes a factor (a regulatory regime, a competitor action, a macro variable, a structural condition) records it in `entity_factor_observation`, joined to the entity, the date, the originating delta or insight, and (when resolved) the outcome it preceded.

The ledger is a frequency record, not a causal claim. It does not assert that factor F causes outcome O; it records that the system observed F preceding O at a given rate over a given history. The distinction matters: causal claims require assumptions the data cannot support, but frequency records support the kind of pattern detection that generates useful hypotheses without overcommitting to causal structure.

The Dot Connector reads the factor ledger during sweeps. When a factor's co-occurrence rate with a particular outcome class shifts meaningfully, either because a previously rare factor is becoming common, or because a previously reliable co-occurrence is breaking down, that shift is a candidate insight. The factor taxonomy itself grows over time as agents introduce new factors; the ledger does not require a closed taxonomy.

Why this is not a causal model

A reader familiar with causal-inference literature might recognise the shape of what is being described and ask why the architecture stops short of explicit causal-graph modelling: directed acyclic graphs of factor influences with learned edge weights, do-calculus over interventions, structural causal models. The reasons are deliberate.

Causal claims about company dynamics in finance have a poor historical track record. Most published causal models do not replicate. The graveyard of failed strategies is full of beautiful causal graphs that turned out to be overfitting. Committing the architecture to causal claims the data cannot defend would be the first thing in this document a sophisticated reader could legitimately call hand-wavy.

Markov assumptions are wrong for the dynamics this system tracks. Path dependence is the violation of the Markov property, and path dependence is the architecture's central premise (§01, §03). A literal Markov model is the wrong mathematical object.

Mechanism attribution and factor co-occurrence are strictly weaker claims than causal modelling, and weaker claims are more defensible. Both produce useful diagnostic signals (right-for-wrong-reasons detection, co-occurrence drift detection) without committing to causal structure the data cannot yet support. They are also strictly compatible with adding causal modelling later: the substrate they require is a strict subset of what a future causal-graph layer would need.

The roadmap to causal modelling is captured in the open questions of §25; the present commitment is to the narrower, defensible primitives. This is the line the system holds at v4.

Threadweave reliability function

For each entity and each horizon tier, the Threadweave reliability score is a function of three inputs:

1. The mean calibration score for the entity at that tier (lower is better).
2. The number of resolved predictions feeding that calibration (more is better, with diminishing returns above approximately 50).
3. The width of the calibration confidence interval (narrower is better, reflects data sufficiency).

The function combines these into a reliability score in [0, 1]. The exact functional form is configured per launch and recalibrated as data accrues; it does not depend on prior assumption, only on scored data. When data is insufficient for a tier (fewer than approximately 10 resolved predictions), reliability falls back to "insufficient" rather than computing a fragile estimate. Threadweave consumers see "insufficient" explicitly, not a misleadingly high or low number.

Belief-update quality scoring (new in v4)

v4 adds belief-update quality as a parallel scoring track alongside Brier/CRPS/decomposed-structured/mechanism-attribution. Three scoring questions, each with its own resolution path.

Track 1 • Posterior calibration. For each belief, posterior confidence at time t is later resolved against the eventual reality of the underlying claim, or against the future evidence the belief made expectations about. Resolution uses Brier or CRPS depending on whether the belief is binary or distributional. Beliefs accumulate calibration curves the same way agents do. *What this scores:* was the system’s confidence in this belief well-calibrated against later reality?

Track 2 • Update-direction correctness. When a `belief_update` moves the posterior away from the prior (probability mass shifts between rival explanations), the direction of that shift is later checked against the evidence pattern that arrived in the resolution window. An update that moved probability toward “rising competitive pressure” is correct in direction if the resolution-window evidence pattern is consistent with that explanation, regardless of whether the magnitude was right. *What this scores:* did the system shift probability toward the right rival explanation?

Track 3 • Diagnosticity calibration. For each `belief_update`, the diagnosticity recorded at step 04c is a forward claim: this evidence should distinguish among rival explanations to *this degree*. After resolution, a diagnosticity verdict is written: was the evidence as diagnostic as claimed, more so, or less? Persistent over-claiming of diagnosticity is a per-agent diagnostic, the same way mechanism over-confirmation is. *What this scores:* did the system correctly judge how much this evidence was going to teach it?

Aggregation, v4 extension

Each track is published per (entity, agent, horizon-tier) and per (belief category, agent, horizon-tier). Aggregates are versioned alongside v3 calibration snapshots. The same horizon-tier vocabulary applies. The Evaluator writes belief-update verdicts independently of outcome verdicts: a prediction can resolve `outcome_confirmed` / `mechanism_unconfirmed` while the underlying belief_updates resolve with high posterior calibration on Track 1 and with under-claimed diagnosticity on Track 3. The four scoring tracks (outcome, mechanism, posterior calibration, update direction, diagnosticity calibration, with mechanism counted as one) can each say something different about the same closed prediction; that is the point.

The internal-vs-external precision boundary

Internally, the system uses likelihood ratios and probabilities at full numeric precision. Externally, the customer-facing surface uses confidence bands unless calibration data supports decimals. The default external presentation:

Internal probability	External band
≥ 0.85	High
0.65 to 0.85	Medium-high
0.40 to 0.65	Medium
0.20 to 0.40	Medium-low
< 0.20	Low

The same convention applies to falsifier distance (far / watch / medium / near / triggered, see §41) and posterior layers (business / valuation / knowability / tail-risk). The architecture commits to no false Bayesian precision in the customer surface. The internal precision is what makes the back-end scoring honest; the external bands are what make the product credible. An agent whose calibration accumulates enough resolved predictions to support narrower published bands earns them empirically; bands tighten with data, never with rhetoric.

What this section does not cover

Selection bias on what gets predicted (does an agent emit predictions only on questions it expects to answer well?), the meta-question of “is calibration the right metric at all” for some downstream uses, and adversarial robustness of the scoring rule are deliberately out of scope. Each is a research question more than an architectural one. The architecture commits to making the data needed to study these questions available; it does not commit to specific answers.

A claim that cannot be scored is not a claim. A scoring rule that cannot be defended is not a moat. This section is what makes the central promise mean what it says.

§16 Identity resolution, anchoring claims to canonical entities

Every commitment in this document depends on one assumption: that a claim arriving today about “Apple” resolves to the same entity as a claim arriving in three years about “AAPL,” and that neither silently merges with a different company that happens to share the ticker after a reassignment. The Resolver is what makes that assumption load-bearing. This section specifies how.

Identity resolution is a problem with sharp edges. Tickers get reused. Names change. ISINs are per-security, not per-issuer. Companies merge, spin off, delist, and re-list. A naïve resolver (match on name, accept the answer) produces a system that quietly contaminates its own track record. A correct resolver is invisible when it works and audit-recoverable when it does not.

The identity stack

Three levels of identifier, in decreasing order of authority:

entity_id (internal). An opaque UUID generated by the system the first time an entity is registered. Every other table in the substrate joins on it. Never changes for the life of the entity, even through mergers, name changes, or restructurings. Never exposed externally.

LEI (canonical external). Legal Entity Identifier, ISO 17442. The 20-character GLEIF-issued code that uniquely identifies a legal entity globally. The single external identifier StahlTrace treats as authoritative. Every entity has exactly one current LEI; succession is tracked through GLEIF’s `succession_event` records, not by reassigning the LEI.

Aliases (everything else). Tickers, ISINs, CUSIPs, SEDOLs, common names, legal names, abbreviations. Stored in a separate table with an explicit validity window: the date range during which the alias was a correct way to refer to this entity. A claim referring to “Facebook, Inc.” in 2020 resolves to the same entity as a claim referring to “Meta Platforms, Inc.” in 2024 because both names are aliases for the same `entity_id`, with non-overlapping validity windows.

The structural commitment: tickers, ISINs, and names are never primary keys. They are search aids that resolve to LEI, which resolves to `entity_id`. A reader who understands only this paragraph understands the most important architectural decision in this section.

The resolver pipeline

The Resolver is a service, not an LLM agent. The pipeline is deterministic logic over an authoritative reference store, with a queue for ambiguous cases. An incoming claim carries some subset of {LEI, ISIN, ticker+exchange, name, jurisdiction}. The Resolver attempts matches in this order, stopping at the first confident match:

1. **LEI direct match.** If the claim carries an LEI, look it up directly. If found and active, return. If found and superseded by a GLEIF succession event, follow the succession chain to the current entity. Confidence: 1.0.
2. **ISIN → LEI lookup.** Look up the security in the alias table to find the issuing LEI, then resolve. Confidence: 0.99.
3. **Ticker + exchange + date.** Look up the alias scoped to that exchange, with the claim's `evidence_time` falling within the alias's validity window. Tickers are never matched globally or without a date scope. Confidence: 0.95.
4. **Name fuzzy match (with jurisdiction filter).** If only a name is provided, fuzzy match against legal-name and common-name aliases, scoped by jurisdiction if known. Match score above 0.92 auto-resolves. Score in 0.75 to 0.92 is held in the bootstrap queue with the top 3 candidates surfaced. Below 0.75 is treated as a probable new entity.
5. **Bootstrap queue.** Anything below confidence threshold is held without committing. Resolution decisions are confirmed (initially manually, longer-term through ensemble cross-checks against multiple authoritative sources) before commit. Sustained backlog signals that the resolver's confidence thresholds need recalibration.

Every successful resolution writes back to the substrate: the alias actually matched on, the alias's validity window at time of match, the confidence score, the timestamp. A resolution decision made today can be recovered, audited, and replayed against an updated alias table without losing the original answer.

The alias table, provenance with validity windows

```
TABLE alias (  
  alias_id          text PK  
  entity_id         text FK-identity  
  alias_kind        enum      -- lei | isin | ticker | cusip | sedol | legal_name | common_name  
  alias_value       text      -- the literal string  
  exchange         text      -- for ticker; null for global aliases  
  jurisdiction      text      -- for legal_name disambiguation  
  valid_from        date  
  valid_to          date      -- null = currently valid  
  source           text      -- gleif | sec_filing | bloomberg | manual  
  superseded_by    text      -- alias_id of the alias that replaced this one  
  confidence        numeric  -- for fuzzy-matched aliases  
)  
INDEX ix_alias_value ON (alias_kind, alias_value, valid_from, valid_to)  
INDEX ix_alias_entity ON (entity_id)
```

The validity window is the load-bearing field. When ticker AAPL on NASDAQ is reassigned in some hypothetical future, the original Apple alias gets `valid_to = reassignment_date`, and a new alias row is created with `valid_from = reassignment_date`. A 2026 claim about “AAPL” still resolves to Apple because the claim’s `evidence_time` falls within the Apple alias’s validity window. A 2031 claim resolves to whoever holds the ticker in 2031.

Corporate actions, six categories, six policies

The Resolver runs a daily sync against GLEIF’s published succession events plus exchange notices for ticker-level changes. Each category is handled differently:

Mergers, simple (one survivor, one absorbed). Auto-processed. The absorbed entity’s `is_active` flag flips to false; its baseline becomes a sealed historical record; all aliases get `superseded_by` pointers to the surviving entity’s aliases. New claims about the absorbed entity resolve to the surviving entity with an explicit note in the resolution metadata. The merger is recorded as a delta of magnitude `regime` on the surviving entity, with the absorbed entity’s prior baseline attached as evidence.

Mergers, complex (multi-party, with divestitures). Detected automatically from GLEIF, then held for additional resolution before the baseline transfer is committed. Multi-party mergers may have multiple “successors” in GLEIF, and only structured attribution can correctly assign which lines of business went where. The Resolver holds claims about the affected entities in a transitional state until disambiguation is complete.

Spin-offs and demergers. GLEIF records the new entity as a separate LEI but does not say which parts of the parent’s baseline (which Deep Insight content, which Financial model lines, which insights, which beliefs) should travel with the spin-off. The architecture commits to writing the attribution explicitly so that future reads can see what was inherited and what was retained. Initial attribution at launch is structured: the v4 belief vector makes this cleaner than v3 because beliefs carry `category` fields that tag them as parent-relevant or subsidiary-relevant.

Delistings. Two sub-cases. *Delisted but operating* (taken private, listed elsewhere, restructured): the security alias gets a `valid_to` date but the entity continues. *Wound up* (liquidated, dissolved): the entity itself is archived. GLEIF’s `RegistrationStatus` field distinguishes these. Auto-processed; both write a delta of magnitude `regime`.

Name changes. Auto-processed. The old name alias gets a `valid_to` date; a new alias is created for the new name. The LEI persists. Recorded as a delta of magnitude `minor` unless other factors elevate it.

Parallel quotations. Multiple ISINs and tickers per LEI is the steady state, not an event. The Resolver selects one listing as the primary for tracking purposes (price, volume, technicals), keeping all other listings as cross-reference aliases.

Primary listing, most liquid wins with stickiness

For entities with multiple active listings, one is designated primary:

1. Compute median 90-day trading volume for each listing, denominated in USD at prevailing FX.
2. Select the highest. This is the primary.
3. Apply stickiness: the primary only changes if a different listing has been more liquid for at least 6 consecutive months. This prevents short-term liquidity spikes from flipping the primary.
4. Recompute monthly. Primary-listing changes are recorded as a delta of magnitude `minor`.

The primary listing affects: which price series SF/04 Technicals (and the v4 price_signal layer in §40) uses; which volume profile is canonical; which exchange-specific corporate actions (splits, dividends) drive baseline events. It does not affect: identity (the entity is the same regardless of where it trades), financials (consolidated accounts apply globally), or evidence ingestion (filings from any jurisdiction are valid).

Customer override: the sealed client codification layer permits per-customer primary-listing overrides. A London fund tracking BHP via the LSE listing rather than the ASX listing can pin the primary to LSE in their own namespace, without affecting the canonical system.

Cold start, registering a new entity

LEI in the claim. Best case. Look up the LEI in the GLEIF database (offline replica, refreshed daily), pull the entity's official record, create the Identity row. Aliases are seeded from GLEIF; additional aliases accrete as future claims arrive.

ISIN or ticker+exchange in the claim, no LEI. Use exchange or vendor data feeds (Bloomberg, Refinitiv, OpenFIGI) to look up the issuer's LEI. If found, proceed as above. If not, hold in the bootstrap queue.

Only a name in the claim. Search GLEIF by name + jurisdiction with a fuzzy-match score. High-confidence matches register automatically. Mid-confidence matches enter the bootstrap queue for ensemble disambiguation. Low-confidence matches are held without committing. The most common source of bootstrap-queue entries, and the most common source of subtle entity-conflation bugs if shortcut.

Identity schema

```
TABLE identity (
  entity_id      text PK           -- internal UUID, never changes
  lei            text UNIQUE       -- canonical external identifier
  legal_name     text              -- current legal name
  jurisdiction    text              -- ISO 3166-2 country code
  registration_status enum         -- active | inactive | merged | dissolved
  parent_entity_id text FK-identity -- for subsidiaries with their own LEI
  primary_listing_id text FK-alias  -- the alias designated as primary for tracking
  created_at     timestampz
  superseded_at  timestampz NULL   -- non-null if entity was merged into another
  superseded_by  text FK-identity NULL
  gleif_last_synced timestampz
)
INDEX ix_identity_lei    ON (lei) WHERE registration_status = 'active'
INDEX ix_identity_active ON (registration_status, jurisdiction)
```

What this section does not cover

Private companies without LEIs, entities outside finance (where LEI may not apply), and reference data licensing (Bloomberg, Refinitiv) are out of scope. The first is handled with internal-UUID-only entities flagged `no_lei` until a LEI is issued. The second is a v5 question. The third is a commercial concern, not an architectural one.

A claim resolves to an entity. An entity stays itself across mergers, listings, and renamings. Without that, every other commitment in this document is rhetorical.

§17 The runtime, how the agents actually execute

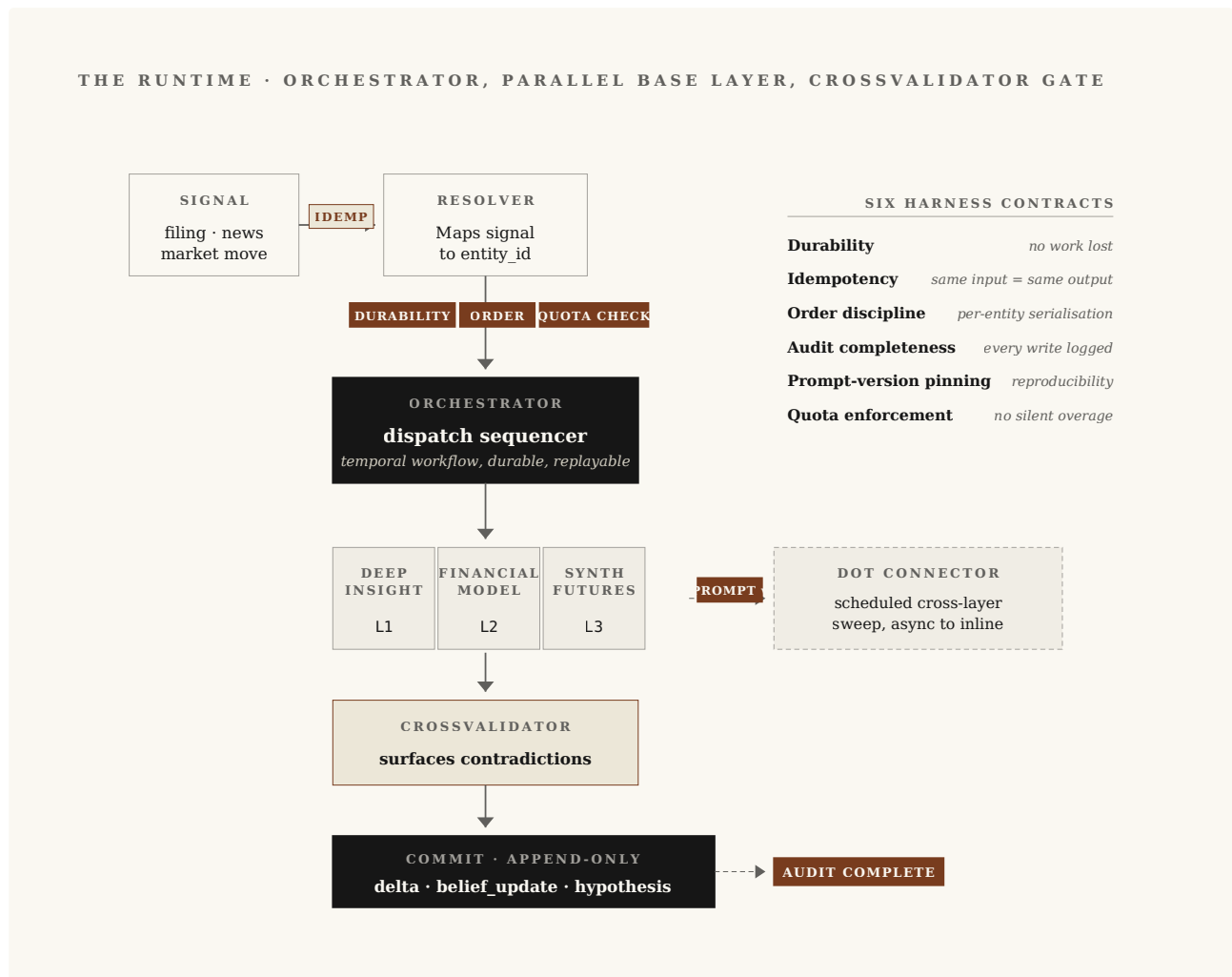


FIGURE 10 · RUNTIME FLOW FROM SIGNAL TO COMMIT, WITH THE SIX HARNESS CONTRACTS ANNOTATED AS GATES. THE DOT CONNECTOR RUNS ON A SEPARATE SCHEDULED CADENCE, ASYNCHRONOUS TO THE INLINE LOOP.

The agent layer in §07 through §14 describes what each agent does. This section describes what runs them: the durable execution layer, the queue, the retry policy, the idempotency contract, and the way the system stays correct when things fail.

Everything in the canonical core is append-only and timestamped. That commitment is meaningless if the runtime can lose work, double-write, or scramble the order in which deltas reach the substrate. The harness is not glamorous, but it is load-bearing: a wrong choice here invalidates the moat the rest of the architecture is designed to hold.

The six harness contracts

The harness is bound by six constraints the architecture imposes on it. These are not preferences; they follow from the epistemic rules in §04 and the access discipline in §22.

Durability. No agent invocation, baseline commit, prediction emission, belief_update, or scored outcome can be lost in transit, even if the LLM provider, the database, or the orchestrator fails mid-step. Every state transition is recoverable from a persistent record.

Idempotency. Every retry of every agent step must produce the same downstream state as the first attempt. A delta that has already been committed cannot be committed again. A prediction that has already been emitted cannot be re-emitted with a later timestamp. A belief_update written in one transaction cannot be duplicated by retry. A nightly sweep that ran successfully cannot be re-run as if it hadn't.

Order discipline. Two signals about the same entity arriving within the same second must be serialised through the Orchestrator for that entity, not run in parallel. Across entities, parallelism is fine. The harness enforces per-entity serialisation without enforcing global serialisation.

Audit completeness. Every agent invocation writes to the Run-log store before any side effect is committed downstream. If the side effect succeeds, the log records success. If it fails, the log records the failure. There is no path through the system that produces a substrate write without a corresponding log entry.

Prompt-version pinning. Every agent invocation records the exact prompt hash and model version that produced the output. A change to a subagent's prompt is a new version; old outputs are never silently re-attributed to the new version. Belief-extractor prompt versions are pinned the same way; a belief authored under version X is tagged with X, and the audit table of failed extractions is also pinned to its version.

Quota enforcement. Every dispatch that will call an LLM consults the quota service before the activity begins. Tenant-scoped activities check tenant token and entity-count budgets; canonical activities check the canonical capacity envelope. Blocked dispatches surface a structured error rather than silently failing. Usage is metered at activity granularity so retries do not double-charge.

The stack

Orchestration: Temporal. Every agent dispatch is a Temporal workflow. Every subagent invocation is a Temporal activity. Workflows are deterministic and replayable; activities are at-least-once with idempotency keys. Per-entity serialisation is enforced via Temporal task queues keyed by `entity_id`. The Orchestrator is the workflow that owns the dispatch sequence; subagents are activities it calls.

State: Postgres + pgvector. All canonical state (identity, baseline, delta, **belief**, **belief_update**, **alternative_explanation_set**, insight, hypothesis, prediction, outcome, calibration, threadweave_path, **update_case**) lives in Postgres. Schema migrations are append-only where possible. Foreign keys are enforced. Every write is wrapped in a transaction; transactions either commit fully or not at all. The Run-log store also lives in Postgres, with an OpenSearch projection for full-text search.

Coordination: Redis. Short-lived coordination only. Distributed locks for the rare cases where Temporal's task-queue serialisation is not sufficient (e.g. cross-entity invariants during the nightly Dot Connector sweep), and a small cache of frequently-read calibration snapshots. Redis is not a source of truth and never holds state that is not reproducible from Postgres.

Blobs and search: S3 + OpenSearch. Raw evidence payloads (filings, transcripts, large jsonb documents, **update_case payloads**) live in S3, addressed by content hash. OpenSearch indexes evidence text and agent learning notes for retrieval. Both are projections of state that ultimately resolves to Postgres pointers; neither is independently authoritative.

Time series: TimescaleDB. Outcomes and (in v4) `price_signal` rows live in TimescaleDB hypertables, on the same Postgres engine, for efficient time-windowed queries.

The idempotency contract

Every agent invocation carries an `idempotency_key` derived deterministically from its inputs. For an inline subagent invocation, the key is `hash(entity_id, claim_id, subagent_id, prompt_version)`. For a nightly Dot Connector sweep, the key is `hash(entity_id, sweep_date, sweep_kind)`. For a Forecaster prediction emission, the key is `hash(hypothesis_id, prompt_version)`. **For a belief_update commit (v4), the key is `hash(entity_id, claim_id, belief_id, layer, prompt_version)`.**

The key is checked before the work is done. If a record exists for that key, the result is returned without re-invoking the agent. The system is safe to retry indefinitely without producing duplicate deltas, duplicate insights, duplicate predictions, or duplicate belief_updates. The cost is that any change to the inputs that should produce a new output must explicitly bump the relevant version field. Silent re-runs that produce different answers are not possible. The right trade.

Retry policy

Resolver, Orchestrator. Exponential backoff, 5 retries over ~2 minutes, then dead-letter. Failure here blocks the inline loop; alerting is immediate.

Base-layer subagents (Deep Insight, Financial, Synthetic Futures). Exponential backoff, 3 retries over ~90 seconds, then mark the proposed delta as `subagent_failed` and continue with whatever subagents did return. CrossValidator runs against the partial set. The Orchestrator records the gap.

CrossValidator. Exponential backoff, 3 retries. If CrossValidator fails after retries, no delta commits, the inline loop fails closed. Failure here is louder than failure in any individual subagent.

Belief extractor (v4). Exponential backoff, 3 retries. If the ensemble fails to converge after retries, the originating delta is held; the gap is logged as an extraction failure in `belief_extraction_attempts`. The delta is re-classified non-material if no existing belief absorbs it on the next dispatch attempt.

Dot Connector (sweep). Long-running activity with checkpointing. If interrupted, resumes from last checkpoint. If a single entity in the sweep fails, the sweep continues for other entities and the failed entity is requeued for the next sweep cycle. No retry budget on the sweep itself.

Forecaster, Evaluator. Exponential backoff, 5 retries over ~5 minutes. Both write through idempotency keys, so retries are safe even if the activity actually succeeded but the response was lost in transit.

The nightly sweep, concrete mechanics

The Dot Connector's scheduled cross-layer synthesis is implemented as a Temporal cron workflow. Runs at 02:00 UTC each day for tier-1 (active) entities, weekly on Sundays for tier-2 (quiet) entities, monthly for tier-3 (dormant) entities. The tier of each entity is recomputed weekly from delta volume, hypothesis age, and user attention signals.

For each entity in the sweep, the workflow:

1. Acquires the per-entity Temporal task-queue lock.
2. Reads the substrate snapshot for the entity at `sweep_start_time`, including the v4 belief_update history.
3. Invokes the Dot Connector activity with idempotency key `hash(entity_id, sweep_date)`.

4. If insights are emitted, writes them to the Insight table; if any are `forecastable=true`, writes child hypotheses to the Hypothesis table.
5. Triggers downstream Forecaster activities for any new forecastable hypotheses.
6. Releases the lock and writes a sweep audit record.

Idempotency means a partially-completed entity from a prior run will not produce duplicate insights when re-run.

Failure modes

LLM provider fails or rate-limits. Activity-level retry with backoff. Provider failover wraps a primary and secondary provider, fails over after N failed attempts on the primary. If both providers fail, the activity dead-letters and the workflow surfaces a visible error.

Postgres unreachable. All workflows pause. No partial writes are committed because every multi-step state transition is wrapped in a single transaction. When Postgres returns, workflows resume from where they left off.

Temporal itself fails. Worker processes restart from durable state. In-flight activities resume; in-flight workflows replay deterministically from event history. No state is lost. No deltas are double-committed.

A subagent returns malformed output. Schema validation rejects the output before any commit. The activity is retried with a regenerated prompt. If retries exhaust, the subagent's contribution is dropped from the inline loop and the gap is logged.

Two claims about the same entity arrive simultaneously. Both are accepted at the Resolver. Both enter the Orchestrator's per-entity task queue. Temporal serialises them by arrival order. The second one runs against the baseline that resulted from the first, not against a stale snapshot. There is no race.

What this section does not cover

Deployment topology, infrastructure-as-code, secrets management, network architecture, and observability dashboards are out of scope. Operational concerns that will evolve with scale.

The harness is not the moat. But a wrong harness invalidates the moat. The contract above is what makes every other commitment in this document mean what it says.

§18 Security model, authentication authorization encryption response

The architecture so far treats access discipline as a data-level concern (the access matrix in §22) and tenant isolation as a runtime concern (§17 + Part V). Neither addresses how a human or service gets into the system, or what happens when their access should not be there anymore. This section specifies the security primitives the rest of the architecture depends on.

Security in this design is not a feature; it is a precondition. The seal in Part V, the audit trail in §17, the per-tenant isolation in §27, and the metering in §31 are all only as strong as the authentication and authorization that gate them.

Authentication

Self-service authentication. OIDC-based, via a managed identity provider (Auth0, Clerk, Stytych, or equivalent; choice deferred). Email + password with mandatory email verification. MFA optional for builders and viewers, mandatory for admins. Session duration: 24 hours active, with rolling refresh.

Enterprise SSO. SAML 2.0 and OIDC both supported for Enterprise-tier customers. Customer's identity provider is the authentication source; StahlTrace consumes assertions and maps to internal user records. JIT user provisioning supported; SCIM for managed user lifecycle deferred to v4.1.

Service-to-service authentication. mTLS between internal services with short-lived certificates (24-hour rotation, automated). API tokens for customer-side integration; the only long-lived credentials in the system, blast radius bounded by per-tenant scoping.

Authorization at the human level

Three user roles, parallel to the agent-level access matrix:

Admin. Full tenant control: create/delete agents, manage users, change subscription tier, view billing, view audit log, configure webhooks, generate API tokens. The tenant's billing-responsible role; one mandatory per customer.

Builder. Author and run agents in the tenant, configure validation rules, override Threadweave parameters, **author tenant-private beliefs**, subscribe and unsubscribe entities (within tier allowance), read tenant-scoped usage. Cannot manage users, cannot change billing, cannot generate API tokens.

Viewer. Read-only access to tenant outputs (agent results, predictions, insights, belief_updates, the customer's view of canonical entity traces). Cannot author agents, cannot subscribe entities, cannot read raw usage logs.

Role assignment is per-user-per-tenant. A user cannot belong to two tenants under the same identity; switching tenants requires separate user records. A deliberate constraint that simplifies the authorization model at the cost of some convenience for consultants who serve multiple customers.

Encryption

In transit. TLS 1.3 for all external traffic. Internal mTLS for service-to-service. No plaintext over any network segment.

At rest. AES-256 for database storage (Postgres, TimescaleDB, S3, OpenSearch). Per-tenant encryption keys for tenant-scoped data, with envelope encryption: a tenant master key encrypts per-table data keys, the master key is held in a managed KMS (AWS KMS or equivalent). Canonical-side data uses a separate canonical master key.

Key management. All encryption keys in a managed KMS; no plaintext keys in application memory beyond the lifetime of a single request. Key rotation is on a 90-day schedule for canonical keys, on customer-driven schedule for tenant keys (default 90 days, configurable to 30 days for compliance-sensitive customers).

Audit of access

Every read of tenant data by canonical operators is logged with a tamper-evident chain: the operator's identity, the elevation justification, the timestamp, the rows read, the duration of the elevation. The audit log is itself stored with append-only semantics; modifications to the audit log require a separate elevation that is itself audited. Customer admins can view all canonical reads of their tenant data through the audit dashboard.

Customer-initiated reads. Standard tenant operations. No elevation needed; standard authentication suffices.

Internal operator reads. StahlTrace internal personnel reading tenant data (for support, debugging, or incident response) requires explicit elevation: a documented reason, a time-bounded grant (default 4 hours, hard maximum 24 hours), and the customer is notified before the elevation begins (synchronous notification for non-emergency, asynchronous for emergency with post-hoc audit). No standing access; every read is logged.

Canonical-side reads. Operators reading canonical-only data (no tenant scope) follow standard authentication. Audit captures the read but no elevation is required.

Token revocation and session management

Tenant-key revocation. A customer admin can revoke a tenant's encryption keys, which renders all tenant data unrecoverable. This is a destructive operation gated by multiple confirmations and a 24-hour cooling-off period for non-emergency revocations. Used in compromise scenarios where the customer needs to be sure no past data can be read.

API token revocation. Immediate. Tokens are checked against a revocation list on every request. Revoked tokens fail closed within seconds of revocation propagation.

Session revocation. Admin-initiated session termination forces re-authentication. Used when offboarding a user or when a session is suspected of compromise.

Compromise response

LLM provider API keys. Treated as compromised by default if any anomaly suggests it. Rotated automatically on a schedule and on demand. Activity audit logs allow post-hoc reconstruction of which prompts were sent under which key.

Database credentials. Held in KMS, never in code. Rotated on a 30-day schedule; emergency rotation on 1-hour timeline. Application reconnection is automatic.

Tenant compromise. If a tenant suspects their credentials are compromised, the response is: revoke all API tokens for the tenant, force re-authentication for all human users, audit log review for the suspected exposure window, optional tenant-key rotation if the customer wants to ensure encrypted data at rest is no longer readable with prior keys.

Canonical compromise. A canonical compromise is treated as an existential incident. The runbook (out of scope for this document) involves: isolating affected systems, preserving forensic state, customer notification (with regulatory disclosure where required), recovery from append-only logs, and post-incident review. Every commitment in this architecture depends on the canonical layer being uncompromised; the discipline of treating canonical compromise as existential is what keeps the discipline real.

What this section does not cover

Specific compliance certifications (SOC 2, ISO 27001, regional equivalents) are operational, not architectural. The architecture is designed to make those certifications achievable; pursuing them is a v4.1 commercial decision.

Security is the floor. Every commitment above this floor only holds if the floor holds.

Part III · The data substrate

Where the moat lives on disk. Eleven logical stores, implemented on a small physical stack. The load-bearing schemas from v3 carry forward; v4 adds five tables (`belief`, `belief_update`, `alternative_explanation_set`, `price_signal`, `update_case`) to support belief-level Bayesian updating. An access matrix that enforces the orchestrator pattern in code. Agents never touch SQL.

§19 Data acquisition, what feeds the substrate

The architecture's epistemic discipline depends on properties of the data the system cannot fabricate: reliable timestamping, deterministic entity resolution, durable provenance, and source-tier metadata. These are not operational concerns layered on top of the substrate, they are architectural commitments the substrate is built to enforce. Without them, append-only timestamping is meaningless, mechanism attribution is unverifiable, and calibration discipline collapses into theatre.

The four input streams

Launch coverage relies on four input streams. Each carries a different latency profile, a different provenance posture, and a different role in the substrate.

Annual filings (10-Ks and equivalents). Authoritative annual disclosure. SEC EDGAR for US-listed entities. Equivalent registers per jurisdiction: ESMA's European financial-disclosure database, Companies House (UK), AFM (Netherlands), BaFin (Germany), AMF (France). EDGAR is the canonical reference standard; equivalent registers are tier-equivalent for non-US entities. Latency: inline; webhook or polling on a 15-minute cadence; substrate-ready within an hour of public availability.

Quarterly disclosures (10-Qs and earnings calls). Quarterly structured filings via the same registers. Earnings call transcripts via S&P Capital IQ Transcripts (enterprise tier) or AlphaSense / Sentieo (lower price points). Latency: filings inline as for 10-Ks; transcripts inline-once-transcribed, typically 4 to 24 hours after the call. Provenance: filing URL plus accepted timestamp for filings; transcript provider plus call timestamp plus speaker attribution for calls.

Ten-year financial history. Normalised balance sheet, income statement, and cashflow statement covering at minimum a ten-year window per entity. S&P Capital IQ as primary working assumption; FactSet Fundamentals as alternative; LSEG Workspace as third option. Each provider normalises line items across companies and reporting standards, the work the architecture does not do internally and explicitly does not try to. Latency: scheduled rebuilds, never inline; nightly full pulls for active entities, weekly for tier-2 entities, monthly for the long tail. Raw vendor responses retained as evidence blobs in S3 so deltas can be reconstructed if vendor data is later revised.

News flow. Real-time news with source-tier metadata. Dow Jones DNA as primary working assumption; Bloomberg News API as alternative; LSEG/Reuters News as third option. Free-tier development uses NewsAPI or equivalent with explicit understanding of degraded source-tier reliability. Latency: inline with severity-based prioritisation. Breaking news routes through the inline loop within seconds; routine news goes to the nightly Dot Connector sweep. Provenance: article URL, publication timestamp, source tier (regulatory wire, primary newswire, financial press, secondary press, blog/social), resolved entity references.

Reference data, the join key layer

Identity resolution (§16) requires authoritative reference data that maps entity-identifying signals to the system's canonical `entity_id`. Two free authoritative sources cover the launch universe:

GLEIF. Global Legal Entity Identifier Foundation. Authoritative LEI registry. Free, daily concatenated file download. Refreshed daily as a scheduled job. The source of truth for LEI-to-entity mapping.

OpenFIGI. Bloomberg's free financial-instrument identifier service. Maps tickers, ISINs, CUSIPs, and SEDOLs to FIGI codes which can then be cross-walked to LEIs. Used by the Resolver as the primary path from market signals to canonical entities.

Reference data is acquired by scheduled jobs, not inline. The Resolver caches recent lookups; the cache is refreshed nightly to absorb new listings, mergers, and ticker reuses.

Price data feed (extended in v4)

In v4, the price data feed is promoted from a downstream input to SF/04 Technicals to a first-class evidence stream feeding the `price_signal` table (§40 details the routing; this section names the acquisition).

Price acquisition. End-of-day OHLCV per primary listing per entity, plus benchmark indices (regional broad-market index, sector index, factor portfolios) and FX rates. Vendor: same data providers as ten-year financial history (S&P Capital IQ, FactSet, LSEG). Latency: nightly batch for end-of-day data; intraday is deferred (per the v3 deferral list).

Decomposition. Raw return is computed for each entity-window pair. Market-adjusted, sector-adjusted, and factor-adjusted returns are computed by subtracting the benchmark or factor-portfolio return from the raw return. The residual is what remains after these adjustments, plus volume z-score, plus persistence (does the move hold at +5d, +20d), plus event-specificity (did peers move similarly), plus reflexivity class (does this entity's stock price feed back into its operating reality, see §40). All computed deterministically in Python, no LLM call; written to `price_signal` rows.

The Resolver as quality gate

Every artifact arriving at the substrate passes through the Resolver, whose responsibilities extend beyond entity resolution to architectural quality gates. Six gates, each enforced before any artifact reaches the canonical store:

Schema validation. The artifact matches the expected structure for its source (XBRL for filings, structured JSON for vendor financials, conforming article schema for news). Malformed artifacts route to a reject queue with explicit failure reason.

Timestamp sanity. The artifact's claimed publication timestamp must be in the past, must be after the entity's earliest known activity, and must be consistent with any other timestamp evidence. Future timestamps and impossible timestamps are blocking errors.

Entity-resolution confidence. The Resolver computes a resolution confidence score per artifact. Scores below threshold (default 0.95 for unambiguous LEI matches; 0.85 for ticker-based matches with corroborating context) route to the bootstrap queue rather than auto-attaching. Auto-attached artifacts carry the resolution confidence as metadata.

Deduplication. The Resolver hashes the artifact content and checks against existing substrate entries before committing. Duplicate artifacts (the same press release wired through three news services) are recorded once with multi-source provenance, not three times.

Source-tier annotation. Every artifact carries a source-tier tag (`regulatory_authoritative`, `primary_newswire`, `financial_press`, `secondary_press`, `vendor_normalized`, `unverified`). Downstream calibration can be stratified by source tier; predictions emitted from regulatory-authoritative sources are scored separately from those resting on secondary press.

Language detection. Non-English artifacts are tagged with detected language and routed through a translation pipeline before delta extraction. Translation quality is itself a metadata field.

Artifacts that fail any gate route to an explicit reject queue. Failures are not silent. Recurring failure patterns surface as architectural improvements rather than silent data loss.

Why acquisition is verifiable artifacts, not LLM retrieval

A sophisticated reader will ask: given that LLMs can already summarise public companies, retrieve recent news, and approximate financial figures via training data and on-demand search, why does StahlTrace operate a separate acquisition layer at all?

LLMs deliver real value on three categories of work, and the agent layer uses them extensively for exactly these: reading and structuring source documents the system has acquired (turning a 10-K into typed deltas, identifying factor mentions, proposing hypotheses); resolving entity references in unstructured text; synthesising across many sources.

What LLMs do not deliver, and what the architecture's epistemic discipline categorically requires:

Verifiable artifact provenance. An LLM can summarise “what's in Novo Nordisk's most recent 10-K.” It cannot produce the 10-K itself as a verifiable artifact with an EDGAR URL, an SEC-stamped accepted timestamp, and an unaltered source document the system can replay later for audit. Append-only depends on the artifact existing as a fact in the world before the system observed it. LLM summaries are not facts in the world; they are generated approximations.

Reliable specific numbers. LLMs hallucinate specific figures with calibrated-sounding confidence. For finance customers, confidently-wrong specific numbers are the failure mode that destroys trust. Vendor-normalised financial history is acquired precisely because the architecture cannot afford LLM-generated numbers entering the substrate as if they were measurements.

Coverage you can audit. An acquisition layer guarantees coverage by virtue of having actually pulled the artifacts; an LLM-only system has coverage by hope.

Predictions before outcomes. The architecture's calibration discipline depends on predictions being timestamped before the outcomes that resolve them. An LLM with web access at prediction time may inadvertently retrieve information from after the prediction window, contaminating the calibration. The acquisition layer's strict timestamping isolates predictions from outcomes by construction.

The agents read. The acquisition layer records. LLMs are excellent agents and inadequate recorders. The architecture separates the two by design.

Customer-supplied feeds, forward compatibility

Some customers will want to feed proprietary research, internal estimates, expert-network notes, or alternative data into the same pipeline as canonical sources. The customer-supplied feed channel inherits the same Resolver gates (schema validation, timestamping, entity resolution, deduplication, source-tier annotation) but carries a distinct source tier (`customer_supplied`) and is sealed to the customer's tenant per §29.

Customer-supplied feeds never contaminate canonical calibration. They feed customer-tenant agents and customer-tenant Threadweave projections, but their predictions and outcomes resolve against customer-tenant calibration only.

What is deliberately deferred

Foreign-language disclosure beyond translation. Non-English filings are accepted via equivalent registers and routed through translation, but jurisdictions where filing structures differ materially from XBRL-style standards (Japan EDINET, China CSMAR with full coverage, smaller emerging markets) are deferred.

Intraday market data. End-of-day pricing, fundamental ratios, and event-driven price moves are in scope. Tick-level intraday is deferred to v5+.

Alternative data. Satellite imagery, credit-card aggregates, web-traffic analytics, app-store data. Commercially valuable but fundamentally different in shape from the four launch streams. They require their own acquisition and quality discipline. Deferred to operations as customers ask for it.

Social media as primary source. Monitored only when it surfaces in regulatory filings, primary-newswire coverage, or executive statements meeting the §38 Public Actor Trace guardrail. Not a primary source; the noise-to-signal ratio at scale defeats calibration discipline.

§20 Data substrate, eleven stores one ID space (amended in v4)

THE DATA SUBSTRATE · TABLES JOINED BY ENTITY_ID · v4 EXTENSION

IDENTITY entity_id canonical_name aliases lei / ticker PK text text[] text	EVIDENCE evidence_id entity_id source · evidence_time payload PK FK tstz S3 blob	BASELINE baseline_id entity_id · version prev_version_id state · is_current PK int int jsonb · bool	DELTA ★ CORE delta_id entity_id · layer applies_to changes · weight PK enum FK jsonb
INSIGHT insight_id entity_id emitted_at parent_deltas statement PK FK tstz FK[] text	HYPOTHESIS hypothesis_id parent_insight entity_id statement · falsifier horizon_tier PK FK FK text enum	PREDICTION pred_id parent_hyp horizon probability · verdict PK FK int enum	OUTCOME outcome_id entity_id observed_at direction · magnitude PK FK tstz enum
BELIEF belief_id entity_id · category current_confidence primary_falsifier expected_evidence expected_price_reaction PK FK · enum numeric text jsonb jsonb	BELIEF_UPDATE update_id belief_id · delta_id prior · posterior channel likelihood_ratio falsifier_state PK FK · FK numeric enum numeric enum	ALT_EXPLANATION_SET set_id belief_update_id explanations priors · posteriors dominant_shift diagnosticity_claim PK FK jsonb[] num[] text numeric	
PRICE_SIGNAL signal_id entity_id · observed_at expected_band · actual unexpected_residual PK FK · tstz numeric numeric	UPDATE_CASE case_id belief_update_id resolution_window three track verdict PK FK int days jsonb	CALIBRATION + PATH entity_id · subagent horizon_tier · brier_mean path_id · reliability inputs pulled · updated_at PK num jsonb text[]	

v4 SUBSTRATE EXTENSION · 5 NEW TABLES

Eleven load-bearing tables. All joined by entity_id. Belief layer is operationally separate but structurally embedded.

FIGURE 3 · TABLES OF THE V4 DATA SUBSTRATE, JOINED BY ENTITY_ID. THE FIVE NEW TABLES INTRODUCED BY V4 (BELIEF, BELIEF_UPDATE, ALT_EXPLANATION_SET, PRICE_SIGNAL, UPDATE_CASE) CARRY THE RUST-ACCENT HEADER. ELEVEN LOAD-BEARING TABLES IN TOTAL.

Different data has different access patterns. Forcing everything into one table is the classic mistake; spreading it across ten technologies is the opposite. **Eleven logical stores in v4** (up from seven in v3.1), implemented on roughly three physical systems. All joined by `entity_id`, the only truly shared identifier.

Store/01 · Identity

Canonical, high-read. Entity IDs, canonical names, aliases, LEIs, tickers, merge history. Every other store joins against this. Shape: relational + pgvector. Mutable: no (aliases append). Size: tiny.

Store/02 · Evidence

Raw, append-only. Every claim that entered the system, normalised and timestamped. The source material. Shape: S3 blob + Postgres metadata. Mutable: no. Size: large, linear.

Store/03 · Baseline projection

Materialised projection. Per-entity structured dossier, materialised from the delta ledger by the replay engine. Every version kept as a snapshot for read latency, but no snapshot is independently authoritative. B-48221/v14 is what the baseline looked like at that moment, recoverable by replaying deltas DI-1 through whatever delta-id closed v14. Shape: PG jsonb snapshots + replay engine. Mutable: no (snapshot rows append-only). Size: medium, reconstructible from ledger.

Store/04 · Delta

The core, multi-indexed. Every operational change ever made, by which subagent, which layer, with what evidence, classification, magnitude, weight, explanation. **In v4, every material delta is linked to one or more belief_updates via a shared transaction.** Shape: PG partitioned. Mutable: no (Evaluator writes score once). Size: grows fastest.

Store/05 · Insight + Hypothesis

The Dot Connector's ledger. Two tables. Insights are the primary output of the nightly sweep, with cross-layer evidence chains. Hypotheses are children: emitted only when the insight is forecastable, with falsifier and horizon. **In v4, evidence chains can cite belief_update rows alongside delta rows.** Shape: PG + partial index. Mutable: verdict and score written once. Size: small.

Store/06 · Prediction

Time-indexed. Every forecast. Horizon, mechanism, probability, falsifier, parent hypothesis, mechanism evidence requirements. Indexed by resolution date. Shape: PG + partial index. Mutable: no (verdict written once). Size: small.

Store/07 · Outcome

Ground truth. Prices, earnings, guidance revisions, rating changes, corporate actions. Time-series. Joined to predictions for scoring. Shape: TimescaleDB hypertable. Mutable: no. Size: large.

Store/08 · Calibration

Derived, hot. Per-entity, per-subagent, per-horizon-tier calibration curves. Derived from delta ↔ outcome joins. Versioned snapshots. **In v4, also per-belief-category aggregates and per-track aggregates (posterior calibration, update direction, diagnosticity).** Shape: PG + Redis cache. Mutable: snapshots preserved. Size: tiny, high-read.

Store/09 · Threadweave path

Versioned path states. Per-entity probabilistic path through opportunity space, at each run. Horizon tiers, branch probabilities, reliability. Shape: PG jsonb versioned. Mutable: no. Size: medium.

Store/10 · Belief (new in v4)

Per-entity belief vector. Each row a named claim about the entity, with current confidence, expected evidence, primary falsifier, falsifier-distance ladder state, and pointers to the deltas and updates that have moved it. Shape: PG + pgvector (for semantic clustering of belief statements across entities). Mutable: `current_confidence` and `falsifier_distance` update on every `belief_update`; `statement`, `expected_evidence`, and `category` are append-only-with-supersession (a belief can be retired and replaced; the old row stays with `status=retired`). Size: medium, grows roughly linearly with entity coverage × beliefs-per-entity (target 8 to 25 active beliefs per entity).

Store/11 · Belief_update + Alternative_explanation_set + Update_case (new in v4)

The belief-update ledger and its supporting tables. Three tables logically grouped:

- **Belief_update.** Every posterior move on every belief, with prior, posterior, evidence decomposition, diagnosticity claim, dominant rival shift, falsifier-distance change, valuation impact. Shape: PG partitioned. Mutable: no, except `retrospective_score`, `update_direction_verdict`, `diagnosticity_verdict` written once by the Evaluator on resolution. Size: grows with material updates, comparable to Delta.
- **Alternative_explanation_set.** Rival explanations as first-class objects. One row per (belief, event) where probability mass shifted between rivals. Shape: PG. Mutable: no. Size: medium, smaller than `Belief_update`.
- **Update_case.** Packaged trajectories: prior state, evidence packet, agent action, system review (ensemble disagreement, diagnosticity verdict, mechanism verdict, cross-validator notes), accepted update, later outcome, calibration result. The §35 corpus is materialised here. Shape: PG (metadata) + S3 (evidence-packet payloads). Mutable: no, except `later_outcome_id` and `calibration_result` written once by the Evaluator on resolution. Size: large, grows with closed material updates.

Store/12 · Price_signal (new in v4)

Decomposed price evidence. End-of-day per (entity, window) row with raw return, market-adjusted return, sector-adjusted return, factor-adjusted return, residual, volume z-score, persistence, event specificity, reflexivity class. Time-series. Joined to `belief_updates` via the §40 routing rules. Shape: TimescaleDB hypertable on the same engine as `Outcome`. Mutable: no. Size: medium, grows with entity coverage × trading days × window types.

Plus an Audit / Run-log store

Every agent invocation, every tool call, every Orchestrator decision is logged with: workflow id, activity id, agent identifier, model and prompt-version pin, input hash, output reference, token counts in and out, cost in micros, latency, and parent-run linkage. The run log is the substrate for replay (§17), prompt-version comparison (§36), and post-incident audit. It lives outside the relational schema because its access pattern is fundamentally different — write-heavy, read-rare-but-deep, full-text searchable — and because keeping it physically separate prevents accidental denormalisation back into the canonical tables.

```

TABLE run_log (
  log_id          text PK
  workflow_id     text          -- Temporal workflow run id
  activity_id     text          -- Temporal activity id
  parent_run_id   text NULL FK-run_log -- nested invocation parent, where applicable
  tenant_id       text
  agent_kind      enum          -- canonical | customer
  agent_id        text          -- canonical agent name OR customer agent id
  entity_id       text NULL     -- when applicable
  emitted_at      timestampz
  prompt_version  text          -- pinned prompt hash
  model           text          -- e.g. claude-opus-4 | claude-sonnet-4
  input_ref       text          -- S3 pointer; large inputs not inline
  input_hash      text          -- sha256 for replay determinism check
  output_ref      text          -- S3 pointer
  output_hash     text
  tokens_in       integer
  tokens_out      integer
  cost_usd_micros bigint        -- 1e6 micros = $1; integer to avoid float
  latency_ms      integer
  outcome_kind    enum          -- success | tool_error | timeout | refusal |
                                -- schema_violation | quota_exceeded
  emitted_rows    jsonb        -- {table: [row_ids...], ...} the writes this
                                -- invocation produced through the orchestrator
)
INDEX ix_runlog_workflow ON (workflow_id, emitted_at)
INDEX ix_runlog_agent_time ON (agent_id, emitted_at DESC)
INDEX ix_runlog_entity_time ON (entity_id, emitted_at DESC) WHERE entity_id IS NOT NULL
INDEX ix_runlog_prompt_ver ON (prompt_version, agent_id, emitted_at DESC)
INDEX ix_runlog_failed ON (emitted_at) WHERE outcome_kind <> 'success'

```

Storage is split: row metadata in Postgres for the indexes, large input/output blobs in S3 addressed by content hash, full-text searchable copy in OpenSearch for incident investigation. The same content hash on input means the same prompt version on the same model produced the same output (or the same error), which is what makes the harness contract on idempotency (§17) auditable rather than aspirational.

Baseline as materialised projection

The v3.1 refinement holds: authoritative truth is **evidence + delta + belief_update + outcome only**. Baseline is a materialised projection, a cached current-state view computed by replaying the delta + belief_update ledger from inception. The system maintains versioned baseline snapshots for read performance, but every snapshot is reconstructible from the ledger and the replay path is the verification mechanism. A baseline snapshot that disagrees with the replay output is a bug in the snapshot, not a competing source of truth.

This refinement strengthens the architecture in three concrete ways. Schema evolution becomes safer: a change to delta interpretation can be validated by replaying historical deltas under the new interpretation and comparing against snapshots. Delta corrections become clean: when an evidence correction supersedes a delta, replaying the corrected ledger produces the corrected baseline automatically. Audit becomes provable rather than asserted: the audit story is no longer “we kept every baseline version” but “every baseline version is reproducible from the ledger.”

The Orchestrator gates delta and belief_update commits, but it does not write authoritative baseline state. It writes deltas and belief_updates; the replay engine derives baseline snapshots as a consequence.

Physical implementation

Most of this is one Postgres instance with pgvector (identity, baseline, delta, belief, belief_update, alternative_explanation_set, insight, hypothesis, prediction, calibration, threadweave, update_case metadata), plus TimescaleDB on the same engine (outcome, price_signal), S3 for blobs and audit logs, Redis for calibration cache. Orchestration runs on Temporal (see §17). Total infrastructure footprint at 100 entities estimated at \$750 to \$850 per month at v4 launch (up from v3.1's \$600 to cover belief-vector storage, update-case payloads, and price_signal time series).

§21 Schemas, the tables that carry the moat (*amended in v4*)

Eleven load-bearing tables in v4 (up from six in v3.1). Baseline and Delta are still the moat. Insight and Hypothesis make the Dot Connector's work addressable and scorable. Prediction and Outcome close the prediction learning loop. Threadweave path is the apex state. **Belief, Belief_update, and Alternative_explanation_set carry the belief-level Bayesian update layer. Price_signal carries decomposed price evidence. Update_case packages trajectories for the corpus.** Calibration is horizon-tier-aware and (in v4) belief-category-aware.

TABLE identity · the entity registry

```
TABLE identity (  
  entity_id          text PK          -- internal UUID, never changes  
  lei                text UNIQUE       -- canonical external identifier  
  legal_name         text              -- current legal name; historical names in alias  
  jurisdiction        text              -- ISO 3166-2 country code  
  registration_status enum              -- active | inactive | merged | dissolved  
  parent_entity_id   text NULL FK→identity -- for subsidiaries with their own LEI  
  primary_listing_id text NULL FK→alias  -- alias designated as primary for tracking  
  created_at          timestampz  
  superseded_at       timestampz NULL   -- non-null if entity was merged into another  
  superseded_by       text NULL FK→identity  
  gleif_last_synced   timestampz  
)  
INDEX ix_identity_lei          ON (lei)  
INDEX ix_identity_jurisdiction ON (jurisdiction, registration_status)  
INDEX ix_identity_active       ON (entity_id) WHERE registration_status = 'active'
```

The Resolver is the single writer. Identity is single-tenant: there is one canonical row per entity across all customer tenants. Aliases (LEIs, ISINs, tickers, CUSIPs, legal names) are kept in a separate `alias` table joined by `entity_id`, so that ticker reuse, share-class fan-out, and parallel quotations do not require splitting an entity.

TABLE *alias* · identifier history

```
TABLE alias (  
  alias_id          text PK  
  entity_id         text FK→identity  
  alias_kind        enum          -- lei | isin | ticker | cusip | sedol |  
                                -- legal_name | common_name  
  alias_value       text          -- the literal string  
  exchange          text NULL     -- for ticker; null for global aliases  
  jurisdiction       text NULL    -- for legal_name disambiguation  
  valid_from        date  
  valid_to          date NULL     -- null = currently valid  
  source            text          -- gleif | sec_filing | bloomberg | manual  
  superseded_by     text NULL FK→alias -- e.g. ticker reassignment  
  confidence        numeric      -- for fuzzy-matched aliases  
)  
INDEX ix_alias_entity      ON (entity_id)  
INDEX ix_alias_value_kind  ON (alias_value, alias_kind) WHERE valid_to IS NULL  
INDEX ix_alias_lei        ON (alias_value) WHERE alias_kind = 'lei'
```

TABLE *evidence* · the immutable inbound record

```
TABLE evidence (  
  evidence_id       text PK  
  entity_id         text FK→identity  
  tenant_id        text          -- canonical or customer tenant  
  source            enum          -- filing_sec | filing_local | transcript |  
                                -- press_release | regulatory_decision |  
                                -- patent_grant | court_filing |  
                                -- exchange_disclosure | news_wire |  
                                -- expert_call | private_data | other  
  source_id         text          -- vendor's stable id, where available  
  source_url        text NULL  
  ingested_at       timestamptz  -- wall-clock now() of ingest  
  evidence_time     timestamptz  -- when in world the underlying event occurred  
  jurisdiction       text NULL    -- where the event happened, if applicable  
  payload_kind      enum          -- text | pdf | structured_xbrl | audio_transcript |  
                                -- image | structured_json | other  
  payload_ref       text          -- S3 pointer; large blobs never inline  
  payload_hash      text          -- sha256 of raw payload  
  payload_bytes     bigint  
  language          text          -- ISO 639-1; null for non-text  
  redaction_class   enum          -- public | tenant_private | restricted  
  superseded_by     text NULL FK→evidence -- corrections supersede prior evidence  
  superseded_reason text NULL     -- correction | retraction | replacement  
)  
INDEX ix_evidence_entity_time ON (entity_id, evidence_time DESC)  
INDEX ix_evidence_ingest      ON (ingested_at DESC)  
INDEX ix_evidence_source_id   ON (source, source_id)  
INDEX ix_evidence_active      ON (entity_id, evidence_time DESC)  
                                WHERE superseded_by IS NULL
```

Evidence is append-only. Corrections do not overwrite; they append a new row that supersedes the prior. The Replay engine, when reconstructing baselines, walks the supersession chain and uses only the active evidence. `evidence_time` and `ingested_at` are kept separately because they answer different questions (what did the world do; when did we hear about it). They diverge constantly and that divergence is itself information (§04 epistemic rule on backdating).

TABLE *baseline* • materialised current state

```
TABLE baseline (
  baseline_id      text PK          -- e.g. 'B-48221/v14'
  entity_id        text FK→identity
  tenant_id        text              -- canonical or customer tenant
  version          int               -- monotonic per entity per tenant
  prev_version_id  text NULL FK→baseline
  produced_at      timestampz        -- wall-clock of materialisation
  produced_by_run  text              -- replay engine run id
  state            jsonb              -- full current-state snapshot:
                                   --   structure, capital, demand, supply,
                                   --   moat, regulation, fragility, sentiment
  state_schema_version text          -- 'baseline_v1' | 'baseline_v2' | ...
  belief_vector_snapshot jsonb        -- [v4] {belief_id: current_confidence, ...}
                                   --   for fast read; ledger remains source of truth
  derived_from_deltas text[] FK→delta -- deltas folded into this version
  derived_from_updates text[] FK→belief_update -- [v4] belief_updates folded in
  is_current       bool
)
INDEX ix_baseline_entity_current ON (entity_id, tenant_id) WHERE is_current = true
INDEX ix_baseline_entity_version ON (entity_id, version DESC)
INDEX ix_baseline_produced_at    ON (produced_at)
```

Baseline is a materialised projection, not source-of-truth (§20). The replay engine is the single writer. `state_schema_version` permits non-breaking schema evolution within a major version (`baseline_v1` → `baseline_v2`); breaking changes increment the document version (`v4` → `v5`) and trigger full replay. The `belief_vector_snapshot` is a v4 read-optimisation: it lets the Delta Feed surface entity belief state without joining belief and belief_update tables on every read. The ledger (delta + belief_update) remains authoritative; any disagreement is a bug in the snapshot.

TABLE *delta* · the operational core

```
TABLE delta (  
  delta_id          text PK  
  entity_id         text FK-entity  
  produced_by       text FK-subagent  
  layer             enum          -- insight | financial | futures | crossval  
  tenant_id         text  
  mode              enum          -- bootstrap | live  
  produced_at       timestampz   -- wall-clock now()  
  evidence_time     timestampz   -- when in world  
  applies_to       text FK-baseline  
  produces         text FK-baseline  
  domain            enum          -- capital, supply, demand, moat ...  
  class             enum          -- 6 classes  
  magnitude         enum          -- noise | minor | material | regime  
  tier              int  
  changes           jsonb        -- [{path, from, to}]  
  explanation       text  
  subagent_confidence numeric  
  orchestrator_weight numeric  
  context_only      bool          -- [v4] true if no belief_update attached  
  retrospective_score numeric    -- Evaluator · once  
  learning_note     text          -- Evaluator · once  
)  
INDEX ix_delta_entity_time ON (entity_id, produced_at DESC)  
INDEX ix_delta_layer       ON (layer, produced_at)  
INDEX ix_delta_material    ON (produced_at) WHERE magnitude IN ('material','regime')  
INDEX ix_delta_orphan      ON (produced_at) WHERE context_only = true  
                           AND magnitude IN ('material','regime')
```

The `context_only` flag is new in v4. It marks deltas that the belief journey could not attach to any belief; per epistemic rule 5 these are excluded from belief calibration aggregates. The orphan index surfaces them for diagnostic review.

TABLE belief · the named claim (new in v4)

```

TABLE belief (
  belief_id          text PK
  entity_id          text FK→entity
  tenant_id          text          -- canonical or customer tenant
  authored_at        timestampz
  authored_by        text          -- agent id (ensemble extractor or layer agent)
  statement          text          -- the named claim, single sentence
  category           enum          -- moat | growth_quality | capital_allocation |
                                   -- management_integrity | demand_durability |
                                   -- supply_resilience | regulatory_exposure |
                                   -- technology_position | structural_future |
                                   -- ...open-ended, curated by Dot Connector
  importance_weight   numeric      -- 0..1, how load-bearing in the thesis
  current_confidence  numeric      -- 0..1, last posterior
  expected_evidence   jsonb        -- structured: what should appear if belief is true
  primary_falsifier   text          -- the single observation that would break it
  falsifier_distance  enum          -- far | watch | medium | near | triggered
  -- four posterior layers, separately maintained where applicable (§41)
  business_posterior  numeric NULL
  valuation_posterior numeric NULL
  knowability_posterior numeric NULL
  tail_risk_posterior numeric NULL
  status              enum          -- active | retired | superseded
  superseded_by       text NULL FK→belief
  retired_at          timestampz NULL
  retired_reason       text NULL    -- stale_no_updates | falsifier_triggered |
                                   -- replaced_by_better | category_consolidation
)
INDEX ix_belief_entity_active ON (entity_id) WHERE status = 'active'
INDEX ix_belief_falsifier_near ON (entity_id) WHERE falsifier_distance IN ('near','triggered')
INDEX ix_belief_category ON (category, entity_id)
INDEX ix_belief_high_importance ON (entity_id, importance_weight DESC)
                                WHERE status = 'active'

```

TABLE *belief_update* · the posterior move (new in v4)

```
TABLE belief_update (
  update_id          text PK
  belief_id          text FK→belief
  delta_id           text FK→delta          -- shared transaction
  entity_id          text FK→entity
  tenant_id          text
  produced_at        timestamptz           -- wall-clock now()
  evidence_time       timestamptz           -- when in world
  prior_confidence    numeric               -- snapshot pre-update
  posterior_confidence numeric              -- new value
  prior_falsifier_dist enum                -- snapshot
  posterior_falsifier_dist enum
  evidence_packet_ref text                  -- S3 pointer
  evidence_reliability numeric              -- 0..1, source-tier × independence
  evidence_persistence enum                -- one_period | structural | unknown
  evidence_independence numeric            -- 0..1
  diagnosticity       numeric              -- 0..1, claimed at update time
  alt_explanation_set_id text FK→alternative_explanation_set
  likelihood_rationale text                -- which rival became more/less likely
  valuation_impact    jsonb NULL           -- per-posterior-layer change; see §41
  layer              enum                  -- insight | financial | futures | crossval
  -- three Evaluator verdicts, written once on resolution
  retrospective_score numeric NULL         -- Track 1: posterior calibration
  update_direction_verdict enum NULL       -- Track 2: correct | incorrect | ambiguous
  diagnosticity_verdict enum NULL          -- Track 3: as_claimed | over_claimed |
                                          -- under_claimed | unverifiable
)
INDEX ix_bu_belief_time  ON (belief_id, produced_at DESC)
INDEX ix_bu_entity_time  ON (entity_id, produced_at DESC)
INDEX ix_bu_unscored     ON (produced_at) WHERE retrospective_score IS NULL
INDEX ix_bu_layer        ON (layer, produced_at)
```

TABLE *alternative_explanation_set* · rival hypotheses as first-class (new in v4)

```
TABLE alternative_explanation_set (
  set_id            text PK
  belief_id         text FK→belief
  event_id          text                  -- the triggering claim/evidence cluster
  entity_id         text FK→entity
  tenant_id         text
  authored_at       timestamptz
  explanations       jsonb                -- [{label, prior_p, posterior_p,
                                          -- evidence_basis, forecastable,
                                          -- falsifier_if_forecastable}]
  dominant_shift     text                  -- which explanation gained the most mass
  diagnosticity      numeric              -- the set-level diagnosticity claim
  diagnosticity_verdict enum NULL         -- Evaluator · once
)
INDEX ix_aes_belief_time  ON (belief_id, authored_at DESC)
INDEX ix_aes_event       ON (event_id)
```

Rival explanations sit beside Hypothesis rather than under it because not every rival is currently forecastable. A rival that becomes forecastable later spawns a Hypothesis row pointing back to the explanation set.

TABLE *price_signal* · decomposed price as evidence (new in v4)

```
TABLE price_signal (
  signal_id          text PK
  entity_id          text FK→entity
  observed_at        timestampz          -- end-of-day or event close
  window            enum                 -- 1d | 5d | 20d | event_window
  raw_return         numeric
  market_adjusted_return numeric          -- return minus index return
  sector_adjusted_return numeric          -- return minus sector ETF return
  factor_adjusted_return numeric          -- residual after Fama-French style controls
  residual           numeric              -- the part the system cannot explain
  volume_z           numeric              -- volume z-score vs trailing
  persistence        numeric              -- fraction of move that holds at +5d, +20d
  event_specificity   numeric              -- 0..1, did peers move similarly
  reflexivity_class   enum                -- non_reflexive | mildly_reflexive |
                                          -- strongly_reflexive
  source_event_id     text NULL           -- if move was triggered by a known event
  expected_reaction_band jsonb NULL       -- Forecaster's pre-event band, if any (§40)
  unexpected_residual numeric NULL        -- residual minus expected_reaction_band midpoint
)
INDEX ix_ps_entity_time      ON (entity_id, observed_at DESC)
INDEX ix_ps_residual_large   ON (observed_at) WHERE abs(residual) > 0.05
INDEX ix_ps_unexpected       ON (observed_at) WHERE abs(unexpected_residual) > 0.03
```

TABLE *update_case* · the corpus row (new in v4)

```
TABLE update_case (
  case_id          text PK
  entity_id        text FK→entity
  tenant_id        text          -- canonical or customer
  privacy_class     enum          -- canonical | tenant_private |
                                  -- tenant_opt_in_abstracted
  prior_state       jsonb         -- belief vector + falsifiers + valuation snapshot
  evidence_packet_ref text        -- S3 pointer
  agent_action      jsonb         -- belief_update_ids, alt_set_ids, deltas
  system_review     jsonb NULL    -- ensemble disagreement, diagnosticity verdict,
                                  -- mechanism verdict, cross-validator notes
  accepted_update_id text NULL FK→belief_update
  later_outcome_id  text NULL FK→outcome
  calibration_result jsonb NULL   -- per-track verdicts, per §15
  resolved_at       timestampz NULL
)
INDEX ix_uc_entity_resolved  ON (entity_id, resolved_at DESC) WHERE resolved_at IS NOT NULL
INDEX ix_uc_unresolved      ON (entity_id) WHERE resolved_at IS NULL
INDEX ix_uc_canonical       ON (resolved_at) WHERE privacy_class = 'canonical'
```

TABLE *insight* · the Dot Connector's primary output

The Dot Connector's emission. Insights are the cross-layer pattern recognition output: they reference the deltas and (in v4) belief_updates that contributed, are scored retrospectively when forecastable, and become parents to hypothesis rows when a falsifier and horizon can be authored.

```

TABLE insight (
  insight_id          text PK
  entity_id           text FK-entity
  tenant_id           text
  emitted_at          timestampz      -- wall-clock of the sweep
  sweep_id            text            -- groups insights from the same run
  parent_deltas        text[] FK-delta  -- deltas that contributed
  parent_belief_updates text[] FK-belief_update -- [v4] belief_updates that contributed
  parent_insights      text[] FK-insight -- prior insights referenced
  statement            text            -- the specific pattern noticed
  layer_evidence        jsonb          -- { deep_insight: w1, financial: w2, ... }
  confidence_at_emit    numeric
  forecastable         bool            -- true → a hypothesis child is emitted
  retrospective_score   numeric        -- Evaluator · once (for scored insights)
)
INDEX ix_insight_entity_time  ON (entity_id, emitted_at DESC)
INDEX ix_insight_sweep        ON (sweep_id)
INDEX ix_insight_forecastable ON (entity_id) WHERE forecastable = true

```

TABLE hypothesis · child of insight only when forecastable

The forecastable derivative of an insight. A hypothesis exists only when a falsifier and a horizon can be authored explicitly. v4 adds `parent_alt_explanation_sets` so a hypothesis can carry forward the rival set against which it was framed.

```

TABLE hypothesis (
  hypothesis_id          text PK
  parent_insight          text FK-insight
  parent_alt_explanation_sets text[] FK-alternative_explanation_set -- [v4]
  entity_id              text FK-entity
  tenant_id              text
  emitted_at             timestampz
  statement              text            -- the forecastable claim
  falsifier              text            -- required
  horizon_tier            enum           -- short | mid | long
  expected_horizon_days   int
  confidence_at_emit       numeric
  confidence_trajectory    jsonb          -- expected signal order
  verdict                enum           -- NULL until resolved
  resolved_at            timestampz
  resolution_source        text
)

```

TABLE prediction · the immutable forecast

Predictions are typed by the prediction taxonomy of §15 (binary, discrete probabilistic, continuous, structured). Every prediction carries an explicit hypothesis parent, an immutable probability (or distribution), a horizon tier, a mechanism, a falsifier, and pre-stated mechanism evidence requirements. The `expected_price_reaction` band is new in v4 and prevents the price-update channel from double-counting evidence already in the prediction (§40).

```

TABLE prediction (
  prediction_id      text PK
  entity_id         text FK-entity
  parent_hypothesis text FK-hypothesis -- every prediction must cite
  triggered_by_delta text FK-delta
  based_on_baseline text FK-baseline
  tenant_id         text
  emitted_at        timestampz
  emitted_by        text
  horizon_tier      enum              -- short | mid | long
  horizon_days      int
  resolve_by        timestampz
  direction         enum
  probability        numeric          -- immutable
  baseline_probability numeric        -- counterfactual
  mechanism         text
  falsifier         text
  mechanism_evidence_requirements jsonb -- typed pre-stated patterns
  expected_price_reaction jsonb NULL  -- [v4] band for double-counting prevention
  verdict           enum              -- NULL until resolved
  mechanism_check    enum              -- NULL until resolved
  resolved_by_outcome text FK-outcome
)

```

TABLE outcome · the resolution

```
TABLE outcome (
  outcome_id          text PK
  entity_id           text FK→identity
  tenant_id           text          -- canonical or customer
  observed_at          timestampz    -- when in world the resolution happened
  ingested_at          timestampz    -- when the system observed it
  resolves_prediction  text NULL FK→prediction
  resolves_hypothesis  text NULL FK→hypothesis
  resolves_belief_update text[] NULL FK→belief_update -- [v4]
  resolves_update_case text NULL FK→update_case      -- [v4]
  resolution_source    enum          -- market_price | guidance_revision |
                                     -- regulatory_decision | filing_disclosure |
                                     -- transaction | merger_close |
                                     -- strategy_announcement | other
  source_evidence_id  text NULL FK→evidence -- if a single evidence row is the resolver
  direction           enum          -- up | down | flat | n/a
  magnitude           numeric       -- domain-typed magnitude
  magnitude_unit       text         -- pct | bps | usd | count | dimensionless
  observed_components  jsonb        -- structured: each component the prediction named,
                                     -- resolved separately (per §15)
  mechanism_observed  jsonb NULL    -- did the named mechanism operate?
                                     -- { confirmed | unconfirmed | contradicted,
                                     --   evidence_basis: [...] }
  notes              text NULL
)
INDEX ix_outcome_entity_observed ON (entity_id, observed_at DESC)
INDEX ix_outcome_resolves_pred  ON (resolves_prediction) WHERE resolves_prediction IS NOT NULL
INDEX ix_outcome_unscored       ON (ingested_at) WHERE resolves_prediction IS NOT NULL
                                AND NOT EXISTS (
                                  SELECT 1 FROM prediction p
                                  WHERE p.prediction_id = resolves_prediction
                                  AND p.verdict IS NOT NULL
                                )
```

The Outcome ingestion subagent is the single semantic writer. Outcomes are append-only; corrections are appended as new outcome rows, with an explicit `superseded_by` chain reusing the same pattern as evidence (omitted from the DDL above for brevity, but the pattern is identical). One outcome row may resolve a single prediction, a hypothesis, several belief_updates simultaneously, and an update_case — which is correct, because in the world a single observed event can be diagnostic for multiple open claims at once.

TABLE calibration · the scorecard

```

TABLE calibration (
  calibration_id      text PK
  entity_id           text NULL FK→identity -- null for system-wide rows
  subagent            text                  -- agent identifier or 'system' for cross-agent
  horizon_tier         enum                  -- short | mid | long
  belief_category      enum NULL             -- [v4] null for delta-only calibration;
                                          -- non-null for belief-track calibration
  track                enum                  -- [v4] outcome | posterior | direction |
                                          -- diagnosticity | mechanism
  scope                enum                  -- per_agent | per_entity_agent | system
  -- Aggregate scoring
  brier_mean           numeric                -- for binary/discrete predictions
  crps_mean            numeric NULL           -- for continuous predictions
  log_score_mean       numeric NULL           -- for discrete probabilistic
  decomposed_mean      jsonb NULL            -- for structured predictions, per-component
  mechanism_confirm_rate numeric NULL         -- mechanism-confirmed / outcomes-confirmed
  -- v4 belief tracks
  posterior_calibration numeric NULL           -- [v4] Track 1: posterior vs realised frequency
  direction_correctness numeric NULL          -- [v4] Track 2: fraction of update directions
                                          -- that moved correctly
  diagnosticity_error   numeric NULL          -- [v4] Track 3: |claimed - realised| diagnosticity
  -- Volume and recency
  n_resolutions        int                    -- number of resolved items in this aggregate
  weight_adjustment     numeric                -- multiplier applied to source-agent's
                                          -- contributions; bounded [0.25, 4.0]
  window_start         timestampz              -- aggregate window
  window_end           timestampz
  updated_at           timestampz
  prev_calibration_id   text NULL FK→calibration -- chain so weight changes are auditable
)
INDEX ix_calib_subagent_horizon ON (subagent, horizon_tier, updated_at DESC)
INDEX ix_calib_entity_agent     ON (entity_id, subagent, horizon_tier)
                                WHERE entity_id IS NOT NULL
INDEX ix_calib_track            ON (track, subagent, updated_at DESC)
INDEX ix_calib_belief_category  ON (belief_category, subagent, updated_at DESC)
                                WHERE belief_category IS NOT NULL

```

The Evaluator is the single writer. Calibration rows are append-only, never updated in place: a “weight adjustment” is a new row pointing to the previous via `prev_calibration_id`. This makes weight history auditable and replayable. The v4 extension is the addition of three belief tracks (posterior, direction, diagnosticity) per the scoring methodology of §15. A single subagent now has up to five rows per (entity, horizon_tier) — outcome, posterior, direction, diagnosticity, mechanism — instead of v3.1’s one. Decomposition is multiplicative: low diagnosticity calibration cannot mask a healthy outcome track.

TABLE threadweave_path · versioned apex state

```
TABLE threadweave_path (
  path_id          text PK          -- e.g. 'TW-48221/v09'
  entity_id        text FK→identity
  tenant_id        text
  intent_assumption enum
                  -- npv_max (default) |
                  -- founder_vision_weighted |
                  -- control_preservation |
                  -- policy_constrained |
                  -- custom (with override profile)

  intent_profile_id text NULL FK→entity_intent_profile -- when overridden
  version           int             -- monotonic per entity per intent
  prev_version_id   text NULL FK→threadweave_path
  authored_at       timestampz
  authored_by_run   text            -- orchestrator run id
  path_state        jsonb           -- { horizons: {1y:{...}, 3y:{...}, 5y:{...}},
                  -- branches, conversion_metrics,
                  -- competitor_shadows,
                  -- key_uncertainty_nodes,
                  -- reachable_states_set }

  reliability       jsonb           -- per-horizon reliability score, function
                  -- of accumulated scored judgment per §15

  inputs_pulled     text[]          -- ids of layers/agents/tables that contributed
                  -- [v4] may include belief_update_ids alongside
                  -- delta_ids

  forward_or_reverse enum           -- forward | reverse | both
  reverse_target_state text NULL FK→future_state -- when reverse traversal was anchored
                  -- to a target future-state

  required_path_id  text NULL FK→required_path
  is_current        bool
)
INDEX ix_tw_entity_current      ON (entity_id, intent_assumption) WHERE is_current = true
INDEX ix_tw_entity_version     ON (entity_id, version DESC)
INDEX ix_tw_reverse            ON (entity_id, reverse_target_state)
                              WHERE forward_or_reverse IN ('reverse', 'both')
INDEX ix_tw_reliability_low     ON (entity_id) WHERE (reliability->>'3y')::numeric < 0.50
```

Threadweave is the only writer. Path versions are append-only; `is_current` is flipped on supersession. The v4 extension permits `inputs_pulled` to reference belief_update IDs, so the path's reliance on specific belief revisions is part of its provenance. The reverse-traversal extension (`forward_or_reverse`, `reverse_target_state`, `required_path_id`) makes the bidirectional projection of §13 explicit in the schema rather than implicit in the run log.

TABLE *entity_factor_observation* · factor co-occurrence ledger

```
TABLE entity_factor_observation (
  observation_id      text PK
  entity_id           text FK→identity
  tenant_id           text
  factor_kind          text
                        -- regulatory_regime | competitor_action |
                        -- macro_variable | structural_condition |
                        -- mechanism | sentiment_pattern |
                        -- technological_breakthrough |
                        -- governance_event | ...
  factor_value         text
                        -- the literal factor instance
  observed_at          timestampz
  source_kind          enum
                        -- delta | insight | hypothesis |
                        -- belief_update -- [v4]
  source_id            text
                        -- pointer to originating row
  outcome_id           text NULL FK→outcome -- linked when resolution arrives
  outcome_observed_at  timestampz NULL
  outcome_class         text NULL
                        -- when joined to outcome
  belief_update_id     text NULL FK→belief_update -- [v4] when source_kind='belief_update'
                        -- or when factor was diagnostic for one
)
INDEX ix_efo_entity_factor ON (entity_id, factor_kind, observed_at DESC)
INDEX ix_efo_factor_value  ON (factor_kind, factor_value)
INDEX ix_efo_outcome_join  ON (outcome_id) WHERE outcome_id IS NOT NULL
INDEX ix_efo_belief_join   ON (belief_update_id) WHERE belief_update_id IS NOT NULL
```

This table powers the Dot Connector’s cross-entity pattern recognition: which factors recur, which co-occur, which precede which outcomes. The v4 extension lets it attach to belief_updates so the Dot Connector can ask, across entities, which factor instances most reliably move which belief categories.

TABLE *entity_intent_profile* · Threadweave intent overrides

```
TABLE entity_intent_profile (
  profile_id          text PK
  entity_id           text FK→identity
  tenant_id           text
                        -- canonical or customer (intent overrides
                        --   are commonly tenant-specific)
  current_intent       enum
                        -- npv_max (default) |
                        --   founder_vision_weighted |
                        --   control_preservation |
                        --   policy_constrained | custom
  intent_evidence      jsonb
                        -- structured: capital allocation history,
                        --   founder statements, board composition,
                        --   shareholder structure, observed actions
                        --   that distinguish intent from npv_max
  intent_confidence    numeric
                        -- 0..1, how confident in the override
  authored_at          timestampz
  authored_by_run      text
                        -- run id that proposed the override
  authored_by_agent    text
                        -- which agent observed the divergence
  superseded_at        timestampz NULL
  superseded_by        text NULL FK→entity_intent_profile
  superseded_reason    text NULL
                        -- behavioural_evidence_changed | review |
                        --   regime_shift | policy_update
)
INDEX ix_eip_entity_current  ON (entity_id, tenant_id) WHERE superseded_at IS NULL
INDEX ix_eip_intent          ON (current_intent, entity_id)
                           WHERE current_intent <> 'npv_max'
                           AND superseded_at IS NULL
```

Intent overrides exist because not every entity behaves as an NPV-maximising agent. A founder-controlled company may weight founder vision; a state-influenced enterprise may weight policy constraint; a takeover-defensive firm may weight control preservation. Threadweave reads this profile when projecting forward, so its forecasts are conditioned on the entity’s actual revealed intent rather than a default assumption.

TABLE *future_state* · target market structures

```
TABLE future_state (
  future_state_id      text PK
  market_or_vertical   text
                        -- e.g. 'vertical-AI/radiology' |
                        --   'EU-grid-storage' | 'global-LNG-shipping'
  description           text
                        -- structural claim:
                        --   '~3 dominant players, top one ~$20B revenue'
  horizon_year          int
                        -- target year, approximate
  pareto_shape          jsonb NULL
                        -- distribution parameters when applicable
  evidence              jsonb
                        -- demand drivers, capability curves,
                        --   regulatory trajectories
  probability           numeric
                        -- system's confidence the structural claim
                        --   holds at horizon
  authored_by_run       text
                        -- Synthetic Futures run that produced it
  authored_at           timestampz
  superseded_at         timestampz NULL
  superseded_by         text NULL FK→future_state
)
INDEX ix_fs_market_horizon  ON (market_or_vertical, horizon_year)
INDEX ix_fs_active          ON (market_or_vertical) WHERE superseded_at IS NULL
```

TABLE *required_path* · necessary conditions for a future-state

```
TABLE required_path (  
  path_id          text PK  
  entity_id        text FK→identity  
  future_state_id  text FK→future_state  
  tenant_id        text  
  authored_by      text          -- Threadweave run id  
  authored_at      timestampz  
  required_conditions jsonb      -- list of typed milestones with falsifiers  
                                -- and horizons; each condition is itself  
                                -- a candidate hypothesis row  
  decision_sequence jsonb NULL   -- sketched ordering and approximate timing  
                                -- (advisory, not part of scoring)  
  conditions_authored_as_hypotheses text[] NULL FK→hypothesis  
                                -- when conditions are individually scorable  
  superseded_at    timestampz NULL  
  superseded_by    text NULL FK→required_path  
)  
INDEX ix_rp_entity_future ON (entity_id, future_state_id)  
INDEX ix_rp_active       ON (entity_id) WHERE superseded_at IS NULL
```

Together with `future_state`, this table makes Threadweave’s reverse traversal first-class: a target structural future has explicit required conditions, each with a falsifier, each scorable as it resolves. v3.1 introduced the pair; v4 makes them join cleanly to hypothesis when conditions are individually forecastable.

Views, the agent read surface

No agent touches SQL. Every agent reads the substrate through a typed view, a thin read layer that assembles exactly the context needed, nothing more. This is where token budgets are enforced.

v3 views: `entity_context_view`, `prediction_candidates_view`, `outcome_sweep_view`, `baseline_audit_view`, `hypothesis_context_view`, `threadweave_inputs_view`. v4 adds three:

- `belief_context_view(entity_id)`: the current belief vector for an entity, with `current_confidence`, `falsifier_distance`, `importance_weight`, recent `belief_updates` per belief, open `alt_explanation_sets`.
- `belief_update_candidates_view(entity_id, claim_id)`: what the layer agent reads when proposing a belief update: affected beliefs, prior, expected_evidence, recent rivals.
- `update_case_resolution_view(case_id)`: what the Evaluator reads when scoring a closed update_case: prior state, evidence packet, accepted update, later outcome, mechanism verdict.

§22 Access matrix, who reads and writes what (*amended in v4*)

The substrate’s discipline is only as strong as the access pattern that enforces it. Every table has a single semantic writer (a “pen-holder”) and a defined set of readers. CrossValidator surfaces contradictions but does not write semantic state. The Orchestrator alone commits deltas, `belief_updates`, and `alt_explanation_sets` to the canonical core.

Read/write matrix

R = read, W = write (semantic), W = write via typed proposal that the Orchestrator commits, W = write-once retrospective field only, W** = write-once verdict and score fields only, em-dash = no access.

Agent	Identity	Evidence	Baseline	Delta	Insight	Hypothesis	Prediction	Outcome	Calibration	Thread
Resolver	W	W	R	—	—	—	—	—	R	—
Orchestrator	R	R	R	W	W	W	—	—	R	R
Deep Insight (6)	R	R/W	R	W*	—	—	—	R	R	—
Financial Model (2)	R	R/W	R	W*	—	—	—	R	R	—
Synthetic Futures (4)	R	R/W	R	W*	—	—	—	R	R	—
Belief Extractor (v4)	R	R	R	—	—	—	—	—	R	—
CrossValidator	R	R	R	W*	—	—	—	—	R	—
Dot Connector	R	R	R	R	W	W	—	—	R	—
Threadweave	R	R	R	R	R	R	R	R	R	W
Forecaster	R	—	R	R	R	R	W	—	R	—
Evaluator	R	R	R	W**	W***	W***	W***	R	W	—
Outcome ingest	R	—	—	—	—	—	—	W	—	—
Price ingest (v4)	R	—	—	—	—	—	—	—	—	—
Replay engine	R	R	W	R	R	R	R	R	R	R

Write-discipline notes

- Base-layer specialists propose deltas and belief_updates via typed tool call. The Orchestrator commits with weight applied. CrossValidator writes deltas only when it records an unresolved disagreement, never a semantic update. No specialist writes to Delta, Belief, or Belief_update directly.
- The Belief Extractor (new in v4) is an ensemble that proposes belief authoring. Authored beliefs are committed by the Orchestrator after the three deterministic gates (ensemble agreement, falsifier presence, expected-evidence presence) clear. The extractor does not commit beliefs itself.
- Evaluator's write to Delta is write-once, restricted to `retrospective_score` and `learning_note`. Everything else on that row is immutable.
- Evaluator's write to Belief_update is write-once, restricted to `retrospective_score`, `update_direction_verdict`, and `diagnosticity_verdict`. The prior, posterior, evidence packet, and rival probabilities at emission are immutable.

- Evaluator's write to Insight, Hypothesis, and Prediction is write-once, restricted to retrospective-score and verdict fields. Dot Connector is the sole semantic writer of Insight. Forecaster is the sole semantic writer of Prediction.
- Evaluator's write to Update_case is write-once on `later_outcome_id` and `calibration_result`. The case's prior state, evidence packet, agent action, and system review are immutable from creation.
- Replay engine is the sole writer of Baseline. The Orchestrator does not write Baseline directly; it writes Delta and Belief_update, the replay engine derives Baseline.
- Resolver writes Identity (with single-tenant discipline) and Evidence. No other agent writes Identity.

What this enforces

The matrix enforces, in code, that:

1. The originating agent of a prediction never reads its own outcome (Evaluator independence, §15).
2. The Belief Extractor never reads existing belief_updates when authoring a new belief (no contamination from the system's own prior reasoning).
3. The replay path is the verification mechanism for Baseline; nothing outside the replay engine can claim authority over current state.
4. Belief and Belief_update have exactly one semantic writer (the Orchestrator), the same access pattern as Delta. The v4 substrate has the same discipline as the v3 substrate, extended.

Part IV · The operation

How it runs, what to ship, what is still open. A worked example traces one signal from arrival to scored outcome through both the operational spine and the belief journey. The launch path is preserved from v3.1 with v4-specific phasing additions. Open questions surface fourteen things not decided, seven from v3.1 and seven new in v4. A small list of operational concerns deliberately out of scope.

§23 The loop, a worked example end to end (amended in v4)

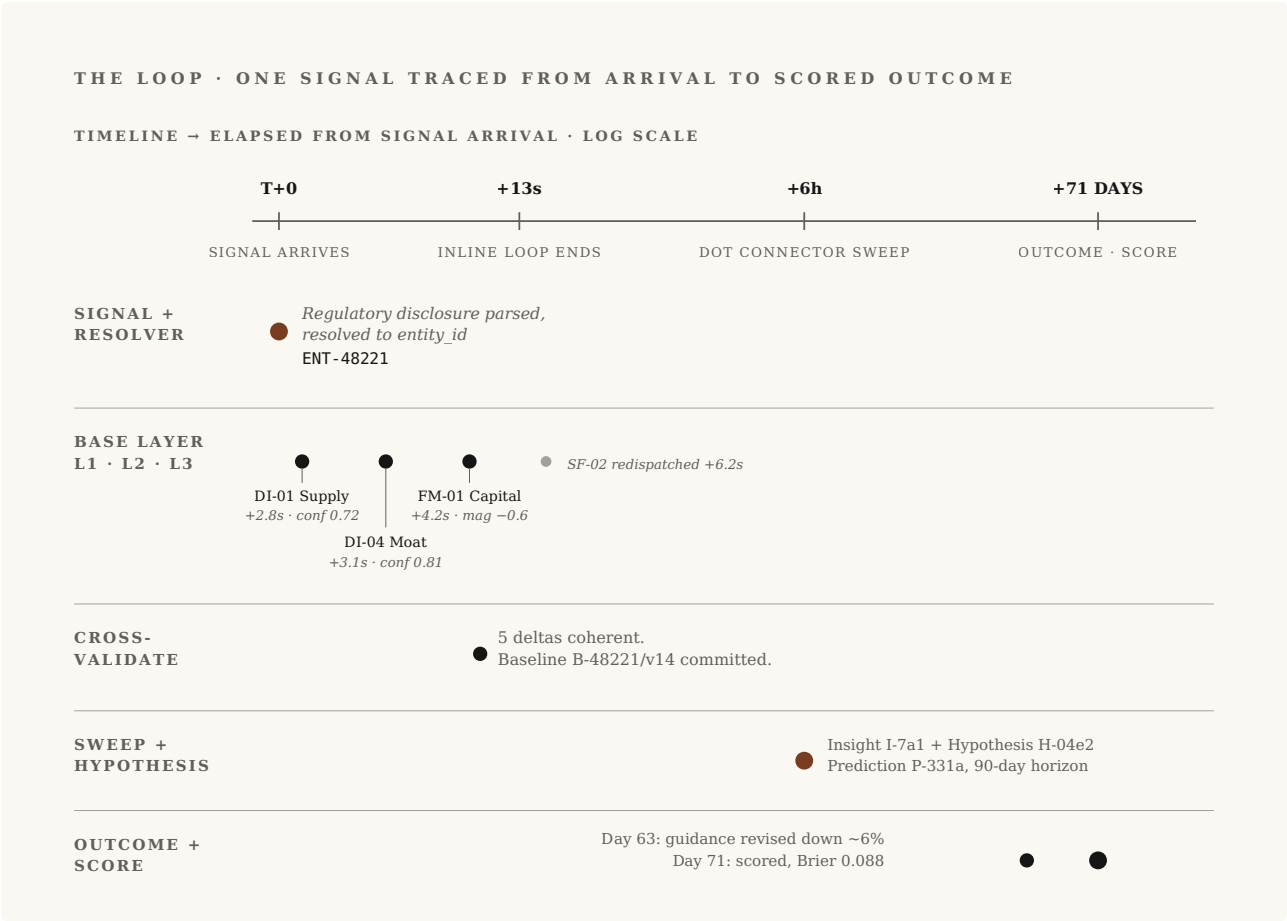


FIGURE 11 · WORKED-EXAMPLE TIMELINE. ENTITY ENT-48221, SIGNAL AT T+0, INLINE LOOP CLOSSES IN 13 SECONDS, SCHEDULED SWEEP AT +6 HOURS, OUTCOME RESOLVES AT DAY 71. THE FULL ARCHITECTURAL CYCLE IN ONE TRACE.

Mid-cap European industrial, ENT-48221. Already bootstrapped. A regulatory disclosure lands: the company is switching its primary supplier due to an enforcement action on the prior one. What happens next, from claim ingest to outcome resolution seventy-one days later, routed through both the six-layer stack and the v4 belief journey.

Pre-existing belief vector at T-0

ENT-48221 carries an active belief vector of 14 named claims at the time the disclosure arrives. The four beliefs that will move in this example:

Belief ID	Statement (compressed)	Category	current_confidence	falsifier_distance	Importance
B-481	Long-term supplier partnership is durable competitive advantage	moat	0.78	far	0.85
B-512	Capital efficiency rests on stable supply structure	capital_allocation	0.71	watch	0.70
B-617	Operational discipline indicates over-optimised process	structural_future	0.42	far	0.55
B-624	Adjacency expansion in 12 to 24 months remains feasible	structural_future	0.66	watch	0.60

Inline loop, T+0 to T+13s

T	Agent	Action
+0ms	Resolver (Haiku)	LEI match unambiguous. Claim C-9f12 attached to ENT-48221. Handed to Orchestrator.
+400ms	Orchestrator (Opus)	Claim touches supply chain, regulation, capital. Dispatches DI-01 Supply, DI-04 Moat, DI-05 Fragility, FM-01 Capital. Holds SF layers until base deltas return.
+2.8s	DI-01 Supply	Proposes Δ: operational surprise implies gap in supplier-risk oversight. Confidence 0.72. Magnitude: material.
+3.1s	DI-04 Moat	Prior baseline cited supplier partnership as part of moat. Proposes Δ: moat durability revised down. Confidence 0.81. Belief journey runs (04a-04g) on B-481.
+3.4s	DI-05 Fragility	Forced switch removes optionality. Proposes Δ: over-optimisation flag raised, adaptability downgraded. Confidence 0.78. Belief journey runs on B-617.
+4.1s	FM-01 Capital	Balance sheet strong, but capex was deployed toward the disrupted supplier. Proposes Δ: capital efficiency contingent on supply structure now changing. Belief journey runs on B-512.

What the belief journey does inside DI-04's proposal (detail)

At +3.1s, DI-04 Moat is processing the disclosure. Step 04a (identify or extract beliefs): the affected belief is B-481, already in the active vector. No extraction needed. Step 04b (retrieve prior): `prior_confidence` = 0.78, `expected_evidence` registered when B-481 was authored includes “stable multi-year supplier relationship,” “supplier-specific cost advantages reflected in margin profile,” “low supplier concentration risk in 10-K

disclosures.” Step 04c (decompose evidence): the disclosure is reliability 0.95 (regulatory primary source), magnitude material, persistence structural (a forced supplier switch is not transitory), independence 1.0 (no prior signal), diagnosticity 0.78 (this evidence does discriminate among rival explanations of B-481).

Step 04d (rival explanations) generates the moat-specific rival set:

Rival explanation	Prior probability	Posterior probability
Moat-supplier-specific (depends on this supplier in particular)	0.30	0.62
Moat-operational-not-structural (rests on operational practice, transferable)	0.45	0.27
Moat-unaffected (the moat does not depend on the supplier)	0.15	0.07
Supplier-independent moat (a different mechanism entirely)	0.10	0.04

Step 04e (likelihood logic): the disclosure looks much more like the supplier-specific rival than like any other. $LR(\text{supplier-specific vs operational-not-structural}) \approx 4.5$; $LR(\text{supplier-specific vs moat-unaffected}) \approx 8.2$. Step 04f (posterior): current_confidence on B-481 falls from 0.78 to 0.46. Step 04g (falsifier distance): B-481 moves from **far** to **medium**. Step 04i (commit): on Orchestrator commit, a **belief_update** row is written for B-481 with the full decomposition, and an **alternative_explanation_set** row records the rival shift. The dominant shift is recorded as **moat-supplier-specific**. Diagnosticity is claimed at 0.78.

Parallel belief journeys run inside DI-05’s and FM-01’s proposals on B-617 and B-512. SF agents are dispatched after CrossValidator’s first pass (see below) and run their own belief journeys on B-624.

Inline loop continues, T+5.7s to T+13.1s

T	Agent	Action
+5.7s	CrossValidator	Checks coherence across 4 proposed deltas + their belief_updates. DI-04 and FM-01 are consistent (moat down ↔ capex productivity at risk). Check/05 runs: probability conservation on B-481 alt_set sums to 1.0; dominant shift is supported by recorded diagnosticity. Flags gap with SF layer; escalates.
+6.2s	Orchestrator (Opus)	Dispatches SF-02 Dynamic Exploration and SF-03 Endgame Paths.
+8.4s	SF-02 Dyn. Expl.	Proposes Δ: supplier change may unlock one adjacency but forecloses two others. Net opportunity-space contraction small but real. Belief journey runs on B-624. B-624 falls from 0.66 to 0.51, falsifier_distance moves from watch to medium .
+10.8s	CrossValidator pass 2	All five deltas + four belief_updates coherent. Check/05 passes on all four alt_explanation_sets. Cross-lens coherence 0.84. Records DI-05 vs SF-02 as resolved disagreement (DI preserved as minority on adaptability).
+13.1s	Orchestrator commit	Commits five deltas and four belief_updates atomically. Replay engine materialises B-48221/v14 as the new baseline snapshot. User-facing notice: “Baseline shifted: B-481 (supplier moat) confidence dropped from High to Medium-low; B-512 capital allocation moved to Watch; B-617 over-optimisation flag raised; B-624 adjacency feasibility moved to Watch.” No hypothesis yet.

The customer surface presents the four belief moves in confidence-band language (per §15), not raw probabilities.

Sweep loop, T+6h

The Dot Connector begins its nightly sweep for the active universe. ENT-48221 has five fresh deltas, four fresh belief_updates, and an unresolved disagreement, so it is high priority.

At +6h+12s, the Dot Connector reads the full substrate for ENT-48221 (recent deltas, recent belief_updates, base-model states, the hypothesis ledger, CrossValidator's disagreement record) and recognises a pattern: **the supplier disruption overlaps with a sector-wide regulatory tightening visible in two peer entities' deltas and three peer-entity belief_updates over the past 30 days**. The pattern is across-entity, made visible only because the substrate carries belief_updates as first-class evidence (per §12 v4 amendment).

At +6h+18s, Insight I-7a1 is written: "ENT-48221's supplier switch fits a sector-wide pattern of enforcement-driven capex redirection; peers likely to face similar moat erosion over two quarters." Evidence chain spans 5 ENT-48221 deltas + 4 ENT-48221 belief_updates + 3 peer-entity deltas + 3 peer-entity belief_updates. Forecastable.

At +6h+19s, hypothesis H-04e2 is emitted: guidance revision within 90 days. Falsifier: reaffirmed guidance at next earnings. Horizon tier: short. **The hypothesis carries expected_price_reaction : a band of [-3%, -1%] residual return on guidance revision, reflecting Forecaster's prior on what a minor guidance revision typically prices in**. This is the v4 double-counting prevention (§40): only price moves outside this band update business beliefs further.

At +6h+22s, Forecaster emits P-331a: direction down, 0.34 probability, 90-day horizon. Mechanism stated, falsifier inherited. Mechanism evidence requirements typed.

At +6h+25s, the user-facing insight publishes with sector-wide framing. Evidence trail, delta lineage, **belief_update lineage**, hypothesis chain, peer-entity references all attached. Ranked top-decile against the user's margin-surprise-risk objective.

At +24h, Threadweave's nightly path run consumes I-7a1, H-04e2, and the four belief_updates. The 3y path weight shifts 4% toward "bounded growth" branch. 5y reliability widens on peer entities too.

Outcome and scoring, T+71d

Day 51 (T+51d). Sector-wide news flow turns. A peer entity flags supply-chain disruption in its quarterly call. ENT-48221's stock declines 4.2% on the day, market-adjusted return -3.1%, sector-adjusted -2.4%, residual -1.8%. **A price_signal row writes**. The unexpected_residual on ENT-48221 is -1.8% minus the Forecaster's expected_price_reaction midpoint of -2.0%, which is +0.2%, well inside the band: the price move is approximately in line with what the system already expected if H-04e2 is correct. No new business-belief update is triggered. **Knowability and tail-risk posteriors update modestly per §41**: knowability tightens by 4 bps because the move was inside expectation, indicating consensus is shifting in the system's direction; tail-risk widens marginally because the timing was earlier than expected.

Day 63 (T+63d). Guidance revised down by 6%. Outcome O-7723 logged: direction down, magnitude -6%.

Day 71 (T+71d). Evaluator runs.

Outcome verdict on P-331a: **confirmed**. Brier 0.088. Direction matched, magnitude was within the predicted range.

Mechanism verdict ensemble: five Evaluator invocations dispatch independently, each receiving only the prediction text, the pre-stated mechanism evidence requirements, and the resolution-window evidence (the 10-Q, earnings call transcript, sector news in the window). None receives the originating agent reasoning. Three return `mechanism_confirmed` (the supplier switch is explicitly cited in management’s revision rationale with quantified pass-through to gross margin). Two return `mechanism_unconfirmed` (the management commentary mentions supplier costs but does not isolate the specific supplier the prediction identified, and a Q3 inventory write-down is a partial confound). Final verdict: `mechanism_confirmed` with ensemble agreement 3/5, ambiguity flag.

Insight retrospective score on I-7a1: strong. The cross-entity sector pattern was visible and the forecast it generated proved correct.

Belief-update verdicts on the four moves committed at T+13s:

Belief	Track 1 (posterior calibration)	Track 2 (update direction)	Track 3 (diagnosticity)
B-481 (moat)	Brier 0.04 (posterior 0.46 was well-calibrated against the eventual confirmation)	correct (probability shifted toward <code>moat-supplier-specific</code> , the rival the resolution evidence supports)	as_claimed (diagnosticity 0.78 was approximately right; the evidence did discriminate as predicted)
B-512 (capital)	Brier 0.07	correct	as_claimed
B-617 (over-optimisation)	Brier 0.18 (posterior shift was modest and the resolution did not strongly confirm either direction)	ambiguous	unverifiable (the evidence chain was too thin to verify diagnosticity claim)
B-624 (adjacency)	unresolved (5y horizon; remains pending)	unresolved	unresolved

Update_case rows: four update_cases are now resolved (one per moved belief), each carrying the full prior state, evidence packet, agent action, system review, accepted update, later outcome, and per-track calibration result. They become canonical update_cases (eligible for §35 corpus inclusion since ENT-48221 is canonical).

Calibration updates. DI-05’s weight for mid-cap industrials tightens marginally on Track 1 (the over-optimisation update was less well-calibrated than the moat update). DI-04’s weight tightens on Track 2 (correct update direction on a moat belief, with `moat-supplier-specific` rival selection well-calibrated). The Dot Connector’s sector-correlation pattern gains calibration weight for enforcement-driven deltas. Forecaster’s `expected_price_reaction` band on guidance revisions carries forward.

Inline loop, claim arrival to committed baseline including belief_updates: under 14 seconds. Insight loop, committed baseline to published insight with sector-wide cross-entity belief pattern: the next scheduled sweep, typically under 24 hours. Learning loop, emitted prediction to scored outcome with per-track belief verdicts: 71 days in this example. Each timescale matters for a different part of the product; the last is what makes the product compound.

§24 Launch scope, what to build (*amended in v4*)

The architecture is designed to scale to the full six-layer stack with belief journey across hundreds of entities. It does not obligate shipping with all of it. The critical path is the shortest route to a closed loop on a narrow universe; everything after that is widening.

Launch universe. Approximately 100 listed EU industrial mid-caps. Enough to see patterns. Small enough to hand-review. Market data is clean and cheap. Outcome resolution is fast.

Launch agent set. Resolver, Orchestrator, Deep Insight (DI/01 Supply only), Financial Model (FM/01 Capital + FM/02 Forecasting), Synthetic Futures (SF/01 Demand only), CrossValidator (with Check/05 active from day one), **Belief Extractor**, Dot Connector (minimal), Forecaster, Evaluator. Ten agents, not seventeen. The Dot Connector ships in a stripped form: it runs a nightly sweep, writes insights, emits hypotheses only for the clearest forecastable cases. Threadweave is post-launch: it requires scored judgment across at least two horizon tiers to be meaningful, and that data does not exist at launch.

v4 phasing overlay

The 90-day launch path of v3.1 is preserved as the core sequence. v4 inserts three phase gates and a final autonomy-validation gate before the belief layer is treated as canonical. See §42 for the phase plan in full; the launch outline below maps the 90-day window onto the operational backbone, with belief-layer milestones tagged.

Weeks 1 to 2 · Identity and schema

Stand up Postgres with the eleven load-bearing tables (Baseline, Delta, **Belief**, **Belief_update**, **Alternative_explanation_set**, Insight, Hypothesis, Prediction, Outcome, Calibration, **Update_case**) plus tenant_id everywhere. Add the **Price_signal** TimescaleDB hypertable. Load the universe with LEIs and tickers. Deploy Resolver with the bootstrap queue.

Ship when Resolver attaches 95%+ of incoming signals to the right entity unattended.

Weeks 3 to 4 · Bootstraps and initial belief vectors

Hand-author 10 baselines against the schema to stabilise it. **For each of the 10 hand-authored entities, also hand-author an initial belief vector of 8 to 15 named claims as a seed corpus.** Then let the Orchestrator dispatch the launch agent set on 90 more, **with the Belief Extractor authoring belief vectors autonomously for these**, reviewed for schema drift only. The hand-reviewed baselines and seed belief vectors double as the Evaluator's early ground truth.

Ship when baseline schema and belief schema both stop changing for seven consecutive days, **and the Belief Extractor's ensemble agreement rate stabilises above 0.65** on the autonomously-authored beliefs (per the §42 phase 1 gate).

Weeks 5 to 6 · Live loop, predictions and belief_updates in a drawer

Ingest everything. Orchestrator runs on every claim, dispatching the launch agent set. **Belief journey runs inside every material delta from day one.** CrossValidator active from day one with Check/05. Dot Connector runs its nightly sweep, reads belief_updates as first-class input, writes insights, emits hypotheses when forecastable.

Forecaster emits predictions into the Prediction table, **including expected_price_reaction bands on price-relevant predictions**. Not yet user-facing. The system is collecting prediction data, belief_update data, and price_signal rows; not distributing alerts.

Ship when predictions carry all required fields and a parent hypothesis ID consistently, **belief_updates carry prior, posterior, alt_set, diagnosticity, and falsifier_distance change consistently across all material deltas for one week, and the rejection-and-reroute rate (deltas re-routed to belief-extraction) is under 25% of material deltas**.

Weeks 7 to 8 · The loop closes

Wire market data as outcome sources. Evaluator runs daily against the prediction table with horizon-tier-aware scoring. **First per-entity, per-subagent, per-horizon-tier, per-belief-category calibration curves appear. First Track 1 (posterior calibration), Track 2 (update direction), and Track 3 (diagnosticity) verdicts attach to closed update_cases**.

Ship when a calibration plot stratified by horizon tier and belief category can be displayed, at least one learning note has measurably improved the system the following week, **and the replay-reproduction rate on a held-out sample is above 0.95** (the v4 phase 2 gate, see §42).

Weeks 9 to 12 · Belief layer hardens

The Bayesian update layer becomes canonical. Pass conditions per §42:

- ensemble agreement rate ≥ 0.75 over 30 consecutive days (autonomous belief authoring is consistent enough)
- replay-reproduction rate ≥ 0.95 on retrospective replays (the system can reproduce its own past beliefs from the ledger, which is the v3 commitment extended to the belief layer)
- rejection-and-reroute rate stabilised below 25%
- probability-mass-shift rate $\geq 60\%$ (most material updates actually shift probability mass, not just nominal posteriors)
- diagnosticity calibration error < 0.20
- 1000 canonical update_cases accumulated across the 100-entity universe

Ship when all six pass conditions are met. Canonical means the belief layer is now load-bearing for customer-facing outputs, not just instrumentation.

Month 3+ · Earn the complexity

Add a Deep Insight or Synthetic Futures specialist only when (a) the Dot Connector's insights keep gesturing at a domain no current subagent covers, or (b) calibration on the existing agents plateaus and the Orchestrator keeps encountering gaps. **Add belief categories beyond the launch set when the Belief Extractor proposes them and they pass the three deterministic gates plus six months of post-authoring calibration.** Threadweave becomes feasible only after roughly 6 months of scored judgment at short and mid horizons.

Graduation test for Threadweave. Reliability score on its first entity must be non-trivial: enough horizon-tier calibration data to make its path claim defensible. **In v4, also enough belief-category coverage that Threadweave's reverse-projection can attach to specific structural beliefs rather than to baseline aggregates.**

§25 Open questions, what still needs decisions before production (*amended in v4*)

Fourteen things not decided, which will shape implementation. Seven carry forward from v3.1; seven are new in v4. Flagging now saves the second iteration.

From v3.1, carried forward

D/01 · Delta-magnitude threshold. When does a datapoint become a delta? A magnitude field is committed (noise · minor · material · regime). Policy: log all four magnitudes, but only **material** and **regime** trigger CrossValidator and Dot Connector. **In v4, only **material** and **regime** also trigger the belief journey.** Noise and minor are audit-only. Revisit after first calibration cycle.

D/02 · Horizon tiers and Threadweave reliability. Resolved in §15 (Threadweave reliability function). Per-tier calibration with conservative reliability score; falls back to “insufficient data” when long-tier judgment is missing. Let reliability grow with data rather than reverse-engineer a formula.

D/03 · Bootstrap chronology. Replay forward, or synthesise retrospectively? The choice is between replaying chronologically (Orchestrator walks forward through time seeing only what was knowable then) or synthesising retrospectively (current model is asked to reason as if from each past date). The architecture’s commitment to System B over System A (§03) makes chronological replay the only honest choice. Retrospective synthesis is acceptable only as a labelled **retrospective_view** artefact (per epistemic rule 4), never as the primary record.

D/04 · Calibration-aggregation cells. Should calibration cells be per (entity, agent, horizon-tier), or coarser? Resolved per §15: per-cell with bootstrap confidence intervals. **In v4, also per-belief-category.** Cells with insufficient data show “insufficient” rather than computing fragile estimates.

D/05 · Selection bias on what gets predicted. Out of scope for the architecture; flagged as a research question in §15. The architecture commits to making the data available for studying it (every delta, every belief_update, every hypothesis).

D/06 · Causal modelling roadmap. Defer to v5+. Ship v4 with mechanism attribution, factor co-occurrence, and the v4 belief layer (which is structurally weaker than causal modelling but stronger than v3). Let scored data accumulate for 12 to 18 months. Use that data to evaluate whether causal-graph modelling is defensible at the next architecture revision. The substrate is designed to make the v5+ direction possible without precluding it.

D/07 · Future-state forecasting limits. Honest acknowledgment of where the framing strains: future-state forecasts have a poor track record industry-wide. Ship v4 with future-state forecasting committed but bounded. Initial coverage focuses on structural patterns (concentration, Pareto distributions, equilibrium counts) rather than identity predictions. **In v4, structural-future beliefs receive their own calibration cell so long-horizon belief drift is visible.** Track agent calibration on future-state predictions separately from shorter-horizon calibration. If, after 18 to 24 months of scored data, the long-horizon track record is genuinely poor, the architecture commits to surfacing this honestly rather than continuing to publish reverse-traversal output as if calibrated.

New in v4

D/08 · Ensemble agreement threshold for belief authoring. Phase 1 of §42 commits to a target rate of 0.75 over 30 consecutive days. The threshold is provisional; the right value depends on what the rejection-and-reroute rate looks like at different thresholds. Recommendation: ship phase 1 at 0.65 (lower bar to avoid stalling on weak ensembles) and phase 2 at 0.75 (stricter once the ensemble has tuned). Revisit empirically after the first 1000 candidate beliefs.

D/09 · Confidence vector vs single number on beliefs. Internally the system carries a likelihood-ratio representation that approximates a confidence distribution. Externally the customer surface presents a single confidence band. Open question: should specific belief categories (especially structural_future) carry an explicit confidence vector externally (e.g. a probability distribution over rival explanations) rather than collapsing to a single posterior? Recommendation: ship single-number posteriors externally for launch; revisit for structural_future beliefs after 6 months of scored data.

D/10 · Cross-entity belief linking. When two entities carry semantically equivalent beliefs (“supplier-concentration risk” on Entity A and Entity B), should the substrate link them? Linking enables sector-pattern detection at the belief level (the v4 Dot Connector amendment depends on it). Risks: spurious linking from prompt-similar belief statements, contamination across tenants. Recommendation: phase 3 of §42 introduces canonical belief categories with curated linking; tenant-private beliefs never link across tenants.

D/11 · Customer-private vs canonical link visibility. A customer authoring a tenant-private belief that semantically matches a canonical belief should be allowed to link to the canonical belief for read-only access to its calibration history. Open question: under what privacy gradient is this safe? Recommendation: opt-in only, with the customer’s belief statement abstracted before any canonical-side write. See §29.

D/12 · Falsifier distance, continuous or five-state. v4 commits to five-state (`far / watch / medium / near / triggered`). A continuous representation is more expressive but harder to communicate and easier to overfit on. Recommendation: ship five-state. Internal representation may become continuous in v4.1 if calibration data shows the discrete states are too coarse.

D/13 · Update_case privacy gradient. Update_cases carry the full evidence packet, agent action, and system review. They are the corpus material. Customer-tenant cases are tenant-private by default. Open question: under what abstraction can a customer-tenant case contribute to the canonical corpus without leaking private positions or research? Recommendation: opt-in abstracted contribution path, where the customer authors a redacted version (entity anonymised, evidence-packet abstracted to a structural pattern) that becomes a canonical update_case while the original stays tenant-private. Ship the path in phase 4 of §42.

D/14 · Per-category auto-retirement window for stale beliefs. A belief that has not been touched by an update in N days is a candidate for retirement (it is no longer load-bearing for the thesis). The right N is per-category: a moat belief might reasonably go untouched for 9 months without being stale; a sentiment belief is stale after 3 months. Recommendation: ship with default per-category windows based on the launch-universe data; let the windows adjust as the system observes which beliefs actually move.

§26 Operational concerns deliberately out of scope

This document is an architecture document, not an operations playbook. Several large topics are absent because they belong elsewhere; naming them here is the architecture's commitment to not having silently overlooked them.

Deployment topology. Cloud provider selection, region distribution, multi-region failover, edge presence. The architecture is cloud-agnostic in its commitments and provider-specific in its operations. Decisions are deferred to operations.

Infrastructure-as-code. Terraform, Pulumi, Crossplane, in-house tooling. The contracts in §17 must hold across whatever infrastructure expresses them; the expression itself is operational.

Observability and alerting. Dashboards, SLO definitions, on-call rotations, paging policy. The architecture commits to making the substrate observable (every commit logged, every agent invocation pinned to prompt version) but does not specify how operators consume that observability.

SOC 2 / ISO 27001 / regional compliance. The security primitives in §18 are designed to make these certifications achievable. Pursuing them is a commercial decision tied to enterprise sales motion.

Penetration testing, bug bounty, red-team programs. Operational. The security model in §18 is the architectural commitment; the testing of that model against adversarial actors is operational.

Customer onboarding and support tooling. How customers actually start, what kind of help-desk interface their analysts have, how billing disputes are handled. All commercial-operational, not architectural.

Vendor selection above the architecture layer. S&P Capital IQ vs FactSet, Auth0 vs Clerk, AWS KMS vs GCP KMS. The architecture commits to capabilities (normalised financial history, OIDC, managed key store) and is deliberately silent on vendor identity.

Cost optimisation playbook. Token budgets per agent class, infrastructure right-sizing schedules, spot vs reserved capacity allocation. The metering discipline of §31 makes optimisation possible; specifying optimisation is operational.

Localisation and i18n beyond data acquisition. The data acquisition layer handles non-English filings via translation (§19). UI localisation, region-specific compliance text, local payment methods are commercial-operational.

The architecture earns its commitments by being silent on the things it does not commit to. Naming them as out-of-scope is the discipline that makes the in-scope commitments mean what they say.

Part V • The sealed client layer

The customer-facing extension surface. Reads canonical state freely. Writes nothing back. The seal is a runtime contract enforced by tenant isolation, audited continuously, and named honestly. v4 extends the seal to belief and update_case rows, adds an opt-in abstracted contribution path, and preserves every other commitment from v3.1.

§27 The sealed client workbench, additive tenant-isolated

Some customers want to extend the canonical system with their own logic: house theses, proprietary scoring rubrics, internal signals, risk overlays, **proprietary belief categories**. They do not want to share, cannot afford to leak, and would not codify inside a vendor's namespace. The client workbench exists for those customers. It is self-service, tenant-isolated, and additive: customers build their own agents alongside the canonical engine without ever modifying it, without exposing their logic, and without any path back to the canonical core.

The workbench is built on top of the canonical six-layer engine and the v4 belief journey. It inherits the canonical entity trace (baselines, deltas, insights, hypotheses, predictions, **belief vectors**, **belief_updates**, **alternative_explanation_sets**, Threadweave paths, **price_signal rows**) in read-only form. From there, the customer creates additional agents, authors private chapter formats, **authors tenant-private beliefs**, produces customer-private deltas and belief_updates, configures Threadweave parameters within their namespace, and maintains a private learning loop scored against customer-defined outcomes. Everything they build is theirs. Nothing they build crosses back.

The customer's logic does not alter the canonical system. Customer agents run alongside canonical agents, not in place of them. The canonical Threadweave continues to run for every entity, unchanged; what the customer can do is configure their own parameter overrides (horizon weights, competitor cohorts, sector tilts) that produce a customer-specific Threadweave reading derived from the same canonical engine. Canonical calibration learns from canonical predictions and canonical belief_updates only. The canonical system is unaware of the customer layer's existence by design.

Sealing is enforced as a tenant-isolation contract in the runtime, not as an architectural impossibility. Customer agents are real code running in customer namespaces, behind isolation boundaries that the rest of the system does not have a path through. There is no API, no internal queue, and no shared write surface that lets a customer agent reach into canonical state. The seal is in the access matrix, the schema partitioning, the queue keys, and the deployment topology. It is not unbreakable in principle; it is unreachable in practice. We name this honestly because architectural honesty is the precondition for the seal mattering at all.

§28 The workbench, what customers can build

The workbench surfaces six classes of capability. Each is sealed in the runtime; together they make the canonical system genuinely extensible without compromising the moat.

Custom agent definition. Customers author agent system prompts, tool configurations, validation rules, and parameter overrides within their tenant namespace. The agent runs against the canonical entity trace (read-only) and writes to tenant-scoped delta rows. Canonical agents do not see these.

Custom belief authoring. Customers can author tenant-private beliefs that supplement the canonical belief vector for any entity. A custom belief carries the same schema as a canonical belief but is scoped to the tenant. Custom-belief updates run through the same belief journey (priors, rivals, posterior, falsifier-distance) but the updates are tenant-private. Canonical agents and the canonical Belief Extractor do not see custom beliefs. Customers can read the canonical belief vector for any entity they have access to, supplement it with their own beliefs, and run their own analysis on the union.

Custom delta and insight production. Customer-defined agents produce delta rows scoped by `tenant_id`. The canonical Orchestrator does not dispatch on these; the canonical Dot Connector does not include them in its sweep. Customer agents can produce their own insight rows with their own evidence chains.

Custom hypothesis and prediction. Customer-authored hypotheses and the predictions they trigger live in tenant-scoped tables. Canonical Threadweave does not consume them. Canonical Evaluator does not score them. Canonical calibration does not learn from them.

Custom outcome and learning. Scored against customer-defined outcome streams. Customer Evaluator runs in the tenant namespace. Customer calibration is computed and stored per tenant. A customer agent's calibration trajectory has no influence on the weight canonical applies to its own equivalent agent.

Custom Threadweave parameter overrides. Customer-specific horizon weights, competitor cohorts, sector tilts, reliability shapes, and (in v4) **belief-importance overrides** are tenant-scoped configuration. The canonical Threadweave engine reads these only when running for that tenant; canonical Threadweave's default behaviour is unchanged for every other tenant and for the canonical-only view.

A worked customer example

A long-only equity fund subscribes to the EU industrial mid-cap universe. Their analyst extends the canonical layer with three additions:

1. A custom belief on each tracked entity for "supply-chain political-risk exposure," with a category they have defined (`political_risk`) and an `expected_evidence` packet they have specified (sanctions exposure, geographic concentration of suppliers, lobbying expenditure on relevant trade policy).
2. A custom Deep Insight specialist `XX/01 ESG-screen` that runs after every canonical inline loop and writes tenant-private deltas about ESG-screen criteria the canonical layer does not track.
3. A custom Threadweave override that downweights the long-horizon reachable states of any entity scoring below the fund's ESG threshold.

All three are tenant-private. The canonical entity trace for these entities continues to update for every other customer using the canonical engine; this customer's view of the same entities adds their custom beliefs, deltas, and Threadweave readings on top.

§29 The seal, what never crosses the boundary (amended in v4)

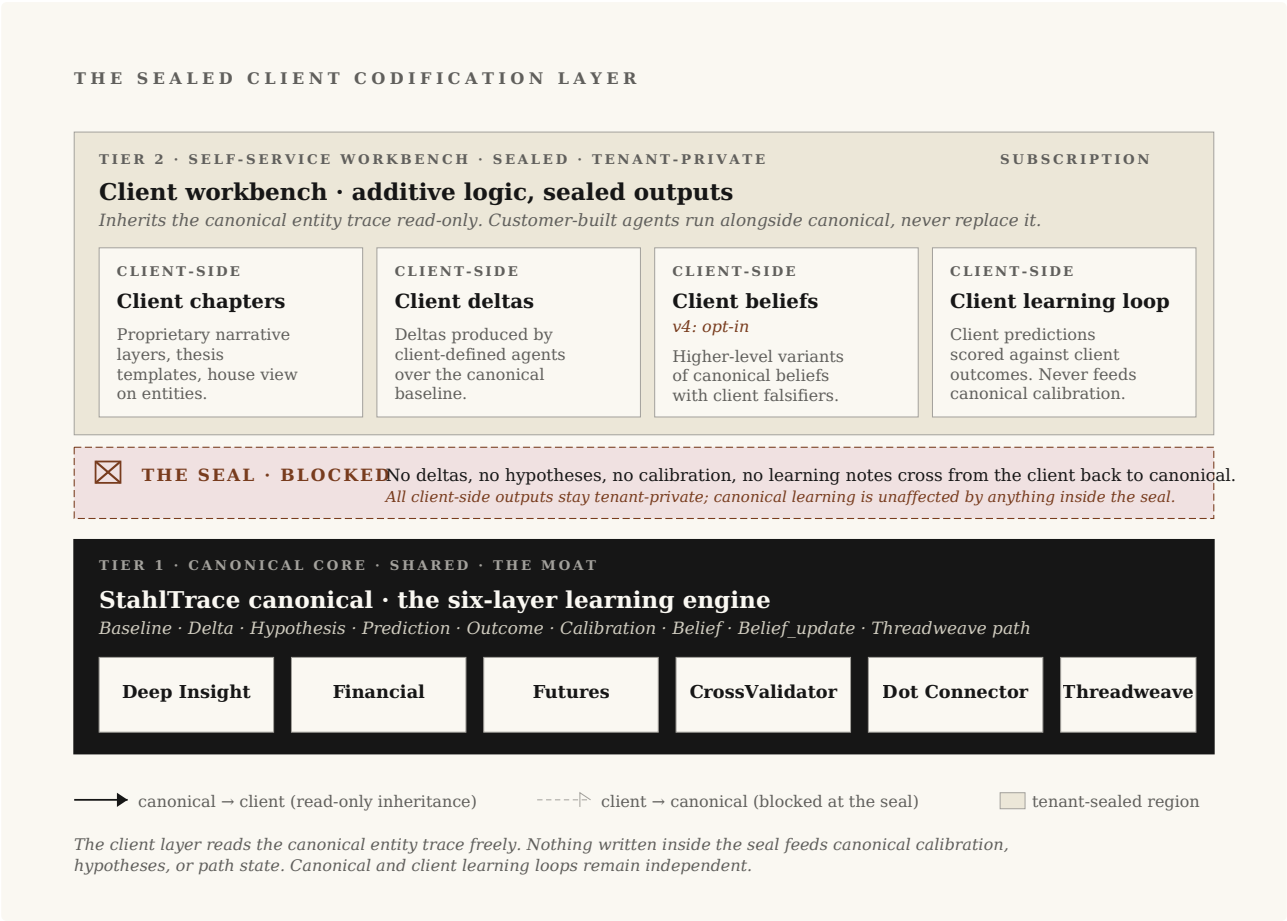


FIGURE 4 · THE SEALED CLIENT LAYER, INHERITING THE CANONICAL TRACE WITHOUT FEEDING BACK INTO IT. V4 ADDS A FOURTH CLIENT-SIDE SURFACE, CLIENT BELIEFS, ACCESSIBLE ONLY VIA THE OPT-IN ABSTRACTED CONTRIBUTION PATH OF §29.

The seal is a runtime contract enforced by tenant isolation. The following things never cross from customer namespaces back into canonical, not because the architecture makes it impossible in principle, but because no path, no API, no shared write surface exists in the running system that would let it happen. The seal is unreachable in practice, audited continuously, and named honestly.

What is sealed (extended in v4)

Customer deltas and insights. Produced by customer-defined agents, written to delta and insight rows scoped by `tenant_id`. Canonical agents have no read path to tenant-scoped rows. The canonical Orchestrator never dispatches on them; the canonical Dot Connector never includes them in its sweep.

Customer beliefs and belief_updates. (new in v4) Tenant-private beliefs live in belief rows scoped by `tenant_id`. The belief_updates that move them live in belief_update rows scoped by `tenant_id`. The `alt_explanation_sets` associated with them live in tenant-scoped rows. Canonical agents do not read tenant-scoped belief rows. Canonical Belief Extractor does not see them. Canonical Dot Connector does not synthesise across them.

Customer hypotheses and predictions. Customer-authored hypotheses and the predictions they trigger live in tenant-scoped tables. Canonical Threadweave does not consume them. Canonical Evaluator does not score them. Canonical calibration does not learn from them.

Customer outcomes and learning. Scored against customer-defined outcome streams. Customer Evaluator runs in the tenant namespace. Customer calibration is computed and stored per tenant. A customer agent's calibration trajectory has no influence on the weight canonical applies to its own equivalent agent.

Customer agent prompts and configurations. Agent system prompts, tool configurations, parameter overrides, validation rules, and customer-proprietary reasoning patterns live in a tenant-private prompt and config store. Canonical operators do not have read access; tenant administrators can read and write within their tenant.

Customer Threadweave parameter overrides. Customer-specific horizon weights, competitor cohorts, sector tilts, reliability shapes, and v4 belief-importance overrides are tenant-scoped configuration.

Customer update_cases. (*new in v4*) Tenant-private update_cases (a customer's full belief-update trajectories with prior state, evidence packet, agent action, system review, accepted update, later outcome, calibration result) are scoped by `tenant_id` with `privacy_class = tenant_private`. Canonical corpus operations do not read them.

Customer-derived structured outputs. Reports, exports, dashboards, and downstream feeds derived from customer logic stay in the tenant namespace. Canonical never aggregates across tenants for any output that includes customer-derived signal.

What is shared (read-only, one direction)

Canonical entity trace. The customer workbench reads canonical Identity, Alias, Evidence, Baseline, Delta, Belief, Belief_update, Alternative_explanation_set, Insight, Hypothesis, Prediction, Outcome, Calibration, Update_case (canonical only), Threadweave-path, and Price_signal tables. The read is typed, scoped, and audited, but unrestricted in breadth. Every customer sees the same canonical trace.

Canonical schema, views, and engines. Customers consume canonical typed views as the inheritance surface. The Threadweave engine itself runs canonically; customers configure parameter overrides that the engine consumes when running for their tenant. A change to canonical schema, views, or engine logic propagates to every tenant; tenant-specific extensions stay local. Same applies to the v4 Belief Extractor and the rival-set generators: their canonical operation is shared; tenant-specific extensions are tenant-private.

The opt-in abstracted contribution path (new in v4)

Some customers will, over time, accumulate customer-private update_cases that represent valuable training signal: real reasoning under uncertainty paired with later outcomes, structurally novel because the customer's domain expertise produced them. The architecture commits to the seal holding by default and offering an opt-in path for those customers who choose to contribute abstracted versions to the canonical corpus.

The path is structural:

1. **Customer-side authoring.** The customer's tenant administrator initiates an abstracted contribution from a tenant-private update_case. The original case stays tenant-private.

2. **Abstraction.** The contributed version is a redacted derivative: the entity is anonymised (replaced with a structural descriptor like “mid-cap European industrial supplier with concentrated supplier base”), the evidence packet is abstracted to its structural pattern (specific dates, magnitudes, and identifiable details replaced with categorical descriptions), the customer’s agent identity is removed, and the customer’s proprietary reasoning is paraphrased to remove distinguishing prompts or rubric language.
3. **Customer review.** The customer reviews and explicitly approves the abstracted version before it is committed.
4. **Canonical commit.** The abstracted version commits to the canonical corpus with `privacy_class = tenant_opt_in_abstracted`. It carries no reference back to the originating tenant. The customer’s quota does not bill for canonical use of the contributed case.
5. **Canonical use.** The abstracted `update_case` is eligible for inclusion in the canonical corpus per §35. It is treated as a canonical row and contributes to the corpus the same way other canonical rows do.

The opt-in is per-case, not per-tenant. A customer who opts in once does not opt in for everything. The customer can revoke a contributed abstracted case; revocation removes it from future canonical reads but does not retroactively undo any model-level use that has already happened (since canonical agents do not directly train on the corpus, this is a non-issue at v4; if it changes in v5+, revocation semantics are revisited).

The architecture does not require this path. Customers who never opt in pay no penalty. The canonical corpus grows without it; the path exists as a way for customers who want to participate in the corpus to do so without breaking the seal. See §35 for the corpus implications and §42 phase 4 for the rollout sequence.

Usage metering as a deliberate seal exception, narrowly scoped

Token counts, entity registrations, **belief-vector sizes**, **belief_update volumes**, and aggregate usage roll up to canonical-side billing. What flows is metadata about consumption, never the agent prompts that produced the consumption, never the deltas or insights or `belief_updates` that resulted, never the customer’s reasoning. The seal in spirit holds; the metering surface is documented in §31.

The seal is one-way and irreversible by default. Canonical feeds client. Client never feeds canonical, except through the explicit, per-case, abstracted, customer-approved opt-in path of §29. If a feature of the canonical system would require client data to function, that feature is out of scope. Not a reason to break the seal.

Decision 13 in §25 flagged the privacy gradient on `update_cases`. The recommendation is the opt-in abstracted contribution path described here, ship in phase 4 of §42.

§30 Delivery and commercial shape

The architecture supports three commercial postures. Each maps to a different customer profile, a different latency expectation, and a different commercial structure.

Subscription tier (self-service workbench). Customer signs up, picks a tier, gets API access and the workbench. Three tiers (Starter, Professional, Enterprise) with progressive capabilities: entity coverage, **belief-vector storage**, custom-agent slots, Threadweave parameter overrides, integration features. Pricing per-seat-per-month plus per-entity-per-month plus token-based overage. The volume tier; most customers land here.

Enterprise SSO and managed deployment. Enterprise customers get SAML/OIDC SSO, dedicated tenant capacity, custom integration support. Same architectural surface; enhanced operational support. Pricing is committed-use with floor and ceiling.

Co-development engagement (rare). A small number of customers pay for direct collaboration on customer-specific extensions: bespoke validators, vertical adaptations, deep workbench customisation. Time-and-materials plus subscription floor. Reserved for strategic accounts; not the default motion.

The architectural surface is the same across all three. Sealing is the same. The runtime contract is the same. What differs is the operational support around the architecture, not the architecture itself.

Customer model

Every customer is a tenant. Every tenant has a unique `tenant_id` that scopes all tenant-private rows. Every tenant has a billing-responsible admin (one mandatory), zero or more builders, zero or more viewers. The customer model in §18 specifies what each role can do.

Entity subscription

Customers subscribe to entities from the canonical universe. A subscription gives the customer read access to that entity's canonical trace and write access to tenant-scoped extensions. Subscriptions count against tier allowance. Unsubscribing releases the read entitlement but preserves any tenant-scoped state the customer wrote (so a customer who unsubscribes and later re-subscribes can recover their custom beliefs, custom deltas, and custom learning trajectory).

§31 Metering, billing, and customer accounts

Metering exists because StahlTrace cannot charge for what it cannot see. The metering surface is the smallest possible window into customer consumption: enough to bill correctly, not enough to leak the seal.

What is metered

LLM token consumption. Per-tenant, per-agent-class, per-day. Summed for billing. The agent's prompt and output content is not part of the metering record; only token counts are.

Entity registrations. Per-tenant, per-day. The number of entities a tenant has subscriptions on. Billed against the tier's entity allowance with overage on a per-entity-per-month basis.

Belief-vector storage. (*new in v4*) Per-tenant, the count of tenant-private beliefs across all subscribed entities. Tier allowances apply (Starter: 0 custom beliefs per entity, Professional: up to 10 per entity, Enterprise: unlimited within reason). Overage charged on a per-belief-per-month basis.

Belief_update volume. (*new in v4*) Per-tenant, the count of tenant-private belief_updates per month. The throughput of the tenant's belief journey. Tier allowances apply with overage charged on a per-update basis.

Update_case storage. (*new in v4*) Per-tenant, the count of tenant-private update_cases retained. Tier allowances apply with overage charged on a per-case-per-month basis.

API request volume. Per-tenant, per-endpoint, per-day. Rate-limited per tier with overage allowed and billed.

Storage. Per-tenant total bytes (deltas, insights, belief_updates, predictions, custom configurations). Charged on a per-GB-per-month basis, with the first N GB included in tier.

Active session count. For seat-based tier components.

How metering enforces quotas

Every dispatch that will call an LLM consults the quota service before the activity begins. Tenant-scoped activities check tenant token and entity-count budgets; canonical activities check the canonical capacity envelope. Blocked dispatches surface a structured error to the customer rather than silently failing. Usage is metered at activity granularity so retries do not double-charge.

Billing transparency

Tenant admins see real-time usage dashboards showing consumption against tier allowance and projected overage for the current billing period. Per-entity, per-agent, and per-belief breakdowns are available. The dashboards do not reveal canonical-side operational metrics (canonical capacity utilisation, peer-tenant comparison, internal cost structure).

Billing reconciliation

Monthly. The metering store rolls up daily usage into monthly invoices. Disputes are handled out-of-band; the metering store is append-only so the underlying record is recoverable.

Customer accounts and lifecycle

Onboarding. Customer admin signs up, configures tenant, invites builders and viewers, subscribes entities. The metering counter starts at activation.

Quota changes. Tier upgrades take effect immediately. Tier downgrades take effect at the next billing period to avoid mid-period quota cliffs.

Suspension. Non-payment or terms-of-service violation triggers suspension. Suspended tenants retain read access to their existing tenant-private state but cannot write or run agents. The 30-day grace period before any deletion is hard-coded.

Cancellation. Customer-initiated cancellation triggers a 30-day data retention window during which the customer can export tenant-private state. After the window, tenant-private state is deleted. Aggregated metering history retains for the regulatory minimum.

§32 API and access surface

The architecture's external surface is an API. Customers integrate by reading canonical state, writing tenant-private state, and consuming streamed events. The API is what makes the architecture usable rather than merely admirable.

API shape

REST + JSON for synchronous operations. Server-sent events (SSE) for streaming subscriptions to real-time deltas, belief_updates, insights, hypotheses, and predictions. WebSocket for bidirectional flows where required (rare; reserved for high-throughput use cases).

Authentication

API tokens for customer-side integration; OIDC for human users in the dashboard. Tokens are tenant-scoped; a token authenticates the calling identity and resolves the tenant context for every request. Per-token rate limits and per-token revocation, both immediate.

Endpoints, by category

Identity and entity registration. Look up an entity by ticker, ISIN, name, LEI; subscribe and unsubscribe; list subscribed entities. Read-mostly.

Canonical entity trace. Read baselines, deltas, insights, hypotheses, predictions, **belief vectors**, **belief_updates**, **alternative_explanation_sets**, outcomes, calibration, Threadweave paths, **price_signal rows**. Filtered by entity, by date range, by layer, by belief category. The canonical read surface for tenants.

Custom agent management. Author, update, version, retire customer-defined agents. List tenant-scoped agents with their current calibration trajectories.

Custom belief management. (*new in v4*) Author, update, retire tenant-private beliefs. Read tenant-private belief vectors. Read merged (canonical + tenant) belief vectors for entities the tenant subscribes to.

Custom delta, hypothesis, and prediction submission. Write tenant-scoped delta, insight, hypothesis, prediction, and (in v4) belief_update rows on entities the tenant subscribes to. Server-side validation enforces typed schemas.

Outcome ingestion. Customers can stream their own outcomes (proprietary data feeds, internal benchmarks) for tenant-scoped scoring.

Threadweave parameter overrides. Configure horizon weights, competitor cohorts, sector tilts, **belief-importance overrides** for the customer-specific Threadweave reading.

Subscriptions and streams. Real-time SSE streams for deltas, belief_updates, insights, hypotheses, predictions on subscribed entities. The Delta Feed (§33) is the v3.1 streaming surface, extended in v4 to surface belief_updates as first-class events alongside deltas.

Audit and dashboards. Read tenant-scoped audit logs, usage metrics, calibration dashboards, billing.

Error model

Structured errors with stable error codes. HTTP status code matches the semantic class (400 for client error, 401 for auth, 402 for quota exhausted, 403 for tenant-isolation violation attempts, 409 for conflict, 422 for typed schema validation failure, 500 for server error). Error responses include a stable `code` field for programmatic handling and a human-readable message.

SDKs

Python and TypeScript SDKs at launch; others (Java, Go, Rust) deferred to demand. The SDKs wrap the REST/SSE surface with type generation from the canonical schema, retry policies, and idiomatic streaming primitives.

Webhook integration

Tenants can configure webhooks that fire on tenant-scoped events: a new prediction, a resolved outcome, a new high-priority insight, a belief moving past a falsifier-distance threshold. Webhooks are at-least-once with idempotency tokens; tenants are responsible for de-duplicating on their side.

What the API does not expose

Canonical-side internal metrics (capacity, peer-tenant comparison, internal cost structure). Direct database access. Raw evidence payloads beyond what the entity trace surfaces (evidence is read through the typed views). Cross-tenant aggregations of any kind.

§33 The Delta Feed (amended in v4)

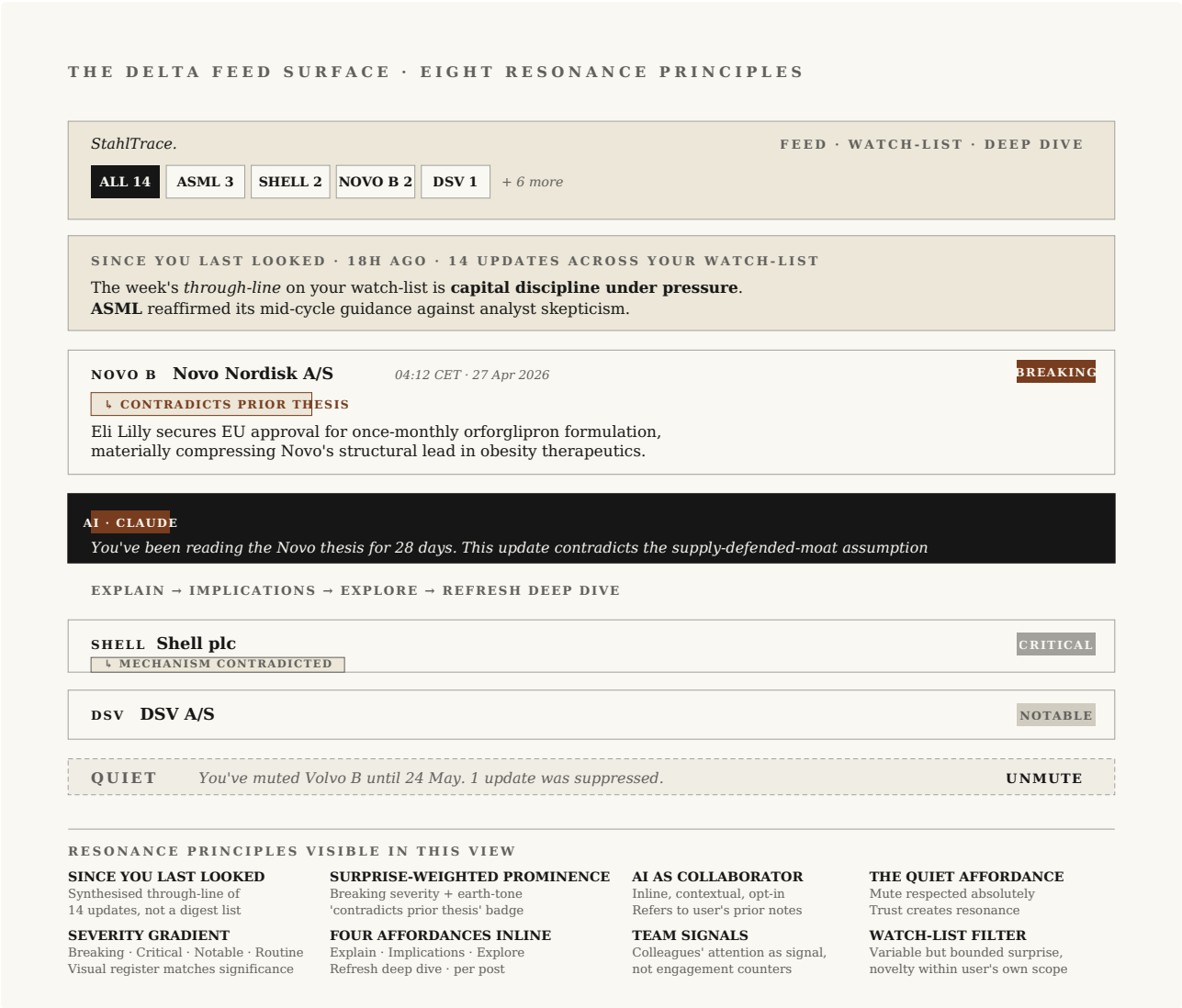


FIGURE 5 · THE DELTA FEED SURFACE, EIGHT RESONANCE PRINCIPLES RENDERED. THE FEED SHOWS ARCHITECTURAL ELEMENTS AS THEY APPEAR TO THE OPERATOR: THE THROUGH-LINE SYNTHESIS, THE PER-UPDATE CONTRADICTION BADGES, AI AS INLINE COLLABORATOR, AND THE QUIET-AFFORDANCE RESPECTING MUTES.

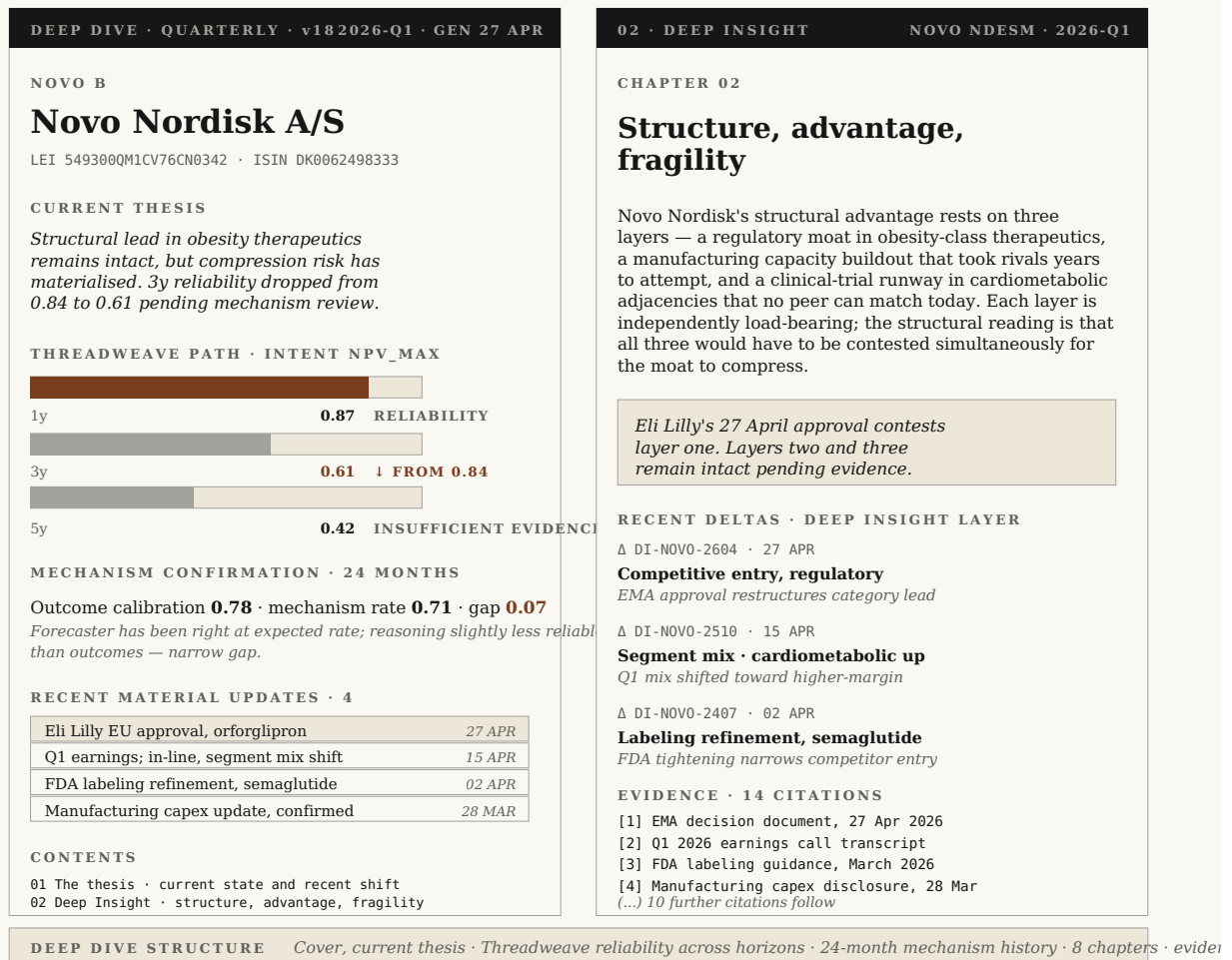


FIGURE 6 · THE DEEP DIVE DOCUMENT LAYOUT. COVER PAGE (LEFT) SHOWS CURRENT THESIS, THREADWEAVE RELIABILITY ACROSS HORIZONS, MECHANISM CONFIRMATION HISTORY, AND A CONTENTS REGISTER. THE CHAPTER EXCERPT (RIGHT) SHOWS HOW EACH CHAPTER FOREGROUNDS RECENT DELTAS AND CITATIONS ALONGSIDE NARRATIVE.

The Delta Feed is the streaming surface where customers consume the system's reactions in real time. v3.1 introduced it as the streaming counterpart to the entity-trace API: a per-tenant SSE stream of deltas, insights, hypotheses, and predictions filtered by entity subscription. v4 extends the Feed to surface belief_updates as first-class events alongside deltas, with backwards-compatible event types so v3.1 consumers continue to function unchanged.

Event types on the Feed

delta.committed. A delta has been committed for a subscribed entity. Carries delta_id, entity_id, layer, magnitude, summary, and references to the deltas it superseded.

belief_update.committed (new in v4). A belief_update has been committed for a subscribed entity. Carries update_id, belief_id, entity_id, prior_confidence, posterior_confidence, prior_falsifier_distance, posterior_falsifier_distance, dominant_shift, layer, and references to the parent delta. The customer-facing presentation uses confidence-band language per §15, with the underlying numeric posterior available for customers who explicitly request raw form.

belief.falsifier_movement (new in v4). A belief has crossed a falsifier-distance threshold (e.g. moved from watch to medium, or from medium to near). Surfaces independently from the underlying belief_update so customers can subscribe specifically to falsifier movements, the highest-leverage signal for analysts watching watch-list-style risk.

insight.published. A Dot Connector insight has been published for a subscribed entity. Carries insight_id, statement, evidence chain, layer evidence, forecastable flag, and a pointer to the child hypothesis if forecastable.

hypothesis.emitted. A child hypothesis has been emitted from an insight. Carries hypothesis_id, statement, falsifier, horizon tier, parent insight reference.

prediction.emitted. A prediction has been emitted from a hypothesis. Carries prediction_id, direction, probability, mechanism, falsifier, horizon, and (in v4) expected_price_reaction band where applicable.

outcome.resolved. An outcome has resolved one or more open predictions for a subscribed entity. Carries outcome_id, entity_id, the predictions resolved, and the verdicts (outcome, mechanism, and (in v4) per-track belief-update verdicts).

update_case.resolved (new in v4). A canonical update_case has been resolved with its per-track calibration result. Carries case_id, calibration_result.

Filter language

Customers can filter the Feed by entity, by event type, by layer, by belief category, by magnitude, by falsifier-distance threshold, by horizon tier. The filter language is typed and validated server-side; invalid filters return 422 errors rather than silently delivering everything.

Delivery semantics

At-least-once delivery with monotonic event sequence numbers per tenant. Customers acknowledge events explicitly; un-acknowledged events redeliver after a configurable window. The Feed is durable: a customer can resume from any sequence number within the retention window (default 30 days).

Backwards compatibility

The v3.1 Feed event types continue to work. v4 consumers see the new event types in addition; v3.1 consumers see only the old types and miss the v4 events. Customers explicitly opt in to v4 event types via a Feed-version header; the architecture commits to keeping v3.1 Feed support for at least 24 months after v4 launch.

What the Feed does not do

It does not stream raw evidence payloads (the evidence is referenced by ID; customers fetch the payload through the entity-trace API if needed). It does not stream cross-tenant patterns. It does not stream canonical-side operational events. It does not stream customer-private belief_updates or update_cases (the Feed is canonical to tenant; tenant-private state stays in the tenant namespace).

The Feed is the customer's window into the architecture's ongoing work. v4 widens that window to include belief moves, falsifier movements, and per-track calibration verdicts. The v3.1 commitments hold; v4 adds.

Part VI · R&D roadmap

What becomes possible after the architecture runs. The canonical architecture (Parts I to V) commits to what gets shipped. This part describes directions the architecture has been designed to enable later, but does not commit to building. Reading the canonical architecture as if it included these commitments would mislead. Reading this part as a roadmap of what is already being built would also mislead. The separation is deliberate, and the chapter is short by design.

§34 Framing, what this part is and is not

The architecture in Parts I to V is a working system. Operate it for long enough, against enough entities, with enough discipline, and several things emerge as natural consequences of the operation rather than as separate engineering ambitions. **In v4, with the belief journey running, the corpus that emerges is materially richer than the v3.1 version: not just resolved predictions with provenance, but full belief-update trajectories paired with reality.** Six directions described here. None is a v4 launch commitment.

What is being committed in this chapter is something narrower: the canonical architecture is designed in a way that makes these directions possible later, and the substrate captures the data those directions would require. Whether and when to pursue them is a decision that depends on operating data the architecture does not yet have.

The chapter is conservative in its claims. The substrate that emerges from operating the architecture is genuinely valuable; whether it leads to a categorically different kind of forecasting AI, a transformative model of business reasoning, or simply better prompts and broader coverage, is a question for empirical testing on real accumulated data. The honest version of this chapter does not predict the answer. It describes the substrate, the directions it enables, and the conditions under which each direction would become tractable.

Three primary directions, one underlying argument

The corpus is what gets produced. A multi-year structured record of business reasoning paired with reality. **In v4, the unit of corpus is the closed update_case: prior state, evidence packet, agent action, system review, accepted update, later outcome, per-track calibration result.** After 24 months of operation across the launch universe, the corpus is a research asset that does not exist anywhere else, because nobody else runs the architecture that produces it.

Prompt evolution is what gets done with it. The first concrete capability the corpus enables. Once the system can see, across a real cohort, which reasoning patterns produce mechanism-confirmed predictions and well-calibrated belief_updates, and which produce mechanism-contradicted ones and poorly-calibrated belief_updates, prompts can be refined systematically rather than by intuition.

Coverage economics is what makes the corpus large enough to matter. The corpus is only as valuable as the diversity of entities and regimes it covers. The architecture has been designed so that the marginal cost per tracked entity is dominated by automation rather than by analyst time. That cost structure opens the door to coverage of the uncovered universe.

Read together: the architecture produces a corpus, the corpus enables prompt evolution, economics enables enough coverage to make the corpus large enough to mean anything. None of these is research speculation. All three follow from operating the system as designed for long enough.

§35 The corpus, belief-update trajectories paired with reality (*amended in v4*)

The richest dataset that emerges from running this architecture is not the predictions, the calibrations, or the path projections. It is the paired record of reasoning and outcome: every belief the system ever held, every update that moved it, the rival explanations considered at the moment of the update, the diagnosticity claimed for the evidence, and the eventual reality that confirmed, contradicted, or refined the claim. **In v4, this is materialised as the update_case ledger.**

This kind of data does not exist in public corpora. Equity research reports are written for persuasion, not falsifiability; they tend to be retrospectively rationalised rather than scored against original commitments. Academic finance datasets contain prices, fundamentals, and event timestamps, but rarely the structured reasoning that preceded the events. Internal hedge fund archives are private and even when accessed are rarely structured for falsifiability. **Most importantly, no existing corpus carries the rival-explanation-set and diagnosticity-claim structure that the v4 belief journey produces, which means no existing corpus supports the empirical study of what good Bayesian reasoning under uncertainty actually looks like at scale.**

What is in the corpus

Resolved update_cases with full provenance. (*new in v4*) For each closed update_case: the prior state (belief vector + falsifiers + valuation snapshot), the evidence packet, the agent action (which beliefs moved, which rivals were considered, what diagnosticity was claimed), the system review (ensemble disagreement, diagnosticity verdict, mechanism verdict, cross-validator notes), the accepted update with prior and posterior, the later outcome that resolved it, and the per-track calibration result (Track 1: posterior calibration, Track 2: update direction correctness, Track 3: diagnosticity calibration). The unit of corpus.

Resolved predictions with full provenance. Carried forward from v3. For each closed prediction: the originating claim and evidence, the agent that proposed it, the prompt version used, the stated mechanism, the falsifier, the horizon, the timestamp, the outcome that resolved it, the verdict (confirmed, falsified, partial, unresolvable), the mechanism verdict (confirmed, unconfirmed, contradicted). **In v4, also linked to the belief_updates that the prediction was derived from.**

Insight chains. The Dot Connector's cross-layer syntheses, with the deltas, **belief_updates**, and prior insights they drew from, the hypotheses they spawned, the outcomes those hypotheses eventually resolved to. An insight that proved load-bearing across multiple downstream predictions is structurally distinguishable in the corpus from an insight that resolved nothing.

Factor co-occurrence histories. The full `entity_factor_observation` ledger across years and entities, joined to outcomes. **In v4, also joined to belief_updates so that factor co-occurrence with belief moves becomes a separately-studyable phenomenon from factor co-occurrence with predictions.**

Intent profiles and deviations. Per-entity intent assumptions, the evidence supporting each, the deviations recorded when companies acted unexpectedly, the eventual resolution of those deviations. A record of when stated intent and revealed intent diverged.

Future-state validations. Structural future-world claims authored at one time, with their evidence, their probabilities, and the eventual reality at the target horizon. **In v4, structural-future beliefs (category `structural_future`) carry their own update trajectories with rival sets specific to long-horizon claims.**

Rival-explanation trajectories. (*new in v4*) For each belief, the historical sequence of `alternative_explanation_sets` considered at update time, the dominant shifts that were claimed, the diagnosticity verdicts that came back at resolution. The corpus enables empirical study of which rival explanations turn out to dominate at which kinds of evidence patterns, across which entity types, across which market regimes.

Price-residual histories. (*new in v4*) The `price_signal` time series joined to `belief_updates`. Empirical study of when price moves are diagnostic for business beliefs (residual outside expected_price_reaction band), when they are non-diagnostic (residual inside band), and when they are misleading (residual signal that did not survive resolution).

What the corpus enables, conservatively framed

The corpus is good for research that is currently impossible because the data does not exist. Several research questions become tractable that are not tractable today:

Reasoning patterns that produce well-calibrated belief_updates. Across entity types and market regimes, which reasoning structures (counterfactual checks, base-rate anchoring, explicit falsifiability, mechanism decomposition, **rival-explanation enumeration**) correlate with high posterior calibration on Track 1, correct update direction on Track 2, and as-claimed diagnosticity on Track 3. Becomes an empirical question rather than a methodological preference.

Drift in what is actually predictive of outcomes over multi-year horizons. Signals that worked in one regime and stopped working in the next are observable in the corpus rather than only in retrospect. The shape of signal decay (how fast, how systematically, with what early indicators) becomes researchable.

Calibration on long-horizon structural forecasts. The kind of predictions §13 describes as “future-state” claims becomes empirically grounded rather than speculative. A few years of corpus data is enough to begin scoring agents on whether their structural predictions about markets at 3 to 5 year horizons resolve accurately.

The empirical structure of rival-explanation distributions. (*new in v4*) What rivals dominate at what kinds of evidence patterns, in what entity types, in what regimes. Whether the system’s rival sets are too narrow (missing dominant explanations that emerge), too broad (carrying rivals that never gain mass), or correctly scoped. Currently unstudied at scale because no data of this shape exists.

Diagnosticity drift detection. (*new in v4*) When a class of evidence (a particular filing pattern, a particular news source, a particular price-signal residual shape) starts proving over-claimed or under-claimed in diagnosticity at scale, the drift itself is signal. Currently unstudied because no system tracks claimed diagnosticity at this granularity over time.

Privacy and contribution

Canonical update_cases. Generated by the canonical architecture against the canonical entity universe. Eligible for canonical research without further opt-in.

Tenant-private update_cases. Generated inside customer tenants. Not eligible for canonical research; stay tenant-private.

Tenant-opt-in abstracted update_cases. (*new in v4*) The contribution path of §29. Customer-private cases that the customer has explicitly approved for canonical inclusion in abstracted form. Eligible for canonical research; abstracted to remove identifying detail.

The architecture commits to being able to grow the canonical corpus without depending on tenant-private contributions. The opt-in path exists for customers who want to participate, not as a structural prerequisite for the corpus to be useful.

Time horizon

For the corpus to support the research questions above with statistical power, several thousand resolved update_cases per belief category are needed. At launch (100 entities, 8 to 25 active beliefs per entity, several material updates per belief per year), the canonical universe is on track to accumulate this volume in 2 to 3 years of operation. The R&D directions in §36 onward become tractable on roughly that timeline.

The corpus is not a research speculation. It is what the system produces by operating. The question is what to do with it once it exists.

§36 Prompt evolution, calibration applied to the prompts not just the outputs

The first concrete capability the corpus enables is systematic prompt evolution. Once the system can see, across a real cohort, which reasoning patterns produce mechanism-confirmed predictions and well-calibrated belief_updates, and which produce mechanism-contradicted ones and poorly-calibrated belief_updates, prompts can be refined empirically rather than by intuition.

What “prompt evolution” means here

Today, agent system prompts are authored by engineers based on a combination of design intent, intuition, and small-scale experimentation. They evolve through manual revision: someone notices a pattern, hypothesises a fix, edits the prompt, observes whether things improve. This works at small scale but does not compound.

At corpus scale, prompt evolution becomes evidence-driven. The Evaluator has, for each prompt version of each agent, a set of resolved predictions with mechanism verdicts and resolved belief_updates with three-track calibration. A prompt revision that systematically improves mechanism-confirmation rates, improves Track 1 posterior calibration, improves Track 2 update direction correctness, or reduces Track 3 diagnosticity over-claiming is empirically better. A prompt revision that does not is empirically worse, regardless of how plausible it sounded.

What the architecture enables, what it does not

Enables. Tracking calibration trajectories per (prompt version, agent class, belief category, horizon tier, market regime). Surfacing patterns of failure (this prompt over-claims diagnosticity on regulatory-exposure beliefs). Comparing prompt variants empirically when scored data accumulates.

Does not enable, by design. Automatic prompt rewriting at scale based on calibration. The architecture deliberately stops short of meta-prompting (an agent that rewrites other agents' prompts based on observed performance) for two reasons: (1) the meta-agent's own calibration discipline would have to be established first, and the substrate to do that does not exist at v4; (2) automatic prompt rewriting introduces a feedback loop between prompts and the corpus that contaminates prompts as a calibration variable. Manual revision based on empirical signals from the corpus is the v4 commitment.

v4 specific evolution

The v4 belief journey introduces three new evaluation surfaces (Track 1, Track 2, Track 3) which means three new dimensions on which prompts can be evaluated. Specifically, the rival-explanation-generation prompts and the diagnosticity-claim prompts inside layer agents are new prompt classes that did not exist in v3.1. Their performance is measured empirically the same way the rest of the system is.

§37 Coverage economics

The corpus is only as valuable as the diversity of entities and regimes it covers. Three things determine the marginal cost per tracked entity at scale:

Acquisition cost. Filings, news, transcripts, vendor data. Roughly fixed per entity per month at launch (vendor licences) and falls per-entity as universe grows (licences are typically not per-entity above some baseline). v4 adds the price_signal feed, which is cheap.

Inline-loop compute. Per material delta, the inline loop runs the layer agents + CrossValidator + the v4 belief journey. Token cost scales with delta volume, not with entity count directly. v4 belief journey adds approximately 30 to 40% inline compute per material delta over v3.1.

Sweep compute. The Dot Connector's nightly sweep runs per active entity. Token cost scales with entity count and (in v4) with belief-vector size. Active universe at launch is small enough that this is bounded.

The total marginal cost per tracked entity at v4 launch is estimated at \$7 to \$9 per month at 100 entities (up from v3.1's \$5 to \$6, reflecting the v4 belief layer's compute additions). The cost falls roughly hyperbolically as universe grows because vendor licences amortise.

The economic argument: at \$7 to \$9 per entity per month, the architecture can defensibly track the uncovered universe (publicly listed companies that current research economics ignore because per-entity analyst cost is too high). At a few thousand entities, the corpus reaches volumes that make the §35 research questions tractable. The economics enable the corpus; the corpus enables the research; the research compounds the system's value.

§38 Public Actor Trace, the Worldview & Game Layer

The architecture's primary commitment is finance. It is also domain-general. Public Actor Trace is the deferred extension to other domains where persistent agents make decisions under uncertainty, and where the same lived-record, belief-update, mechanism-attribution discipline produces a comparably valuable corpus.

What it is

A separate vertical (or set of verticals) using the same architecture: same six-layer stack, same belief journey, same substrate, same scoring discipline, applied to entities that are not companies. Examples include: government policy actors (decisions under uncertainty, accountable to electorates and constituents, with measurable outcomes), regulatory bodies (decisions with deferred consequences and observable enforcement patterns), large institutional investors (positions with disclosed timestamps and resolved performance), foundations and large-grant programmes (multi-year commitments with measurable outcomes).

Why it is deferred

Three reasons. The architecture's discipline has not been validated against any vertical yet; pursuing two simultaneously dilutes the discipline. The data acquisition layer is finance-specific at launch (EDGAR, GLEIF, S&P Capital IQ); other verticals require parallel acquisition pipelines. The customer motion is finance-specific at launch; pursuing other verticals requires a parallel sales motion the company is not yet equipped for.

What the architecture commits to

The substrate is domain-general; entities are typed, but the typing is not finance-specific. The belief schema is category-driven, and categories are open-ended; new domains add new categories. The agent set is structurally composable; new domains author new specialists alongside the finance-launch set. The Threadweave intent framework explicitly accommodates non-NPV objectives (`political_or_regulatory_objective` , `other`).

The architecture's domain-generality is the option value. v4 commits to finance; the substrate is what makes Public Actor Trace possible later, not what closes it off.

Guardrails on public-actor coverage

When Public Actor Trace ships, three guardrails are committed: (1) coverage is restricted to actors with sufficient public reasoning trace to support the architecture's evidence requirements (private deliberations are out of scope by design); (2) belief categories applied to public actors are conservatively narrow at launch (decision-making consistency, public reasoning quality, deviation from stated commitments) rather than aspirationally broad; (3) the calibration metrics published per actor make explicit the long-horizon nature of most public-actor decisions and the consequent calibration uncertainty.

§39 Preconditions

For the R&D directions in this part to become tractable, several preconditions must hold. These are not commitments; they are conditions under which the directions stop being aspirational and become work.

The architecture must run for long enough to accumulate corpus volume. §35 estimates 2 to 3 years for the launch universe to reach research-tractable corpus size. Premature pursuit of R&D directions before this volume exists produces preliminary work, not foundational work.

Calibration must remain credible. If the canonical calibration discipline drifts (Evaluator over-confirming mechanisms, prompt versions silently re-attributed, retrospective passes contaminating live records), the corpus loses its scientific value before research can use it. The v4 commitment to autonomous-only scoring with deterministic gates (§42) is in part a precondition for the corpus to be useful.

Tenant opt-in for the abstracted contribution path must materialise. (*v4 specific*) The canonical universe alone may not produce enough diversity of entity types, market regimes, and reasoning patterns for some R&D questions. The opt-in abstracted contribution path of §29 is the architecture's way of growing diversity without breaking the seal. If no tenants opt in, the corpus is bounded by canonical-only growth, which still supports many research questions but not all.

Compute economics must hold. The §37 cost analysis assumes vendor licences amortise, sweep compute scales sub-linearly, and inline compute per delta does not balloon as the belief layer matures. If any of these assumptions fails, coverage economics constrain the corpus and the directions in this chapter become more expensive to pursue.

Operational discipline must compound. The §17 runtime contracts, the §22 access matrix, the §29 seal, and the §42 phase plan are operational commitments that have to hold continuously. Slippage on any of them does not just cost a feature; it costs corpus value, because the corpus is only as good as the discipline that produced it.

What the architecture does not commit to

It does not commit to any of the R&D directions becoming products. It does not commit to causal modelling, agent simulation, future-state forecasting, prompt evolution, or Public Actor Trace as named features in any specific release. It commits to building an architecture where these directions are structurally possible later, and to operating that architecture with enough discipline that the data they would require is genuinely there when the time comes.

The R&D roadmap is conditional. The architecture's commitments are not. The separation is what keeps the commitments meaningful.

Part VII · The Bayesian belief layer

This part describes the architecture's most consequential v4 change: the elevation of beliefs and belief updates from implicit consequences of the delta stream to explicit, typed, scored objects. The change is structural, not cosmetic. It alters what the substrate stores, what the agents emit, what the scoring methodology measures, and what the sealed client can present. The four sections that follow describe the change in increasing depth: §40 establishes price as a Bayesian evidence channel and

explains the architectural separation that keeps it from contaminating mechanism reasoning; §41 specifies the four-layer posterior structure and the falsifier ladder that makes belief revision concrete; §42 describes the phase plan that takes the architecture from v3.1 substrate to full v4 operation, and defends the autonomy commitment that runs through every phase; §43 is a decision memo that summarises what is being committed to, what is at risk, and what remains open.

§40 Price as Bayesian evidence

The problem v3.1 left implicit

The v3.1 architecture treated price as one signal among many. CrossValidator could check that an emitted view did not contradict observable price action. The Financial Model could read price history into its valuation work. The Macro & Cycle agent could read regime indicators that included price-derived series. None of this was wrong, but it left a question unaddressed: what role does a price move play when an agent has already authored a belief, and the price subsequently moves in a direction that information theory says should update that belief?

The v3.1 answer was, in effect, “the next sweep will reconsider.” A price move would be visible in the next agent run; agents would re-emit views; CrossValidator would check consistency; deltas would be issued where views had genuinely shifted. This works for slow, structural revisions. It does not work for the case the architecture is supposed to be best at: the case where a new piece of evidence (a price move, a filing line, a transcript phrase) should sharpen or weaken a belief the system already holds, and the sharpening or weakening is itself the news.

The v4 architecture makes the role explicit. Price is a Bayesian evidence channel: it carries information about beliefs, and that information must be combined with prior beliefs in a defined way, with defined safeguards, to produce posterior beliefs.

The decomposition that keeps it honest

Treating price as evidence without safeguards is dangerous. Price moves for many reasons: information arrival, liquidity events, positioning changes, regime shifts, narrative momentum, reflexive feedback loops. Naively updating belief on every price move is a recipe for the system to track market mood rather than the underlying world. The architecture’s defence is a decomposition.

Every belief that is exposed to price evidence carries an `expected_price_reaction` band. The band is a category-conditional, agent-authored estimate of how the market should move if the belief is correct, expressed as a directional band (e.g., “modest upward drift with quarterly earnings cadence”, “sharp downward reaction on regulatory tightening events”). The band is not a price target; it is an expectation about the relationship between belief truth and price behaviour, against which actual price moves are measured.

When price moves, the architecture computes a residual: actual price action minus expected price reaction conditional on the current belief. The residual is the *unexpected* component. Only the unexpected residual receives Bayesian weight on the underlying business or world belief. The expected component is treated as confirming the belief at its current strength; it does not produce a fresh update. This is the architecture’s primary defence against double-counting evidence and against drift toward market-tracking.

A worked illustration: the architecture holds belief `B_growth_durability = high` for an entity. The associated `expected_price_reaction` band is “moderate positive drift with quarterly results, shallow drawdowns on macro shocks.” The entity reports earnings; price rises five percent. If the band’s quarterly expectation was “three to seven percent up on a beat”, the unexpected residual is small, and the belief is confirmed at its existing strength

without being sharpened. If price had risen twenty percent, the unexpected residual is large and positive; this is genuine new evidence that the belief was understated, and a Bayesian update follows. If price had fallen ten percent on the same beat, the unexpected residual is large and negative; this is evidence against the belief, and an update in the opposite direction follows.

Likelihood ratios, internally precise; bands, externally surfaced

Inside the architecture, the strength of an update is represented as a likelihood ratio: the conditional probability of the observed evidence given the belief, divided by the conditional probability given the belief's primary alternative. Likelihood ratios are the right internal representation because they compose multiplicatively across independent evidence and admit clean Bayesian arithmetic.

Externally, in sealed client surfaces and agent-emitted text, the architecture does not expose raw likelihood ratios. They are translated into a five-band qualitative scale: High, Medium-high, Medium, Medium-low, Low. The translation table is fixed at calibration time and does not vary by surface. The motivation for the translation is twofold: (1) raw likelihood ratios convey false precision to operators, who will read 4.7 differently from 4.2 when the difference is within calibration noise; (2) the bands are stable across small calibration adjustments in a way that raw ratios are not.

The scoring methodology (§15) measures both: the internal likelihood ratios are checked for arithmetic coherence (multiplicative composition produces consistent posteriors regardless of evidence ordering), and the external bands are checked for calibration (Medium-high updates should, over time, be associated with the predicted posterior shifts at the predicted frequencies).

Three update kinds

The architecture distinguishes three kinds of belief update, each authored against a different evidence template:

Mechanism updates. New evidence about the causal mechanism a belief depends on. Filing language that confirms or contradicts the asserted business model; product disclosures that ratify or undermine the competitive thesis; management commentary that strengthens or weakens the inferred capital allocation pattern. Mechanism updates flow through Deep Insight, are scored by the Evaluator under the existing canonical methodology, and are the dominant kind of update in volume.

Tail updates. New evidence about the distribution of outcomes, particularly low-probability extreme outcomes. Regulatory action that opens a previously closed downside; technological breakthroughs that open a previously closed upside; structural shifts that change the shape of the distribution rather than its centre. Tail updates flow through Synthetic Futures, often involve regime reasoning, and are scored under the long-horizon methodology of §15.

Price updates. New evidence in the form of price action that, after subtracting the `expected_price_reaction` band, produces a non-trivial unexpected residual. Price updates are authored by a dedicated belief-update agent (§07), are constrained to operate only on beliefs with valid `expected_price_reaction` bands, and are subject to the diagnostic-test-elimination rule below.

The diagnostic test elimination rule

Price-derived updates carry a specific risk that mechanism and tail updates do not: the price move may have a confounding cause that the architecture failed to register. Liquidity events, index rebalances, large-block trades, derivatives expirations, and forced selling can all produce price moves that look diagnostic but are not. The architecture's defence is the diagnostic test elimination rule: a price move is admitted as evidence only if it survives a battery of diagnostic tests that attempt to eliminate it as confounded.

The tests are applied automatically by the belief-update agent and are cheap to run. They include checks for known liquidity events on the day, abnormal volume signatures consistent with mechanical flows, divergence from sector and factor moves of the same magnitude, and proximity to known scheduled events that might explain the move. If any test triggers, the price move is logged as observed but not admitted as Bayesian evidence on belief; it remains visible to other agents (Macro & Cycle, the Financial Model) for their independent purposes. If all tests pass, the move is admitted, the unexpected residual is computed, and a price update is authored.

The architectural separation

A central commitment of the v4 design is that price evidence updates business and world beliefs through one channel, and updates valuation through a separate channel. The two channels share inputs but produce typed outputs that are kept distinct in the substrate, in the agent contracts, and in client surfaces.

The motivation is the architecture's most foundational commitment: that mechanism reasoning and valuation reasoning are different cognitive operations, with different evidence requirements, and combining them prematurely produces worse reasoning of both kinds. v3.1 honoured this commitment by having Deep Insight reason about mechanism and the Financial Model reason about valuation, with Threadweave pulling both together at the synthesis layer. v4 extends the separation into the belief layer: a single price move can update the business belief (via the unexpected residual after diagnostic-test elimination) and update the valuation belief (via direct entry into the Financial Model's discount and growth assumptions), and the two updates are recorded as separate `belief_update` rows with separate evidence trails.

This separation is what §41 calls *posterior separation*. It is structural, not stylistic. The substrate has separate `belief` rows of category `business_thesis` and `valuation_thesis`; price evidence flows into both but through architecturally distinct paths; the sealed client surfaces them as separate panels with separate calibration metrics.

What this section does not commit

It does not commit to price being the dominant evidence channel. The expected ratio of update volume across mechanism, tail, and price is roughly 6:1:3 by count at launch, with mechanism updates carrying greater individual weight. It does not commit to any particular calibration of likelihood ratios at launch; calibration is a phase-plan output, with §42 specifying when the architecture is judged to be calibrated. It does not commit to price evidence being admitted on every belief category; categories like `governance_quality` and `competitive_moat` may have null or very wide `expected_price_reaction` bands, in which case price moves of any magnitude will rarely be diagnostic, and updates will overwhelmingly come through the mechanism and tail channels.

Price as Bayesian evidence is a power feature of v4, but the architecture's discipline around it is what makes it safe. The decomposition, the diagnostic tests, the band translation, and the channel separation are not optional refinements; they are the conditions under which admitting price as evidence does not corrupt the rest of the system.

§41 Posterior separation and the falsifier ladder

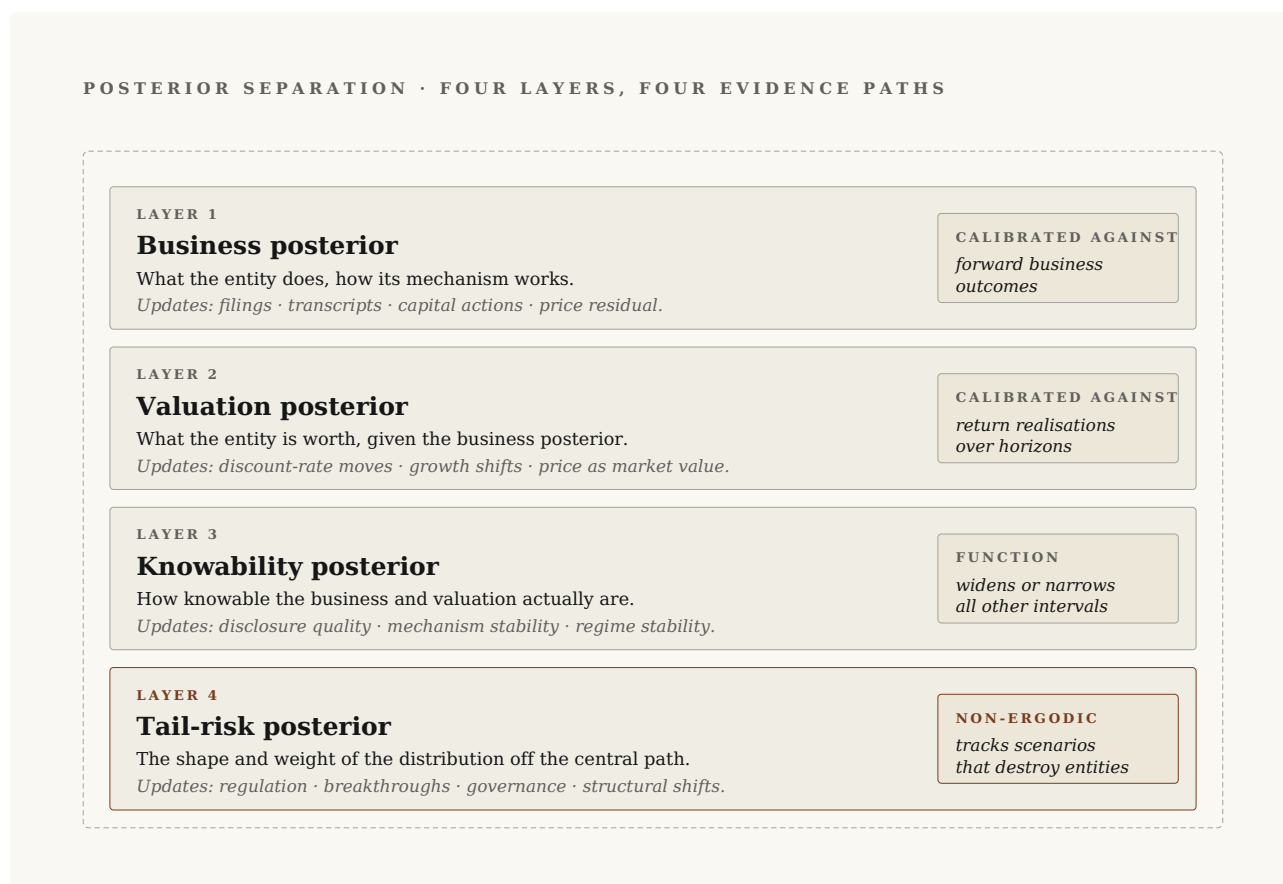


FIGURE V2 · THE FOUR POSTERIOR LAYERS OF THE V4 BELIEF STRUCTURE. EACH LAYER HAS ITS OWN EVIDENCE REQUIREMENTS, ITS OWN SCORING DISCIPLINE, AND ITS OWN CALIBRATION TARGET. SYNTHESIS HAPPENS AT THE THREADWEAVE LAYER; THE UNDERLYING ARITHMETIC STAYS SEPARABLE.

The four posterior layers

A belief about an entity, in the v4 architecture, is not a single object. It is a composition of four distinguishable posterior layers, each with its own evidence requirements, its own scoring discipline, and its own surface in the sealed client. The four layers are:

Business posterior. The architecture's view of the underlying business: what it does, how it makes money, what its competitive position is, how durable its mechanism is, what the management's revealed pattern is. The business posterior is updated primarily by mechanism evidence (filings, transcripts, product disclosures, capital allocation

actions) and secondarily by the price-update channel after `expected_price_reaction` decomposition. Its calibration is measured against forward business outcomes (revenue, margin, market position, capital deployment) over horizons matched to the belief categories.

Valuation posterior. The architecture's view of what the business is worth, conditional on the business posterior. The valuation posterior is updated primarily by changes in valuation inputs (discount rates, growth assumptions, terminal multiples) which themselves are updated by macro evidence and by the business posterior shifts. Price moves enter the valuation posterior directly, not as Bayesian evidence on what the business is worth, but as observed market valuation against which the model's valuation is compared. The valuation posterior is calibrated against return realisations over matching horizons.

Knowability posterior. The architecture's view of how knowable the business and valuation are. Some businesses are highly observable (transparent disclosure, well-understood mechanisms, stable regimes); others are deeply unobservable (limited disclosure, novel mechanisms, unstable regimes). The knowability posterior is updated by evidence of observability itself: disclosure quality, mechanism stability, regime stability. It is the layer that makes the architecture's epistemic humility operational; high knowability supports high-confidence positions, low knowability requires wider confidence intervals at every other layer.

Tail-risk posterior. The architecture's view of the distribution of outcomes, particularly the shape and weight of the tails. Most of the business and valuation posteriors operate on central tendencies; the tail-risk posterior operates on what is true off the central path. It is updated by tail-evidence (regulatory action, technological breakthrough, structural shift, governance event) and by Synthetic Futures regime work. It is the layer that holds the architecture's non-ergodic commitments: scenarios that could destroy the entity are tracked separately from scenarios that adjust expected outcomes.

Why four layers

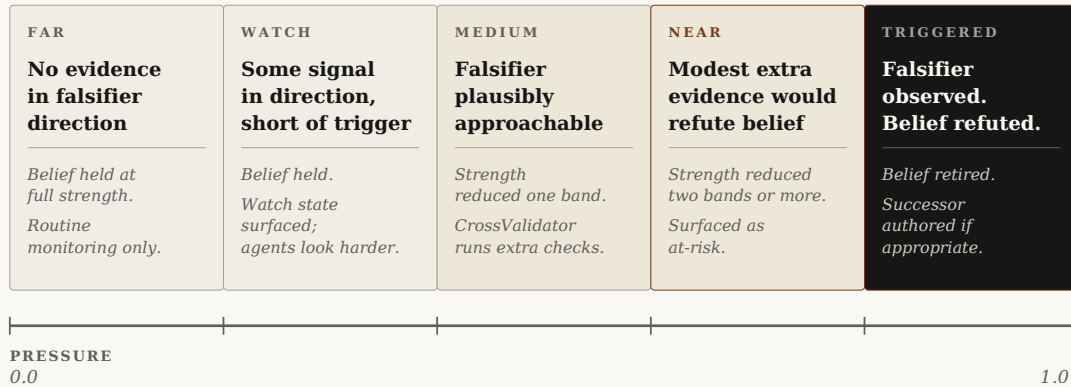
The four-layer decomposition is not arbitrary. It corresponds to four distinguishable kinds of question that the architecture must answer separately to answer well:

1. *What is the business and how does its mechanism work?* (business)
2. *What is the business worth, given the mechanism?* (valuation)
3. *How well can we know either of the above?* (knowability)
4. *What lies beyond our central case?* (tail-risk)

A v3.1 view that bundled these into a single confidence-weighted opinion lost information that operators repeatedly asked for. Several v3.1 review cases in pilot operation showed analysts asking, in effect, "I see the system thinks the business is excellent and the price is high; is the system uncertain about the business, or about the valuation, or about regimes?" The bundled view could not answer; the four-layer decomposition can.

The decomposition also makes belief-update arithmetic clean. Mechanism evidence updates the business posterior without necessarily updating the valuation posterior (the business may be confirmed at the same expected level it was already at). A regime shift in interest rates updates the valuation posterior without necessarily updating the business posterior. A regulatory action may update the tail-risk posterior dramatically while leaving the business posterior unchanged. Each layer's update is independently auditable; the composition into a synthesis view happens at the Threadweave layer (§13), with the layers visible separately to the operator throughout.

THE FALSIFIER LADDER · FIVE STATES, ONE CONTINUOUS PRESSURE METRIC



UNDERLYING METRIC

The five states are a discretisation of a continuous pressure metric on [0, 1] used by the scoring methodology.

FIGURE V3 · THE FALSIFIER LADDER, FIVE STATES FROM FAR TO TRIGGERED, MAPPED AGAINST A CONTINUOUS PRESSURE METRIC. STATE TRANSITIONS ARE THEMSELVES BELIEF_UPDATE EVENTS; THE OPERATOR SURFACE SHOWS THE DISCRETE STATE, THE CALIBRATION DISCIPLINE USES THE CONTINUOUS VALUE.

The falsifier ladder

Every belief in the architecture carries a typed `primary_falsifier`: a description, in operational terms, of the evidence that would refute the belief if it were observed. The falsifier is authored at belief creation, is enforced as non-empty (Gate 2 in the autonomy commitment, §42), and is updated as the belief is updated. It is what makes belief revision concrete rather than rhetorical.

But a falsifier alone does not tell the operator how close to triggering it currently is. The architecture answers this with the falsifier ladder: a five-state classification of the current relationship between observable reality and the belief's falsifier.

State	Meaning	Operational behaviour
Far	No current evidence in the direction of the falsifier; the falsifier is structurally distant from observable reality.	Belief is held with full confidence per its current strength; routine monitoring only.
Watch	Some evidence in the direction of the falsifier exists, but well short of triggering.	Belief is held, but the watch state is surfaced; agents are tasked to look for additional evidence.
Medium	Evidence has accumulated to the point where the falsifier is plausibly approachable; the belief is under pressure.	Belief strength is reduced one band; surface annotations make the pressure explicit; CrossValidator runs additional checks.
Near	The falsifier is close to triggering; only modest additional evidence would refute the belief.	Belief strength is reduced two bands or more; the belief is flagged in client surfaces as at-risk; Synthetic Futures regime work is invoked to model the near-trigger world.

State	Meaning	Operational behaviour
Triggered	The falsifier has been observed; the belief is refuted.	The belief is retired; a successor belief is authored if appropriate; the retirement and any successor are recorded with full provenance.

The falsifier-ladder state is computed on every relevant evidence arrival and is itself a tracked field on the belief. State transitions are themselves `belief_update` events; the audit trail therefore preserves not just what the belief was, but how close to falsification it stood at every point in its history.

The pressure metric

Below the five-state ladder, the architecture maintains a continuous pressure metric: a real number on [0, 1] representing the agents' assessed proximity to falsification. The five-state ladder is a discretisation of this metric for operator surfaces; the underlying continuous value drives the scoring methodology's calibration of falsifier accuracy. A belief that spent 80 percent of its life at pressure 0.1 and then jumped to 0.95 before triggering is a different kind of belief from one whose pressure rose smoothly from 0.1 to 0.95 over the same period; the calibration methodology distinguishes them.

The pressure metric is internal. It is not surfaced to operators in raw form, for the same reason raw likelihood ratios are not: false precision. Operators see the five-state ladder and the band-translated belief strength; the continuous pressure metric is for the architecture's own calibration discipline.

Linking falsifier states to update authoring

The falsifier ladder is not just a status indicator; it is a control surface for the belief-update agent. Updates against beliefs in the **Far** state require unusually strong evidence to overcome the prior; updates against beliefs in the **Watch** or **Medium** states are weighted normally; updates against beliefs in the **Near** state are weighted with elevated sensitivity, because the belief is already under pressure and small additional evidence may be enough to trigger.

This is technically just an application of Bayesian arithmetic with a non-uniform prior strength, but expressing it through the ladder makes the behaviour legible: an operator can understand why a small piece of evidence produced a large belief shift if they can see that the belief was already in a Near state, and equally can understand why a large piece of evidence produced a small belief shift if the belief was Far and the prior was strong.

Posterior separation in practice: a worked composition

Consider an entity for which the architecture holds:

- Business posterior: `growth_durability = high`, falsifier = "two consecutive quarters of revenue contraction with no identified cyclical or one-off cause"; ladder state = Watch (one quarter of unusually weak revenue has occurred).
- Valuation posterior: `valuation_band = mid-range`, falsifier = "valuation multiples expand 50 percent from current levels without commensurate business posterior strengthening"; ladder state = Far.
- Knowability posterior: `knowability = high`, falsifier = "a material disclosure quality decline (restated financials, auditor change with concerns, governance-flagged management departure)"; ladder state = Far.
- Tail-risk posterior: `tail_risk_weight = moderate`, falsifier = "no tail-relevant events have materialised in 24 months"; ladder state = Watch (a regulatory inquiry was opened in the prior quarter and remains pending).

A subsequent quarter shows: revenue growth resumes (above-trend); price rises 15 percent on the print; the regulatory inquiry is closed without action.

The four updates that follow are:

- Business posterior: the revenue rebound moves the falsifier ladder from Watch back to Far; the belief strength increases one band; the update is mechanism-channel, scored by the Evaluator under the canonical methodology.
- Valuation posterior: the price move is decomposed; the `expected_price_reaction` band for a beat of this magnitude was 8 to 12 percent up, so the unexpected residual is small (3 to 7 percent excess); the valuation posterior is sharpened slightly toward “mid-range, upper end” but does not change band; the update is recorded with explicit decomposition.
- Knowability posterior: no relevant evidence; no update; the belief retains its strength, age increments.
- Tail-risk posterior: the regulatory closure moves the ladder from Watch back to Far; the belief strength is reduced (lower tail risk); the update is tail-channel, scored under the long-horizon methodology.

Four typed updates, four separate audit trails, four separate calibration tracks. The synthesis at the Threadweave layer composes them into a single operator-facing view, but the underlying operations remain separable, auditable, and individually scorable. This separability is the architectural commitment that posterior separation makes concrete.

Posterior separation is the v4 architecture’s most important conceptual commitment. It says: bundling business, valuation, knowability, and tail-risk into a single confidence-weighted opinion is not just a UI choice; it is an epistemic mistake that produces worse calibration and less useful operator surfaces. Keeping the layers separable, with disciplined arithmetic across them, is what makes the belief layer worth building.

§42 Phase plan, migration, and the autonomy commitment



FIGURE V4 · THE SIX-PHASE PLAN. EACH PHASE HAS EXPLICIT ENTRY AND EXIT GATES. PROGRESS IS GATED CONTINUOUSLY, NOT JUST AT PHASE BOUNDARIES; PHASE REGRESSION PAUSES ANY CHANNEL WHOSE AUTONOMOUS METRICS DEGRADE.

The plan, in six phases

Moving from v3.1 substrate to full v4 operation is a multi-phase undertaking. Each phase has explicit entry conditions, explicit exit conditions, and explicit autonomy gates. No phase advances on schedule pressure; each advances only when its measured gates pass. The phases are:

Phase 0 · Substrate extension (months 0 to 3). Extend the v3.1 substrate with the five new tables (`belief` , `belief_update` , `alternative_explanation_set` , `price_signal` , `update_case`) per the schemas in §21. Migrate existing v3.1 beliefs (which were implicit in agent emissions and thread synthesis) into explicit `belief` rows, authored retrospectively from the existing canonical reasoning trace. No new belief-update behaviour is enabled. The agent set continues to operate as in v3.1; the substrate is extended without behavioural change.

Entry condition: v3.1 substrate operating per its commitments, scoring methodology calibrated. *Exit condition:* 100 percent of currently-held v3.1 implicit beliefs migrated to explicit `belief` rows with non-empty `primary_falsifier` and non-empty `expected_evidence` ; migration audited and signed off by the autonomous validation pipeline (the same gate batteries that govern live belief authoring, applied retrospectively).

Phase 1 • Mechanism updates only (months 3 to 6). Enable belief-update authoring for the mechanism channel only. Deep Insight emissions that bear on existing beliefs produce `belief_update` rows; the Evaluator scores them; the business posterior begins to evolve via update arithmetic rather than via fresh emission. Price evidence and tail evidence are observed but do not yet author updates.

Entry condition: Phase 0 exit conditions met; mechanism-channel update agent ensemble achieves agreement rate ≥ 0.75 on a 200-case canonical replay set. *Exit condition:* 1000 mechanism `belief_update` rows authored autonomously; replay-reproduction rate (re-running the agent ensemble on the same evidence produces the same update direction within one band) ≥ 0.95 ; rejection-and-reroute stabilisation (the proportion of update attempts rejected by gates is stable and not declining due to gate evasion) confirmed.

Phase 2 • Tail updates added (months 6 to 9). Enable belief-update authoring for the tail channel via Synthetic Futures. Regime shifts and tail-relevant events produce `belief_update` rows on the tail-risk posterior. The mechanism channel continues operating from Phase 1.

Entry condition: Phase 1 exit conditions met; tail-channel update agent achieves agreement rate ≥ 0.75 on a 100-case canonical replay set (tail evidence is rarer; calibration set is smaller proportionally). *Exit condition:* 200 tail `belief_update` rows authored autonomously; replay-reproduction rate ≥ 0.95 ; diagnosticity calibration error < 0.20 (i.e., the difference between asserted likelihood ratio and observed posterior shift, averaged across updates, is under 20 percent).

Phase 3 • Price evidence enabled, single channel (months 9 to 12). Enable the price-update agent to author updates against business posteriors only, after `expected_price_reaction` decomposition and the diagnostic test elimination battery. Tail and valuation posteriors are not yet exposed to price evidence in the update channel.

Entry condition: Phase 2 exit conditions met; `expected_price_reaction` bands authored on at least 80 percent of business posteriors in the launch universe; diagnostic test battery validated against a 500-case historical price-move set with false-positive rate < 0.10 and false-negative rate < 0.20 . *Exit condition:* 500 price `belief_update` rows authored autonomously against business posteriors; probability-mass-shift rate (the proportion of admitted price updates that produce non-trivial posterior movement) ≥ 60 percent (low rates indicate the price channel is capturing noise; high rates above 90 percent indicate insufficient diagnostic discipline).

Phase 4 • Price evidence on valuation and tail-risk posteriors (months 12 to 15). Extend price-update authoring to valuation posteriors (the channel where price moves enter as direct evidence on what the market thinks a business is worth) and to tail-risk posteriors (where unusual price action with regime correlation may carry tail evidence). Posterior separation per §41 is fully operational.

Entry condition: Phase 3 exit conditions met; valuation posteriors carry explicit valuation likelihood functions for at least 80 percent of the launch universe; tail-risk posteriors carry explicit price-correlation profiles for at least 50 percent. *Exit condition:* 200 valuation-channel and 100 tail-channel price `belief_update` rows authored; cross-channel calibration coherence verified (a single price move that produces updates across multiple posteriors does not double-count evidence).

Phase 5 • Full operation (months 15 onward). All channels (mechanism, tail, price-on-business, price-on-valuation, price-on-tail) are operating. Calibration tracks are accumulating per §15. The architecture is in steady-state v4 operation. The 2-to-3-year corpus accumulation of §35 begins from Phase 5 entry.

Entry condition: Phase 4 exit conditions met. *Exit condition:* this phase has no exit; it is the architecture's operating state.

Migration of existing v3.1 beliefs

The Phase 0 migration is the architecture's most operationally delicate step. v3.1 held beliefs implicitly: an agent emission stating "we believe the company's growth is durable" was a belief, but it was not stored as a typed belief row. Migrating these to explicit `belief` rows requires retrospective authoring of:

- The `belief_id` (newly assigned).
- The `category` (one of the canonical categories, conservatively narrow at launch).
- The `subject_entity` (already known).
- The `strength_band` (translated from the v3.1 confidence vector via the calibration table).
- The `primary_falsifier` (authored by the same agent that emitted the original view, applying the v4 contract that no belief is admitted without a non-empty falsifier).
- The `expected_evidence` (authored similarly).
- The `expected_price_reaction` band (where applicable; some beliefs will have null bands, in which case the price channel does not apply to them).
- The provenance trail back to the v3.1 emissions and threads from which the belief was inferred.

This work is performed by the same agent ensemble that will author live beliefs in Phase 1 onward, applied retrospectively. The autonomy gates apply: ensemble agreement, non-empty falsifier, non-empty expected evidence. Retrospective beliefs that fail gates are not admitted; the v3.1 implicit view they came from is logged but does not migrate.

The migration is therefore lossy by design. The architecture does not commit to preserving every v3.1 implicit view as a v4 belief. It commits to preserving every v3.1 implicit view that survives the v4 belief contract. Views that v3.1 expressed loosely without fitting the typed contract are dropped; this is a feature, not a bug, because it means the v4 belief corpus starts with structurally clean rows rather than with grandfathered exceptions.

The autonomy commitment, defended

A central commitment of v4, stated in §15 and recurring throughout this document, is that belief authoring, scoring, and migration validation are fully autonomous. There is no human review queue for new beliefs, no human approval step for belief retirements, no human sign-off on migration outputs. The architecture operates against four mechanisms that together must justify the decision to forgo human review:

- 1. Ensemble agreement.** A new belief, an update, or a migration row is admitted only if a multi-agent ensemble agrees on it. The threshold is configurable per channel but is at least 0.75 across the ensemble at launch. Disagreement across ensemble members produces ensemble-flagged outputs that route to a deeper review pipeline (still autonomous: a more conservative ensemble with stricter gates), not to human queues.
- 2. Non-empty primary falsifier.** Every belief and every update must declare what would refute it. This is gate-enforced; an emission with empty falsifier is rejected at the substrate level, not at a review layer. The gate is deterministic and not subject to ensemble override.
- 3. Non-empty expected evidence.** Every belief must declare what evidence would support or refine it going forward. Same gate enforcement, same determinism, same non-overridability.

4. Replay reproduction. The architecture continuously samples its own emitted beliefs and updates, replays them through the agent ensemble on the original evidence, and measures whether the same update is produced. The target reproduction rate is ≥ 0.95 . Reproduction failures trigger investigation (also autonomous: the diagnostic agent examines what changed); reproduction rate trends are continuously monitored, and a sustained drop below 0.95 is a Phase regression event that pauses the channel until calibration is restored.

The defence of this commitment is structural, not philosophical. The core motivation is corpus integrity: §35 commits to a corpus of update trajectories that is purely system-authored and system-scored. A human review step in this loop contaminates that property. It produces a corpus that reflects the joint behaviour of the system and the reviewers, which is not reproducible (reviewers retire, change their standards, and behave inconsistently across cases) and not scientifically valuable in the way the architecture's R&D directions require.

A secondary motivation is operational: human review at the volume v4 will operate at (thousands of belief updates per quarter at full scale) is not feasible, and partial review (sampling a fraction of updates for human inspection) creates a two-tier corpus that is worse than either fully autonomous or fully reviewed.

A tertiary motivation is calibration: human reviewers are not better calibrated than well-disciplined agent ensembles for this kind of structured judgement. The v3.1 scoring methodology already operates without human-in-the-loop scoring at the canonical layer, and the v4 belief layer extends the same commitment to belief authoring and update.

The commitment is testable. The four gates produce continuous metrics (ensemble agreement rate, falsifier-empty rejection rate, expected-evidence-empty rejection rate, replay reproduction rate). If any of these degrades below its commitment threshold for a sustained period, the architecture's autonomy commitment is failing on its own terms, and the channel is paused. The commitment is not "we trust the agents"; it is "we have built the gates that justify operating without human review, and we monitor them continuously."

What could go wrong, and the architecture's response

Several failure modes deserve explicit acknowledgement:

Ensemble collusion. If ensemble members are trained on overlapping data and share systematic biases, they may agree on wrong outputs. The architecture's response is deliberate ensemble diversity: members are sourced from different model families, different training cuts, and different prompt traditions. The ensemble is rebuilt periodically, with diversity audited.

Gate gaming. If agents learn to produce outputs that pass the gates without genuinely satisfying their intent (a syntactically-valid but contentless `primary_falsifier`, for example), the gates lose their force. The architecture's response is content checks on gate outputs: the falsifier and expected-evidence fields are themselves structured (typed, with required sub-fields like observability, time-bound, source) and validated against the structure, not just for non-emptiness.

Calibration drift. Even with stable gates, the agent ensemble's calibration may drift over time as model versions change, prompt versions iterate, and the substrate evolves. The architecture's response is the §15 calibration discipline: continuous measurement against forward outcomes, with alerts on drift, and prompt-version provenance ensuring no silent recalibration.

Volume runaway. If the architecture authors too many beliefs and updates, the corpus becomes unwieldy and individual rows lose meaning. The architecture's response is per-category retirement windows (open question §05 in §43): beliefs that have not been updated within their category-specific window are auto-retired. This is one of the architecture's open commitments rather than a fully-specified mechanism.

The phase plan is the v4 architecture's commitment to incremental, gated rollout. The autonomy commitment is the architecture's commitment to building gates strong enough to operate without human review. Both commitments are testable. Both are pause-able if the gates degrade. Neither is "we trust the system"; both are "we have built mechanisms that justify our operating choices, and we monitor them."

§43 Decision memo

What v4 commits

The v4 architecture commits to:

- A typed belief layer (`belief` , `belief_update` , `alternative_explanation_set` , `price_signal` , `update_case`) integrated into the v3.1 substrate.
- A four-layer posterior decomposition (business, valuation, knowability, tail-risk) with separable update arithmetic per §41.
- A falsifier ladder with five states and a continuous pressure metric per §41.
- Three update channels (mechanism, tail, price) with the diagnostic test elimination rule and `expected_price_reaction` decomposition per §40.
- A six-phase rollout plan with autonomous pass conditions per §42.
- A four-gate autonomy commitment (ensemble agreement, non-empty falsifier, non-empty expected evidence, replay reproduction) per §42.
- Three additional scoring tracks (belief strength calibration, falsifier accuracy, update diagnosticity) per §15.
- A sealed-client surface that exposes belief and update structure within the seal per §29.
- An optional, opt-in abstracted contribution path that preserves the seal per §29.
- A corpus of update trajectories sufficient to support R&D directions over a 2-to-3-year horizon per §35.

What v4 does not commit

It does not commit to:

- A specific ensemble size, model mix, or prompt configuration; these are operational details subject to ongoing tuning per the §17 runtime contracts.
- Specific likelihood-ratio calibration values; these emerge from Phase plan exit conditions and may shift as the corpus grows.
- Public Actor Trace, causal modelling, agent simulation, or future-state forecasting as named features; these are R&D directions per Part VI, not roadmap items.
- A specific timeline beyond the phase plan's month markers, which themselves are gated rather than scheduled.
- A specific revenue or coverage outcome; the architecture is built to support a coverage strategy, not to guarantee one.

What is at risk

Six failure modes are explicitly tracked, and the architecture commits to avoiding each:

1. **Summaries-as-updates.** The architecture must not let agents emit prose summaries dressed as belief updates. Every `belief_update` row must include explicit evidence reference, explicit prior strength, explicit posterior strength, and explicit likelihood ratio. The substrate gate enforces non-emptiness; the §15 scoring discipline enforces meaningfulness.
2. **False precision.** Internal likelihood ratios must not leak into client surfaces. The five-band external scale is the only confidence representation operators see; the continuous pressure metric is internal. Surface audit per §28 enforces this.
3. **Price-as-truth.** Price evidence must always be decomposed against `expected_price_reaction`; the unexpected residual is the only fresh Bayesian weight on business beliefs. Architectural separation per §40 prevents the system from drifting into a market-tracker.
4. **Seal erosion.** The seal must not weaken as the belief layer matures. The §29 commitments hold without modification; the new belief and update tables inherit the seal contracts; the opt-in abstracted contribution path is the only growth channel that crosses the seal boundary, and it does so with explicit per-tenant authorisation.
5. **RL overclaiming.** The architecture does not claim that the corpus of update trajectories enables specific reinforcement learning outcomes. It claims that the corpus is structurally suitable for R&D investigation, and that the structural properties (typed updates, autonomous authoring, calibration trails) are necessary conditions. Sufficiency is open. Per Part VI, the R&D directions are conditional on preconditions that include corpus volume and calibration credibility, not just structure.
6. **Belief proliferation.** The architecture must not author beliefs faster than it retires or updates them. The per-category retirement window (open question, see below) is the planned mechanism; until it ships, manual category audits at quarterly cadence are the interim discipline.

Open questions

Seven questions remain open and are committed to be resolved before or during the phase plan:

1. *Ensemble agreement threshold.* Launch threshold is 0.75. Whether 0.75 is the right number, or whether it should differ by channel, is an open calibration question. Resolution: empirical, via Phase 1 and Phase 2 operation.
2. *Confidence vector vs scalar number.* Whether internal likelihood ratios should be represented as scalars or as vectors across multiple evidence dimensions is an open structural question. Resolution: deferred to Phase 3 calibration work.
3. *Cross-entity belief linking.* Whether beliefs about one entity should be allowed to update beliefs about a related entity (a competitor, a supplier, a regulator) automatically, with what arithmetic, is open. Resolution: deferred to Phase 4 or later.
4. *Customer-private vs canonical link visibility.* Whether tenants in the opt-in abstracted contribution path can see the canonical universe's update structure (not contents) is open. The default is no; whether a structurally-revealing opt-in could be useful is open. Resolution: deferred until at least one tenant opts in.

5. *Falsifier distance: continuous vs five-state.* The five-state falsifier ladder is a discretisation; the continuous pressure metric is the underlying value. Whether to expose the continuous metric in any operator surface is open. Resolution: empirical, via operator feedback in Phase 1 onward.
6. *Update_case privacy gradient.* The `update_case` table holds canonical update trajectories that are intended for R&D corpus access. Whether different research uses warrant different privacy gradients (full access for internal R&D, abstracted access for external collaborations, etc.) is open. Resolution: deferred until R&D collaborations are concrete.
7. *Per-category auto-retirement window.* Each belief category will have an auto-retirement window (a duration after which a belief that has not been updated is automatically retired). The windows have not been calibrated. Resolution: empirical, via Phase 5 operation; until then, manual quarterly audits substitute.

The risks, restated as commitments

The architecture's response to its risks is to commit to monitoring and pause-ability. It does not commit to never failing; it commits to detecting failures fast and pausing the channels that fail. The phase plan's autonomous pass conditions are not just entry gates; they are continuously monitored, and a sustained breach of any of them in any phase is treated as a phase regression. Phase regression pauses the channel that breached, returns it to its previous phase's gate set, and requires the gates to repass before progress resumes.

This is the architecture's most important operational commitment: that progress is gated continuously, not just at phase boundaries; that pause is cheap and rapid; that the cost of pausing a channel that is failing its gates is much lower than the cost of letting a degrading channel continue to author corpus.

The v4 architecture is a substantial extension of v3.1, but it is also a continuation of v3.1's discipline: typed contracts, autonomous operation, separable layers, gate-enforced commitments, calibration that compounds. The Bayesian belief layer is not a paradigm shift; it is the explicit form of what v3.1 was already doing implicitly, structured so the explicit form can be measured, calibrated, and used as research substrate. The four-gate autonomy commitment is the architecture's way of saying: we know what would have to be true for this to be safe, we have built the mechanisms to keep those things true, and we will pause anything that drifts.

End of v4.0 architecture document.

This document represents a commitment by StahlTrace Lda to build, operate, and discipline a Bayesian belief-update layer on top of the v3.1 architecture. The commitments here are testable and pause-able. The directions described are conditional on the preconditions in §39. The scope at launch is described in §24; the open questions in §25 and §43 are committed to be resolved on the phase plan's cadence.

Confidential. © Ronald Moesker / StahlTrace Lda · Architecture v4.0.